EKNATHALABS · INTERVIEW PREP

Azure Production — EknathaLabs Interview Guide

Platform, SRE & DevOps interview guide

Questions	**170 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/azure_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What is Microsoft Entra ID (Azure AD) — how does RBAC work in Azure?

IAM & EntraID

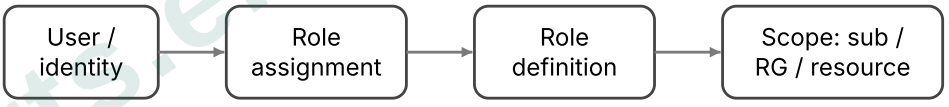
🎙 Spoken answer · ~86 s

Microsoft Entra ID, formerly Azure AD, is Azure's cloud identity service. It manages users, groups, applications, and service principals, and handles authentication through modern protocols like OAuth 2.0, OpenID Connect, and SAML, plus features such as single sign-on, multi-factor authentication, Conditional Access, and identity governance. Azure RBAC is the authorisation system for Azure resources, separate from Entra directory roles. A role assignment has three parts: a security principal, a role definition, and a scope. The principal is a user, group, service principal, or managed identity. The role definition is a set of allowed actions, such as Reader, Contributor, Owner, or a custom role, plus not-actions and data actions. The scope is where it applies: management group, subscription, resource group, or an individual resource, and permissions inherit downward. Assignments are additive: a principal's effective access is the union of all assignments, with deny assignments only from features like Blueprints or managed applications. Best practices: assign roles to groups rather than individuals, use the narrowest scope and least privilege, prefer built-in roles, use Privileged Identity Management for just-in-time elevation of powerful roles, avoid standing Owner access, use managed identities for workloads, and review access regularly. Remember Entra roles, like Global Administrator, control the directory, whereas Azure RBAC controls resources.

Likely follow-ups

- How do Entra roles differ from Azure RBAC roles?
- Why assign roles to groups instead of individuals?

🗺 Entra ID and Azure RBAC




```
# Assign a role to a user/SP on a resource group:
```

```
az role assignment create --assignee user@company.com \
```

```
  --role "Contributor" --scope /subscriptions/SUB/resourceGroups/prod-rg
```

```
# List role assignments:
```

```
az role assignment list --scope /subscriptions/SUB --output table
```

```
# Create custom role:
```

```
az role definition create --role-definition file://custom-role.json
```

```
# Workload Identity for AKS pods (no secrets):
```

certs.eknathalabs.com

```
az aks update --resource-group rg --name aks --enable-workload-identity --enable-oidc-issuer
```

Azure RBAC: 4 fundamental roles (Owner, Contributor, Reader, User Access Administrator) plus 100+ built-in roles. Scope hierarchy: Management Group → Subscription → Resource Group → Resource. Assign at the highest appropriate scope — a Contributor on a Resource Group can manage everything in it. Service Principals (app identities) and Managed Identities replace username/password for service authentication. Workload Identity: AKS pod federated with Entra ID service principal — no credential files, auto-rotating tokens.

📚 **Official docs**

- Kubernetes: RBAC
- Microsoft Entra ID: What is Microsoft Entra ID?

Q2 What is a Managed Identity in Azure — how does it differ from a Service Principal?

IAM & EntraID

🎙 **Spoken answer · ~89 s**

Both are identities that applications use to authenticate to Azure services, but they differ in how credentials are managed. A service principal is the identity of an application in a tenant, created through an app registration. It authenticates with a secret or certificate, or federated credentials, which you must create, store, rotate, and protect; secrets expire and can leak. A managed identity is a special service principal whose credentials are fully managed by Azure: you never see or handle a secret. There are two types. System-assigned identities are tied to the lifecycle of a single resource, such as a VM or App Service, and deleted with it. User-assigned identities are standalone resources that can be attached to multiple resources and persist independently, useful for sharing permissions across a set of resources or pre-provisioning access. The workload obtains tokens from the instance metadata service or platform endpoint, and uses them to access Key Vault, Storage, SQL, and other Entra-authenticated services. Prefer managed identities whenever the workload runs on Azure, since they remove credential management risk. Use service principals when the workload runs outside Azure, such as CI systems, but prefer workload identity federation with OIDC over long-lived secrets. In AKS, use workload identity to give pods their own identity. Always grant least privilege through RBAC.

Likely follow-ups

- When must you use a service principal instead of a managed identity?
- What is workload identity federation?

```
# System-assigned Managed Identity (tied to resource lifecycle):
```

```
az vm identity assign --resource-group rg --name myvm  # auto-enabled for VM
```

```
# User-assigned Managed Identity (reusable across resources):
```

certs.eknathalabs.com

```
az identity create --resource-group rg --name myapp-identity
```

```
az vm identity assign --resource-group rg --name myvm --identities myapp-identity
```

```
# Grant identity access to Key Vault:
```

```
az keyvault set-policy --name myvault --object-id <identity-object-id> \
```

```
  --secret-permissions get list
```

```
# In application code: no credentials needed
```

```
credential = DefaultAzureCredential()   # uses Managed Identity when on Azure
```

Managed Identity: Azure manages the credential lifecycle — no secrets to rotate, no credential leaks. System-assigned: created with the resource, deleted when the resource is deleted, 1:1 relationship. User-assigned: independent lifecycle, can be assigned to multiple resources — use when multiple services share the same identity. Service Principal with certificate/secret: use when Managed Identity is not available (cross-tenant, CI/CD outside Azure). For AKS: Workload Identity is the Kubernetes-aware equivalent.

> 📚 **Official docs**
>
> - Azure: Managed identities

certs.eknathalabs.com

Q3 What is a Virtual Network (VNet) — how do you design one for production AKS?

Networking

🎙 Spoken answer · ~89 s

A Virtual Network is Azure's private network boundary, with an address space divided into subnets, where resources get private IPs; it connects to other networks through peering, VPN, or ExpressRoute. Designing for production AKS starts with address planning: choose non-overlapping ranges across VNets and on-premises, and size subnets for growth. With the Azure CNI, pods get IPs from the VNet subnet, so the node subnet must be large enough for nodes times pods per node; Azure CNI Overlay uses a separate pod CIDR, saving VNet addresses, which is usually the better default for large clusters. Segment into subnets: one for AKS nodes, separate subnets for private endpoints, application gateway or ingress, Azure Firewall, and bastion, each with network security groups. Use a hub-and-spoke topology with the cluster in a spoke, and a central firewall for egress control via user-defined routes. Decide on a private cluster, so the API server has a private endpoint, and plan DNS for it. Use private endpoints for dependencies like Key Vault, ACR, and storage. Add network policies, using Azure NPM, Calico, or Cilium, for pod-level isolation. Plan egress using NAT gateway or firewall for predictable outbound IPs. Enable NSG flow logs and monitor. Define everything as code, and keep subnet capacity for upgrades, since surge nodes need extra IPs.

Likely follow-ups

- Why is Azure CNI Overlay often preferred for large clusters?
- Why leave spare IP capacity in the node subnet?

```
az network vnet create --resource-group rg --name prod-vnet --address-prefix 10.0.0.0/8
```

```
# Subnets:
```

```
az network vnet subnet create --vnet-name prod-vnet --name aks-nodes   --address-prefixes
10.240.0.0/16
```

```
az network vnet subnet create --vnet-name prod-vnet --name aks-pods    --address-prefixes
10.241.0.0/16
```

```
az network vnet subnet create --vnet-name prod-vnet --name ingress      --address-prefixes
10.242.0.0/24
```

```
az network vnet subnet create --vnet-name prod-vnet --name db-subnet    --address-prefixes
10.243.0.0/24
```

```
# NSG on subnet:
```

```
az network nsg create --resource-group rg --name aks-nsg
```

```
az network vnet subnet update --vnet-name prod-vnet --name aks-nodes --network-security-
group aks-nsg
```

certs.eknathalabs.com

AKS uses Azure CNI (recommended for production): each pod gets a real VNet IP, enabling native VNet routing, NSG-level pod traffic control, and direct pod-to-service communication. Plan address space carefully: AKS clusters can have many pod IPs (N nodes × max_pods_per_node). Use Azure CNI Overlay or Cilium for larger clusters where IP exhaustion is a concern. Private AKS: API server on private IP, accessible only from VNet or peered networks.

📚 **Official docs**

- Azure Virtual Network
- Azure Kubernetes Service (AKS)

Q4 What is AKS — what does Azure manage vs what you manage?

Containers & AKS

🎙 **Spoken answer · ~86 s**

AKS is Azure's managed Kubernetes service. Azure manages the control plane: the API server, etcd, scheduler, and controller manager, which are hosted, patched, and made highly available by Microsoft, with the standard tier offering a financially backed uptime SLA and the free tier having no SLA; you don't pay for control plane machines. You manage the data plane and workloads. That includes node pools, their VM sizes, scaling configuration, and the operating system upgrade cadence, though node images can be auto-upgraded and AKS provides node image upgrade channels; Kubernetes version upgrades, which you initiate or automate by channel; networking choices such as CNI, ingress, and network policies; identity and RBAC; security hardening and policy; add-ons and their configuration; storage classes; monitoring; backup; and the applications themselves. The node resource group holds the VMs, disks, and load balancers, and shouldn't be modified directly. Integrations help: Entra ID for authentication, managed identities, Azure Policy, Key Vault CSI driver, Container Insights, Defender for Containers, and Application Gateway. Compared with self-managed Kubernetes, you save the substantial work of operating the control plane, but you still own cluster design, upgrades, security, and operations. Plan regular upgrades because older versions go out of support, and use automatic channels with maintenance windows where suitable.

Likely follow-ups

- What does the node resource group contain?
- What is the difference between free and standard tiers?

🗺 **AKS responsibility**

Azure manages	You manage
Control plane, etcd	Node pools, upgrades
API server SLA	Workloads, networking policy

certs.eknathalabs.com

```
az aks create --resource-group rg --name prod-aks \
  --node-count 3 --node-vm-size Standard_D4s_v5 \
  --enable-managed-identity --network-plugin azure \
  --enable-workload-identity --enable-oidc-issuer \
  --enable-cluster-autoscaler --min-count 3 --max-count 10
az aks get-credentials --resource-group rg --name prod-aks
# Upgrade cluster:
az aks upgrade --resource-group rg --name prod-aks --kubernetes-version 1.30
```

Azure manages: control plane (API server, etcd, scheduler), node OS security patches (auto-upgrade options), control plane SLA (99.95% with AZs). You manage: node pools, workloads, networking within VNet, RBAC for Kubernetes objects. Key AKS add-ons: Azure Monitor (Container Insights), Azure Key Vault CSI driver (mount secrets as volumes), Azure Policy for Kubernetes (OPA Gatekeeper policies), NGINX/AGIC ingress. Upgrade: control plane first, then node pools (az aks nodepool upgrade).

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q5 What is the Application Gateway Ingress Controller (AGIC) — how does it work?

Containers & AKS

🎙 Spoken answer · ~85 s

The Application Gateway Ingress Controller lets AKS use Azure Application Gateway, a layer-seven load balancer with WAF, as the ingress for the cluster. AGIC runs as a pod in the cluster, watching Kubernetes Ingress, Service, and endpoint resources; when you create or change an Ingress, it translates them into Application Gateway configuration, such as listeners, routing rules, backend pools, and health probes, and updates the gateway through Azure Resource Manager. Because pods are reachable directly with Azure CNI, the gateway sends traffic straight to pod IPs, skipping the extra hop through kube-proxy and node ports. Benefits: a managed, highly available ingress outside the cluster that doesn't consume cluster resources, TLS termination, URL routing, autoscaling, and an integrated web application firewall for protection against common attacks. It can be deployed as an AKS add-on, which is managed by Azure, or via Helm. Considerations: updates go through ARM, so propagation can be slower than in-cluster ingress controllers; the gateway needs a dedicated subnet; there are limits on rules and listeners; and the gateway must be able to reach pods, so networking matters. Microsoft is steering new deployments to Application Gateway for Containers, a newer, faster, Gateway API-based option. Choose based on WAF needs, performance, and operational preferences.

Likely follow-ups

- Why does AGIC send traffic directly to pod IPs?
- What is Application Gateway for Containers?

```
az network application-gateway create --resource-group rg --name appgw \
```

```
  --sku Standard_v2 --capacity 2 --vnet-name prod-vnet --subnet ingress
```

```
az aks enable-addons --resource-group rg --name prod-aks \
```

```
  --addons ingress-appgw --appgw-id /subscriptions/.../applicationGateways/appgw
```

```
# Ingress resource uses AGIC annotations:
```

```
kubernetes.io/ingress.class: azure/application-gateway
```

```
appgw.ingress.kubernetes.io/ssl-redirect: "true"
```

AGIC provisions Azure Application Gateway rules from Kubernetes Ingress resources. Application Gateway is an L7 load balancer with WAF, SSL termination, URL routing, and session affinity. AGIC watches Ingress resources and translates them to App Gateway routing rules automatically. Alternative: NGINX ingress controller (simpler, more portable, no Azure dependency). Use AGIC when you need WAF integration, autoscaling (App Gateway v2 scales pods automatically), or Azure-native PaaS integration.

certs.eknathalabs.com

📚 **Official docs**

- Kubernetes: Ingress
- Azure Application Gateway

Q6 What are the storage options in Azure — Blob, Disk, Files, NetApp?

Storage

🎙 **Spoken answer · ~85 s**

Azure offers several storage services for different needs. Blob Storage is object storage for unstructured data: backups, logs, media, data lakes, with hot, cool, cold, and archive tiers, lifecycle management, and immutability policies. With hierarchical namespace it becomes Data Lake Storage Gen2 for analytics. Azure Disks are block storage for VMs and AKS persistent volumes, in types such as Premium SSD, Premium SSD v2, Standard SSD, and Ultra Disk; a disk attaches to one node at a time and is zonal, with zone-redundant options. Use for databases and single-writer workloads. Azure Files provides managed SMB and NFS file shares that can be mounted by many clients, ideal for shared content, lift-and-shift applications, and ReadWriteMany volumes in Kubernetes. Azure NetApp Files is a high-performance managed file service based on NetApp technology, with very low latency and high throughput, for demanding workloads like SAP, HPC, and databases, at a higher price. A rough rule: unstructured scalable data, Blob; a fast disk for one machine, Managed Disks; shared file system, Azure Files; ultra-high-performance enterprise file workloads, NetApp Files. Consider redundancy options like LRS, ZRS, and GRS, encryption with customer-managed keys, private endpoints, and access via managed identities rather than account keys. Match performance and cost to the workload.

Likely follow-ups

- When would you choose Azure NetApp Files over Azure Files?
- What do LRS, ZRS and GRS mean?

certs.eknathalabs.com

```
Azure Blob:   object storage (S3 equivalent), hot/cool/archive tiers
```

```
Azure Disk:   block storage, attached to single VM (EBS equivalent)
```

```
              Premium SSD (P10-P80), Ultra Disk (sub-ms latency)
```

```
Azure Files:  SMB/NFS shared filesystem (EFS equivalent)
```

```
Azure NetApp: enterprise NFS for latency-sensitive workloads
```

```
# AKS persistent volume:
```

```
kubectl get storageclass       # managed-premium, azurefile-csi, etc.
```

```
kubectl apply -f pvc.yaml      # request a managed disk or Azure Files share
```

```
# AKS CSI drivers handle provisioning and attachment
```

AKS storage: managed-premium StorageClass provisions Azure Premium SSD disks (ReadWriteOnce). azurefile-csi provisions Azure Files shares (ReadWriteMany — shared across pods). For databases: Premium SSD or Ultra Disk. For shared config or user uploads: Azure Files. For backup/archives: Blob with lifecycle management. Azure NetApp Files: high-performance NFS for SAP, Oracle, or latency-sensitive Linux workloads — expensive but very fast.

> 📚 **Start here (official docs)**
>
> - Microsoft Azure documentation

certs.eknathalabs.com

Q7 What is Azure SQL Managed Instance — how does it differ from Azure SQL Database?

Databases

🎙 Spoken answer · ~86 s

Both are managed SQL Server offerings, but with different scope and compatibility. Azure SQL Database is a platform-as-a-service database built for new cloud applications: a single database or elastic pool, with the highest degree of abstraction, automatic patching, backups, and high availability, built-in scaling including serverless and Hyperscale tiers, and usage-based pricing. It doesn't support every instance-level feature: no SQL Agent, cross-database queries are limited, and no linked servers in the same way. Azure SQL Managed Instance is a managed full SQL Server instance with near-complete compatibility with on-premises SQL Server: SQL Agent, cross-database queries and transactions, linked servers, CLR, Service Broker, database mail, and instance-level features. It lives inside your virtual network with a private IP, which makes it the natural landing zone for lift-and-shift migrations with minimal application changes, while still being managed. Trade-offs: Managed Instance is more expensive, has longer provisioning and scaling operations, and fewer elasticity options. Choose SQL Database for new applications, microservices, SaaS, and when you want maximum elasticity and lower cost; choose Managed Instance for migrating existing SQL Server workloads that rely on instance-scoped features. Consider also SQL Server on a VM when you need full OS control. Plan high availability with zone redundancy and geo-replication or failover groups.

Likely follow-ups

- Which instance-level features does Managed Instance support that SQL Database lacks?
- When does a VM make sense over either?

```
# Azure SQL Database (PaaS, serverless option):
```

```
az sql db create --resource-group rg --server myserver --name mydb --service-objective S3
```

```
# Azure SQL Managed Instance (full SQL Server engine in VNet):
```

```
az sql mi create --resource-group rg --name mymi --subnet /subscriptions/.../subnets/mi-subnet \
```

```
  --admin-user sqladmin --admin-password <password>
```

```
# Connection string:
```

```
mymi.public.abc123.database.windows.net,3342  # or private endpoint
```

Azure SQL Database: fully managed PaaS, serverless option (scales compute to zero when idle), geo-redundancy. Best for: new applications, microservices, variable load. Azure SQL Managed Instance: 99% SQL Server compatibility, runs in your VNet, supports SQL Server Agent, cross-database queries, CLR, linked servers. Best for: lift-and-shift of on-prem SQL Server workloads. PostgreSQL Flexible Server: fully managed PostgreSQL in your VNet, high availability with zone-redundant standby, read replicas.

certs.eknathalabs.com

📚 **Official docs**

- Azure SQL documentation

Q8 What is Azure Monitor — how do you use it with AKS?

Monitoring

🎙 **Spoken answer · ~88 s**

Azure Monitor is the platform for collecting, analysing, and acting on telemetry. It brings together metrics, stored in a time-series database, and logs, stored in Log Analytics workspaces and queried with Kusto Query Language. For AKS: platform metrics for the cluster come automatically; Container Insights, enabled as an add-on, deploys an agent that collects node and pod performance data and container logs into a Log Analytics workspace, with prebuilt views for clusters, nodes, controllers, and containers; control plane logs, such as API server, audit, and controller manager, are enabled through diagnostic settings and sent to the workspace, storage, or event hub. Azure Managed Prometheus and Managed Grafana add Prometheus-style metrics collection and dashboards. Application Insights provides application performance monitoring, distributed tracing, and dependency maps, via OpenTelemetry-based instrumentation. Alerts: metric alerts, log query alerts, and activity log alerts, tied to action groups that notify or trigger automation. Practical points: choose what to collect, since log ingestion drives cost, using data collection rules to filter namespaces and tables; set retention and use basic logs tier for verbose data; build workbooks and dashboards; and alert on user-impacting symptoms and key cluster health conditions like node not ready, pending pods, and restarts. Use diagnostic settings consistently across resources, and manage the configuration as code.

Likely follow-ups

- How do data collection rules help control Log Analytics cost?
- What does Container Insights collect?

```
# Enable Container Insights on AKS:
```

```
az aks enable-addons --addons monitoring --name prod-aks --resource-group rg \
```

```
  --workspace-resource-id /subscriptions/.../workspaces/myworkspace
```

```
# KQL query in Log Analytics:
```

```
KubeEvents | where Reason == "OOMKilling" | project TimeGenerated, Name, Message
```

```
ContainerLog | where LogEntry contains "ERROR" | summarize count() by
bin(TimeGenerated,5m), ContainerName
```

```
# Metric alert:
```

```
az monitor metrics alert create --name cpu-high --resource $AKS_ID \
```

certs.eknathalabs.com

```
  --condition "avg Percentage CPU > 80" --action $ACTION_GROUP_ID
```

Azure Monitor: metrics (platform + custom), logs (Log Analytics workspace), alerts (metric and log-based), dashboards, Application Insights (APM for application code). Container Insights adds AKS-aware views: node/pod CPU/memory, controller health, container logs. KQL (Kusto Query Language) is the query language for Log Analytics — powerful for aggregation, joining tables, and time-series analysis. For Prometheus-compatible metrics: Azure Monitor Managed Prometheus + Azure Managed Grafana (zero-ops equivalent of kube-prometheus-stack).

📚 **Official docs**

- Azure Monitor
- Azure Kubernetes Service (AKS)

Q9 What is Azure Managed Prometheus and Managed Grafana — how do you set them up?

Monitoring

🎙 **Spoken answer · ~88 s**

Azure Monitor managed service for Prometheus is a fully managed, Prometheus-compatible metrics store, and Azure Managed Grafana is a managed Grafana instance for dashboards. Together they give the Prometheus and Grafana ecosystem without running and scaling the servers yourself. Setup for AKS: create an Azure Monitor workspace, which stores the Prometheus metrics; create an Azure Managed Grafana instance; then enable the managed Prometheus add-on on the cluster, either from the portal, CLI, or infrastructure as code, linking the cluster to the Azure Monitor workspace and the Grafana instance. This deploys a metrics agent that scrapes default targets such as the kubelet, cAdvisor, kube-state-metrics, and node exporter. Add custom scrape configs or pod monitors, which are compatible with the Prometheus operator custom resources, to collect application metrics. Grafana comes with preconfigured dashboards for Kubernetes, and you can import community dashboards, and connect the Azure Monitor workspace as a Prometheus data source, authenticated through managed identity. Recording rules and alert rules can be defined as Prometheus rule groups in Azure, with alerts sent to action groups. Control cost by managing cardinality and which metrics are scraped, and set retention. Grant access to Grafana through Entra ID with role assignments. Manage everything as code, and combine with logs and traces for full observability.

Likely follow-ups

- Why watch metric cardinality with managed Prometheus?
- How do application teams add their own metrics?

```
# Enable Managed Prometheus on AKS:
```

```
az aks update --resource-group rg --name prod-aks \
```

```
  --enable-azure-monitor-metrics \
```

cors.eknathalabs.com

```
  --azure-monitor-workspace-resource-id /subscriptions/.../accounts/mypromws
```

```
# Create Managed Grafana:
```

```
az grafana create --name mygrafana --resource-group rg
```

```
# Link Prometheus workspace to Grafana:
```

```
az grafana data-source create --name mygrafana --definition file://prometheus-
datasource.json
```

```
# Same Prometheus PromQL, same Grafana dashboards, zero infra to manage
```

Azure Managed Prometheus: Prometheus-compatible (same SDKs, ServiceMonitors, PromQL, recording rules) but fully managed — no StatefulSets, no storage management. Stores data in Azure Monitor Workspace. Azure Managed Grafana: managed Grafana service, auto-connected to Azure data sources (Azure Monitor, Managed Prometheus, Log Analytics). For AKS teams, this is the zero-ops path to full observability — equivalent to GCP Managed Prometheus on the Azure side.

> 📚 **Official docs**
>
> - Google Cloud Managed Service for Prometheus
> - Prometheus: Overview
> - Grafana documentation

certs.eknathalabs.com

Q10 How does Azure DevOps work — Pipelines, Repos, Artifacts?

CI/CD

🎙 Spoken answer · ~85 s

Azure DevOps is Microsoft's suite for the software delivery lifecycle. Azure Repos offers Git repositories with branch policies, pull requests, required reviewers, build validation, and code owners. Azure Pipelines runs CI/CD, defined as YAML pipelines in the repository, or classic visual pipelines, with stages, jobs, and steps; it uses Microsoft-hosted agents or self-hosted agents, supports templates for reuse, variable groups and secure files, environments with approvals and checks for deployment gates, and service connections to authenticate to Azure and other systems, ideally using workload identity federation. Azure Artifacts hosts packages for NuGet, npm, Maven, Python, and universal packages, with feeds, upstream sources, and retention policies. Azure Boards handles work tracking with backlogs and sprints, and Test Plans handles manual and exploratory testing. A typical flow: a developer opens a pull request, branch policy triggers a validation build with tests and scans, a merge triggers a pipeline that builds, publishes an artefact or image, and deploys through environments to dev, staging, and production, with approvals before production. Good practice: pipelines as code, shared templates, least-privilege service connections, secret handling through Key Vault, protected branches, and tracking deployment metrics. Compared with GitHub Actions, Azure DevOps suits organisations wanting an integrated suite with strong enterprise controls.

Likely follow-ups

- How do environments and approvals gate production deployments?
- What are service connections and how do you secure them?

```
# azure-pipelines.yml:
trigger: [main]
pool: {vmImage: ubuntu-latest}
stages:
- stage: Build
  jobs:
  - job: BuildTest
    steps:
    - task: Docker@2
      inputs: {command: buildAndPush, containerRegistry: my-acr}
- stage: Deploy
```

certs.eknathalabs.com

```
  environment: production   # requires approval
```

```
  jobs:
```

```
  - deployment: DeployAKS
```

```
    strategy:
```

```
      runOnce:
```

```
        deploy:
```

```
          steps:
```

```
          - task: KubernetesManifest@1
```

Azure DevOps is a complete platform: Repos (Git hosting), Pipelines (CI/CD), Boards (Agile), Artifacts (package registry), Test Plans. Pipelines: multi-stage YAML, environment gates (manual approval), service connections (to Azure subscriptions, Docker registries, Kubernetes clusters). Azure Container Registry (ACR) is the native image registry — use Managed Identity for AKS to pull from ACR without credentials. GitHub Actions + OIDC federation is increasingly preferred over Azure DevOps Pipelines for teams on GitHub.

> 📚 **Official docs**
>
> - Azure DevOps documentation

certs.eknathalabs.com

Q11 How do you deploy to AKS from a CI/CD pipeline securely?

CI/CD

🎙 Spoken answer · ~87 s

Secure deployment to AKS means minimising stored secrets and limiting what the pipeline can do. Authentication: the pipeline authenticates to Azure using workload identity federation, OIDC, between the CI system and an Entra application or managed identity, so there are no long-lived secrets; the identity gets only the roles it needs, for instance push rights to ACR and a scoped role on the cluster. For cluster access, use Azure RBAC for Kubernetes authorisation, or Kubernetes RBAC with a namespace-scoped role, rather than cluster-admin credentials. Build stage: scan code and dependencies, build the image, scan it, push to ACR with an immutable tag by commit SHA, and sign it. Deploy stage: apply manifests or Helm charts with a deploy identity limited to the target namespace, or follow GitOps, where the pipeline only updates the config repository and Flux or ArgoCD in the cluster pulls changes; this keeps cluster credentials out of CI entirely. For private clusters, run deployments from self-hosted agents inside the network or use az aks command invoke. Add policy gates: Azure Policy or admission control to allow only signed images from trusted registries; protect production with approvals and environment checks. Retrieve application secrets at runtime from Key Vault via the CSI driver, not from pipeline variables.

Likely follow-ups

- Why is GitOps considered more secure for cluster deployments?
- How does a pipeline deploy to a private cluster?

```
# GitHub Actions: OIDC federation (no credentials)
- uses: azure/login@v2
  with:
    client-id: ${{ secrets.AZURE_CLIENT_ID }}
    tenant-id: ${{ secrets.AZURE_TENANT_ID }}
    subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
- uses: azure/aks-set-context@v3
  with: {resource-group: rg, cluster-name: prod-aks}
- run: kubectl apply -f k8s/
# Or: update Helm values / Kustomize manifests in GitOps repo
# ArgoCD or Flux watches the manifests repo and syncs to AKS
```

certs.eknathalabs.com

OIDC federation: configure a Federated Identity Credential on an Entra ID app/managed identity, mapping it to the GitHub Actions OIDC token. No client secret needed — GitHub OIDC token exchanges for a short-lived Azure access token. The service principal needs AKS Cluster User role + deployment permissions in Kubernetes RBAC. For GitOps: the pipeline pushes to the manifests repo, ArgoCD/Flux handles the actual kubectl apply — this keeps deployment credentials out of CI entirely.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q12 What is Azure Private Endpoint — how does it differ from Service Endpoint? Networking

🎙 **Spoken answer · ~86 s**

Both let resources in a VNet reach Azure PaaS services more securely, but they work differently. A service endpoint extends your VNet's identity to the service: traffic from the subnet to the service, like Storage or SQL, takes the Azure backbone, and the service can restrict access to requests from specified subnets. However, the service still has its public IP and endpoint; the destination remains a public address, and it doesn't give on-premises or peered-network access to the resource privately. A private endpoint creates a network interface in your subnet with a private IP address from your VNet, mapped to a specific resource instance, via Azure Private Link. Traffic goes to that private IP, and you can disable public network access to the resource entirely. It works from peered VNets and on-premises over VPN or ExpressRoute, supports specific sub-resources, and protects against data exfiltration to other instances of the service. It requires DNS configuration so the service name resolves to the private IP, usually with private DNS zones, and has a per-hour and per-gigabyte cost. Service endpoints are simpler and free, suitable when you only need subnet-level restriction. Private endpoints are the stronger, preferred option for security-sensitive and hybrid scenarios. The most common issue is DNS misconfiguration.

Likely follow-ups

- Why does a private endpoint prevent exfiltration better?
- What DNS setup does a private endpoint need?

```
# Private Endpoint (private IP in your VNet for PaaS service):
```

```
az network private-endpoint create --resource-group rg --name mydb-pe \
```

```
  --vnet-name prod-vnet --subnet db-subnet \
```

```
  --private-connection-resource-id /subscriptions/.../servers/mydb \
```

```
  --group-id sqlServer --connection-name mydb-conn
```

```
# DNS: mydb.database.windows.net resolves to 10.x.x.x (private IP)
```

certs.eknathalabs.com

```
az network private-dns zone create --resource-group rg --name
privatelink.database.windows.net
```

```
# Service Endpoint (extend VNet identity to public endpoint):
```

```
az network vnet subnet update --vnet-name prod-vnet --name db-subnet --service-endpoints
Microsoft.Sql
```

Private Endpoint: PaaS service (SQL, Storage, Key Vault) gets a private IP in your VNet — traffic never leaves Azure backbone network, service can disable public access. Full network isolation. Service Endpoint: optimises routing from subnet to Azure backbone but the service still has a public IP — less secure than Private Endpoint. Use Private Endpoints for production databases and storage. Private DNS zones are required to resolve PaaS service hostnames to private IPs.

📚 **Official docs**

- Azure Private Endpoint

Q13 What is Azure Virtual Machine Scale Sets — how does it work with AKS node pools?

Compute

🎙 **Spoken answer · ~88 s**

Virtual Machine Scale Sets manage a group of identical VMs, with built-in autoscaling, load balancer integration, zone distribution, and rolling upgrades. They have two orchestration modes: uniform, for identical instances, and flexible, which allows mixing and more control. AKS node pools are implemented as scale sets, one per node pool: when you create a node pool, AKS creates a VMSS in the node resource group. The cluster autoscaler adjusts the scale set's instance count based on pending pods; you set minimum and maximum node counts per pool. Node pools let you run different VM sizes and configurations in one cluster: a system pool for core components like CoreDNS, and user pools for workloads, such as GPU pools, spot pools, or memory-optimised pools, selected with labels, taints, and tolerations. Availability zones can be configured so nodes spread across zones for resilience. Upgrades use surge: AKS adds extra nodes, cordons and drains the old ones, and replaces them, with max-surge controlling speed and extra capacity. Node image upgrades roll out new OS images. Recommendations: separate system and user pools, use multiple pools for different workload profiles, avoid manual changes to the scale set outside AKS because they can cause drift, size subnets for surge, and set pod disruption budgets for safe draining.

Likely follow-ups

- Why separate system and user node pools?
- How does surge upgrading work?

```
# AKS node pool = VMSS under the hood
```

certs.eknathalabs.com

```
az aks nodepool add --resource-group rg --cluster-name prod-aks \
  --name gpunodes --node-vm-size Standard_NC6s_v3 --node-count 1 \
  --enable-cluster-autoscaler --min-count 0 --max-count 5
# Manual scale:
az aks nodepool scale --resource-group rg --cluster-name prod-aks --name workers --node-count 5
# Node pool upgrade:
az aks nodepool upgrade --resource-group rg --cluster-name prod-aks --name workers --kubernetes-version 1.30
```

AKS node pools are backed by VMSS — the Cluster Autoscaler scales VMSS instances based on pending pod requirements. Multiple node pools: system pool (system-critical pods — use at least Standard_D2s_v3), user pools (workload pods), specialised pools (GPU, spot). Spot node pools: 60-90% cost savings, can be evicted with 30 seconds notice — use for batch jobs or fault-tolerant workloads with toleration. Node auto-provisioning (KEDA-based) is the Azure equivalent of Karpenter for dynamic node type selection.

> 📚 **Official docs**
>
> - Azure Virtual Machine Scale Sets
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q14 What is Azure Functions and when do you use it vs containers on AKS?

Compute

🎙 Spoken answer · ~84 s

Azure Functions is a serverless compute service that runs code triggered by events: HTTP requests, timers, queue messages, blob changes, Event Hubs, and more, with bindings that simplify input and output integration. Hosting plans: Consumption scales automatically and charges per execution, with cold starts; Flex Consumption and Premium offer pre-warmed instances, virtual network integration, and longer runtimes; Dedicated plans run on App Service capacity. Durable Functions add stateful orchestration for workflows. Use Functions for event-driven and spiky workloads, glue code, scheduled tasks, lightweight APIs, and processing streams or files, where you want to pay only for use and avoid managing infrastructure. Use containers on AKS when workloads are steady and high throughput, where per-execution pricing exceeds container costs; need long-running processes; require custom runtimes, sidecars, or fine control over networking and scheduling; run many interdependent microservices with a platform team; or must be portable across clouds. There is middle ground: Azure Container Apps offers serverless containers with KEDA scaling and Dapr, and Functions can even run on Kubernetes through KEDA. Choose on workload shape, operational capacity, cost at expected scale, latency tolerance for cold starts, and team skills. Many architectures mix them: AKS for core services and Functions for event glue.

Likely follow-ups

- What is the role of Durable Functions?
- Where does Azure Container Apps fit between Functions and AKS?

```
az functionapp create --resource-group rg --name myfunc \
  --storage-account mystorage --consumption-plan-location eastus \
  --runtime python --runtime-version 3.11 --functions-version 4
# Deploy function:
func azure functionapp publish myfunc
# Trigger types: HTTP, Timer, Service Bus, Event Hub, Blob, Cosmos DB
# KEDA (Kubernetes Event-Driven Autoscaling) on AKS:
helm install keda kedacore/keda -n keda
# Scale to zero based on queue depth, no Functions needed
```

Azure Functions: serverless, scales to zero, pay-per-execution, up to 10-minute timeout (Premium plan: unlimited). Best for: event-driven processing, short-lived tasks, variable/unpredictable traffic. AKS: persistent workloads, long-running processes, stateful services. KEDA (Kubernetes Event-Driven Autoscaling) brings

certs.eknathalabs.com

Functions-like scale-to-zero to AKS — scale pods based on queue depth, event hub lag, or custom metrics. For platform teams: KEDA on AKS is often preferred over Functions for standardisation (same K8s tooling everywhere).

📚 **Official docs**

- Azure Functions
- Azure Kubernetes Service (AKS)

Q15 How does Azure Key Vault integrate with AKS — what are the patterns?

IAM & EntraID

🎙 **Spoken answer · ~89 s**

Key Vault stores secrets, keys, and certificates, with access control through RBAC or access policies, auditing, soft delete, and purge protection. On AKS, the preferred integration is the Secrets Store CSI Driver with the Azure Key Vault provider, available as an AKS add-on. You define a SecretProviderClass naming the vault and the objects; pods mount a CSI volume and the driver fetches secrets and writes them as files in the pod, optionally also syncing them to Kubernetes Secrets and exposing as environment variables. Authentication should use Microsoft Entra Workload ID: a Kubernetes service account is federated to a managed identity, which is granted a role such as Key Vault Secrets User, so pods get Key Vault access without any stored credentials. Older approaches, like pod identity and storing service principal credentials, are deprecated or inferior. Alternative patterns: the External Secrets Operator syncs Key Vault values into Kubernetes Secrets; or the application calls Key Vault directly with the SDK using workload identity, which supports caching and rotation handling. Best practices: use RBAC and least privilege per workload, enable the rotation feature for mounted secrets, use private endpoints for the vault, enable purge protection and logging, avoid putting secrets in environment variables baked into images, and encrypt Kubernetes secrets at rest with Key Vault-managed keys.

Likely follow-ups

- Why is workload identity better than service principal credentials for Key Vault access?
- What does a SecretProviderClass define?

certs.eknathalabs.com

```
# Key Vault CSI Driver (mount secrets as volumes):
```

```
az aks enable-addons --addons azure-keyvault-secrets-provider --name prod-aks --resource-group rg
```

```
# SecretProviderClass:
```

```
spec:
```

```
  provider: azure
```

```
  parameters:
```

```
    vaultName: myvault
```

```
    objects: |
```

```
      array:
```

```
      - objectName: db-password
```

```
        objectType: secret
```

```
# Pod mounts /mnt/secrets-store/db-password as a file
```

```
# External Secrets Operator (sync to K8s Secrets):
```

```
kubectl get externalsecret db-credentials  # syncs Key Vault → K8s Secret
```

Two patterns: CSI Driver mounts secrets as files/env vars directly from Key Vault (no K8s Secret created). External Secrets Operator syncs Key Vault secrets into native K8s Secrets (applications read normal K8s Secrets — no Key Vault SDK needed). CSI Driver is more secure (secret never stored in etcd). ESO is more compatible (works with any application that reads K8s Secrets). Both use Workload Identity — no credentials in pod specs.

> 📚 **Official docs**
>
> - Azure Key Vault
> - Azure Kubernetes Service (AKS)
> - HashiCorp Vault documentation

certs.eknathalabs.com

Q16 What is Azure DDoS Protection — how does it work with Application Gateway?

Networking

🎙 Spoken answer · ~86 s

Azure DDoS Protection defends against volumetric, protocol, and resource-layer attacks on public IP addresses. The Network tier, applied to a virtual network, and the IP Protection tier, applied per public IP, use adaptive tuning that learns your normal traffic profile and automatically detects and mitigates attacks at the network edge at layers three and four. Both give telemetry, alerts, mitigation reports, and the paid tiers add rapid response support and cost protection, which credits scaling costs during an attack. Basic infrastructure protection is automatic for all Azure services. With Application Gateway: the gateway's public IP sits in a protected VNet, so volumetric attacks are mitigated before reaching it, while the Application Gateway WAF handles layer seven attacks such as SQL injection, cross-site scripting, and bots, with managed rule sets, rate limiting and custom rules. They are complementary: DDoS protection absorbs network floods, and the WAF inspects HTTP traffic. For comprehensive protection, add Azure Front Door with its WAF and global edge to absorb attacks closer to the source, restrict origin access to Front Door only, size autoscaling for the gateway, and monitor DDoS metrics. Build resilient applications: autoscaling, caching, and a design that degrades gracefully. Run attack simulations with approved partners, and have a response runbook.

Likely follow-ups

- Which layers does DDoS Protection cover versus the WAF?
- Why put Front Door in front of the gateway?

```
# Enable DDoS Network Protection on VNet:
```

```
az network ddos-protection create --resource-group rg --name myplan
```

```
az network vnet update --resource-group rg --name prod-vnet --ddos-protection-plan myplan
```

```
# App Gateway WAF policy:
```

```
az network application-gateway waf-policy create --resource-group rg --name mywaf
```

```
az network application-gateway waf-policy managed-rules rule-set add \
```

```
  --policy-name mywaf --type OWASP --version 3.2
```

```
az network application-gateway update --name appgw --waf-policy mywaf
```

DDoS Network Protection: volumetric and protocol attacks mitigated automatically at the Azure network edge — L3/L4 DDoS. WAF on Application Gateway: L7 attacks (SQLi, XSS, command injection) — OWASP 3.2 managed rules + custom rules. Together they cover all DDoS layers. For AKS exposed via Application Gateway: App Gateway WAF protects against web attacks, DDoS Protection handles volumetric attacks. Azure DDoS Response Team available 24/7 with Standard plan for attack assistance.

certs.eknathalabs.com

📚 **Official docs**

- Cloudflare Learning: DDoS attacks
- Azure Application Gateway

Q17 How do you debug a CrashLoopBackOff pod on AKS?

Troubleshooting

🎙 **Spoken answer · ~88 s**

CrashLoopBackOff means a container starts and exits repeatedly, with Kubernetes increasing the delay between restarts. I begin with kubectl describe pod to see the events, last state, exit code, and restart count, then kubectl logs, including the previous flag, to read the logs from the last crashed container. Exit codes help: zero with repeated restarts suggests the process finishes instead of staying alive; one usually points to an application error; 137 with OOMKilled indicates memory limit exceeded; 139 suggests a segmentation fault; 143 is SIGTERM. Common causes: application exceptions at startup; missing or wrong configuration, environment variables, ConfigMaps, or secrets; failing dependencies such as a database being unreachable or credentials wrong; incorrect command or entrypoint; permission issues, such as file permissions or a security context; failing liveness probes that kill slow-starting containers, solved by startup probes or longer delays; and insufficient resources. On AKS, also check Key Vault CSI mounts, workload identity setup, and image architecture. To investigate further, run an ephemeral debug container, or start the image with an overridden command to explore it, and compare with a working environment. Check Container Insights for restart trends and memory metrics. Fix the root cause, whether code, config, or limits, then add alerts on restart counts, and readiness gates to prevent bad rollouts.

Likely follow-ups

- What does exit code 137 tell you?
- How can a liveness probe create a crash loop?

```
kubectl get pods -n prod
```

```
kubectl describe pod myapp-xxx -n prod  # check Events section
```

```
kubectl logs myapp-xxx -n prod --previous  # logs from crashed container
```

```
kubectl logs myapp-xxx -n prod -c initcontainer  # init container logs
```

```
# Check resource limits:
```

```
kubectl describe node <node> | grep -A10 Allocated
```

```
# Check events for OOMKill:
```

```
kubectl get events -n prod --sort-by=.lastTimestamp | grep -i oom
```

certs.eknathalabs.com

```
# Exec into a working pod for comparison:
```

```
kubectl run debug --image=myapp:v1 --rm -it -- /bin/sh
```

CrashLoopBackOff: container starts, crashes, Kubernetes restarts it, exponential backoff kicks in. Common causes: application error on startup (missing env var, failed DB connection), OOMKill (memory limit too low), liveness probe failing and killing a healthy container, init container failing (dependency not ready), and permission errors (file not readable, port binding failure). Always check --previous logs first — current container may have no logs if it crashes immediately.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Kubernetes: Debug Pods
- Azure Kubernetes Service (AKS)

Q18 How do you optimise AKS costs on Azure?

Cost

🎙 **Spoken answer · ~87 s**

AKS cost optimisation works at several layers. Right-size: set accurate resource requests and limits so nodes are well packed, using VPA recommendations or tooling like the AKS cost analysis add-on and Kubecost; choose VM sizes that fit workloads and use multiple node pools for different profiles. Autoscale: use Horizontal Pod Autoscaler or KEDA for pods and the cluster autoscaler, or node auto-provisioning, which is Karpenter-based, for nodes; scale non-production down overnight, using start-stop or scaling pools to zero. Use spot node pools for fault-tolerant workloads for big discounts. Commitments: reserved instances or savings plans for the steady baseline. Use Arm-based VM sizes where workloads support them for price-performance. Pick the free or standard tier of the control plane appropriately. Storage: right-size disks, delete unused persistent volumes and snapshots, and choose suitable disk tiers. Networking: reduce cross-zone and egress traffic, and review load balancer and public IP counts. Observability costs: tune Container Insights and Log Analytics ingestion with data collection rules, use basic logs and shorter retention, and limit metric cardinality. Cleanup: remove unused clusters, namespaces, and images in ACR via retention. Governance: tag and use labels for cost allocation by team, set budgets and alerts, and review cost regularly. Measure before and after, and avoid cuts that hurt reliability.

Likely follow-ups

- How does node auto-provisioning reduce waste?
- What logging settings reduce Log Analytics cost?

```
# Spot node pools for non-critical workloads:
```

```
az aks nodepool add --name spot --spot-max-price -1 --priority Spot --eviction-policy
Delete
```

certs.eknathalabs.com

```
# Right-size nodes with Azure Advisor:
```

```
az advisor recommendation list --category Cost --output table
```

```
# Stop/start AKS cluster (dev/test):
```

```
az aks stop --resource-group rg --name dev-aks    # pause billing on nodes
```

```
az aks start --resource-group rg --name dev-aks
```

```
# Azure Reservations (1-3 year VM commitment):
```

```
# 40-72% savings on VM compute vs PAYG
```

```
# Microsoft Azure Hybrid Benefit (if you have Windows Server licences)
```

AKS cost levers: Spot node pools for batch/fault-tolerant workloads (60-80% savings), right-sizing via Azure Advisor and Container Insights metrics, stopping dev/test clusters outside business hours, Azure Reserved VM Instances for predictable baseline (1-3 year commitment), and cluster consolidation (fewer larger clusters vs many small ones). Container Insights + Azure Cost Management tags enable per-namespace, per-team cost allocation for chargeback.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q19 How do you configure Horizontal Pod Autoscaling on AKS?

Containers & AKS

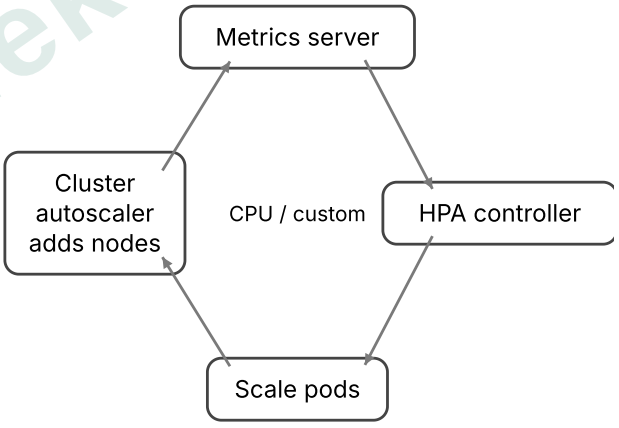
🎙 Spoken answer · ~85 s

The Horizontal Pod Autoscaler adjusts replica counts based on observed metrics. Prerequisites: the metrics server, which AKS provides by default, and, crucially, resource requests defined on containers, because CPU utilisation is calculated relative to requests. Configure it by creating an HPA resource targeting a Deployment: specify min and max replicas and a metric, for example average CPU utilisation of seventy percent, using the autoscaling/v2 API, which also supports memory, custom, and external metrics. Example by command: kubectl autoscale deployment web with cpu-percent seventy, min two, max ten. The controller periodically compares the current metric to the target and calculates the desired replicas. Tune behaviour with the behaviour field: scale-up and scale-down policies, stabilisation windows to avoid flapping, and rate limits. For event-driven or queue-based scaling, use KEDA, available as an AKS add-on, which scales on sources like Service Bus queue length, Event Hubs lag, or Prometheus queries, including scale to zero. Pair HPA with the cluster autoscaler so new pods that can't be scheduled trigger node additions. Avoid using HPA and VPA on the same metric. Set sensible minimums for availability, with PodDisruptionBudgets, ensure readiness probes so new pods receive traffic only when ready, and load test to validate. Monitor scaling events and adjust targets.

Likely follow-ups

- Why must containers have CPU requests for HPA to work?
- When would you use KEDA instead of plain HPA?

🗺 HPA on AKS




```
kubectl autoscale deployment myapp --cpu-percent=70 --min=3 --max=20
```

```
# Custom metric HPA (KEDA):
```

```
apiVersion: keda.sh/v1alpha1
```

```
kind: ScaledObject
```

certs.eknathalabs.com

```
spec:
  scaleTargetRef: {name: myapp}
  minReplicaCount: 0
  maxReplicaCount: 50
  triggers:
  - type: azure-servicebus
    metadata:
      queueName: orders
      queueLength: "10"     # scale when queue depth exceeds 10 messages/replica
```

HPA v2: scale on CPU, memory, custom metrics (Prometheus), or external metrics (Azure Service Bus queue depth). KEDA extends HPA to scale on any event source (Service Bus, Event Hub, Storage Queue, HTTP traffic, Cron schedule). Scale-to-zero with KEDA: pods go to 0 when queue is empty, scale up when messages arrive — impossible with native HPA. Cluster Autoscaler scales nodes to accommodate scaled-up pods. The full stack: KEDA scales pods → Cluster Autoscaler scales nodes.

> 📚 **Official docs**
> - Kubernetes: Horizontal Pod Autoscaling
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q20 An AKS pod is running but the Service is not routing traffic to it — how do you debug?

Troubleshooting

🎙 Spoken answer · ~83 s

When a pod is running but the Service isn't routing, I verify the chain from Service to endpoints to pod. First, check the Service and its endpoints: kubectl get endpoints, or endpointslices, for the service. If the endpoints list is empty, the selector doesn't match the pod labels, or the pods aren't Ready; compare the Service's selector with the pod's labels exactly. If pods are running but not ready, readiness probes are failing, so the pod is removed from endpoints; check probe configuration and the application's actual health. Next, confirm the ports: the Service's targetPort must match the container port the application listens on, and the application must bind to all interfaces, not just localhost. Test from inside the cluster: run a debug pod and curl the pod IP directly, then the Service's cluster IP, then the DNS name, to find which layer fails. If direct pod access works but the Service doesn't, check kube-proxy or the dataplane. If DNS fails, check CoreDNS. Check network policies that might block traffic between namespaces, and NSGs if traffic crosses nodes. For external access, check the Service type, load balancer provisioning and health probes, or the ingress rules and backend configuration.

Likely follow-ups

- What does an empty endpoints list tell you?
- How do you isolate which layer is failing?

```
# Step 1: Check pod is actually Ready
```

```
kubectl get pods -l app=myapp -o wide   # STATUS and READY columns
```

```
# Step 2: Check Service selector matches pod labels
```

```
kubectl describe service myapp          # Selector vs pod labels
```

```
# Step 3: Check Endpoints
```

```
kubectl get endpoints myapp             # should list pod IPs
```

```
# Step 4: Check readiness probe
```

```
kubectl describe pod myapp-xxx | grep -A5 Readiness
```

```
# Step 5: Test directly to pod IP
```

```
kubectl run tmp --rm -it --image=curlimages/curl -- curl http://10.240.0.5:8080/health
```

```
# Step 6: NSG blocking pod subnet traffic?
```

```
az network nsg rule list --resource-group MC_rg_prod-aks_eastus --nsg-name aks-nsg
```

certs.eknathalabs.com

Most common causes: readiness probe failing (pod Running but not Ready, so Endpoints are empty), Service selector does not match pod labels (typo in label name/value), pod listening on wrong port (containerPort != service targetPort), and on AKS with Azure CNI: NSG rule blocking traffic between subnets. If Endpoints is empty, check selector. If Endpoints has IPs but traffic fails, check the port and readiness probe.

📚 **Official docs**

- Kubernetes: Debug Services
- Azure Kubernetes Service (AKS)

Q21 What is Azure Cosmos DB — when do you use it?

Databases

🎙 **Spoken answer · ~85 s**

Azure Cosmos DB is a globally distributed, multi-model NoSQL database service with single-digit-millisecond latency, elastic scale, and turnkey multi-region replication. It supports several APIs: the native NoSQL API with JSON documents, plus MongoDB, Cassandra, Gremlin graph, and Table APIs. Key concepts: data is partitioned by a partition key you choose, which determines distribution and scale, so choosing a high-cardinality key with even access is the most important design decision; throughput is measured in request units per second, provisioned manually, with autoscale, or serverless, and every operation consumes RUs depending on its cost. It offers five tunable consistency levels, from strong to eventual, with session consistency as the common default, letting you trade consistency for latency and availability. You can replicate to multiple regions with multi-region writes and automatic failover, and it provides availability SLAs up to five nines. Use it for globally distributed applications, high-throughput low-latency workloads, IoT and telemetry, personalisation, shopping carts, and flexible schemas. Avoid it for complex relational queries and joins, heavy analytical workloads, or when costs at high RU consumption outweigh benefits. Manage cost by modelling data for access patterns, tuning indexing policy, avoiding cross-partition queries, and choosing the right throughput mode. Secure with private endpoints and managed identities.

Likely follow-ups

- Why is partition key choice the most important design decision?
- What are Cosmos DB's consistency levels?

```
az cosmosdb create --resource-group rg --name mycosmosdb \
```

```
  --default-consistency-level Session --locations regionName=eastus
```

```
az cosmosdb sql database create --account-name mycosmosdb --name mydb
```

```
az cosmosdb sql container create --account-name mycosmosdb --database-name mydb \
```

```
  --name mycontainer --partition-key-path /userId --throughput 400
```

certs.eknathalabs.com

Cosmos DB: globally distributed NoSQL database with multi-model API (SQL/Core, MongoDB, Cassandra, Gremlin, Table). Turnkey global distribution — replicate to any Azure region with one click. Five consistency levels (Strong to Eventual). 99.999% availability SLA with multi-region writes. Use for: globally distributed apps where latency matters, high-throughput workloads that need horizontal scale, flexible schema documents. Partition key choice is critical — poor partitioning causes hot partitions and throttling. Use Autoscale throughput to avoid over-provisioning.

📚 **Official docs**

- Azure Cosmos DB

Q22 What is Azure Front Door — how does it differ from Application Gateway?

Networking

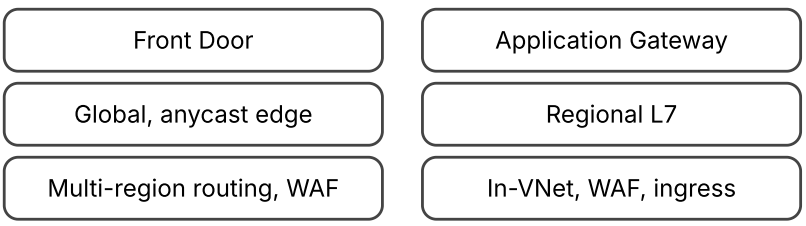
🎙 **Spoken answer · ~90 s**

Azure Front Door is a global, layer-seven entry point that uses Microsoft's edge network to accelerate and secure web applications. It provides anycast routing to the nearest edge, TLS termination, caching and compression, URL path routing, health probes with failover across origins in multiple regions, rules engine, and an integrated WAF and DDoS protection. It can route to origins anywhere: Azure, on-premises, or other clouds. Application Gateway is a regional layer-seven load balancer that lives inside a VNet, providing path-based and host-based routing, SSL termination, WAF, autoscaling, and ingress for AKS through AGIC; it routes within a region to backends in your virtual network. Key differences: scope, global versus regional; position, at the edge of the network versus inside a VNet; and capabilities, such as caching and global failover in Front Door, and private backend routing and tight VNet integration in Application Gateway. They're often combined: Front Door at the edge for global routing, acceleration, and a first WAF layer, forwarding to regional Application Gateways that front the VNet workloads; lock down the gateway to accept traffic only from Front Door using its service tag and header validation. Use Front Door alone for a simple global site with public origins, Application Gateway alone for a single-region application, and both for global applications with private backends.

Likely follow-ups

- How do you make a regional gateway accept traffic only from Front Door?
- When is Front Door alone sufficient?

🗺 **Front Door vs App Gateway**



certs.eknathalabs.com

```
az afd profile create --resource-group rg --profile-name myfd --sku Premium_AzureFrontDoor
```

```
az afd endpoint create --resource-group rg --profile-name myfd --endpoint-name api
```

```
az afd origin-group create --resource-group rg --profile-name myfd --origin-group-name
backends
```

```
az afd origin create --origin-group-name backends --host-name
app1.eastus.cloudapp.azure.com --origin-host-header app1.eastus.cloudapp.azure.com
```

Application Gateway: regional (single Azure region), L7 load balancer, WAF, best for intra-region traffic. Azure Front Door: global CDN and L7 load balancer, routes to the nearest healthy backend across regions, edge PoPs in 100+ locations worldwide. Use Front Door for: global user base (reduces latency by routing to nearest region), global failover (health-check-based routing away from unhealthy regions), DDoS + WAF at the edge, and SSL offload at the CDN edge. The combination: Front Door (global) → App Gateway (regional WAF) → AKS is a common enterprise pattern.

> 📚 **Official docs**
>
> - Azure Application Gateway
> - Azure Front Door

certs.eknathalabs.com

Q23 How do you use Azure Monitor alerts and action groups for incident response?

Monitoring

🎙 Spoken answer · ~86 s

Azure Monitor alerts detect conditions and action groups define what happens next. Alert types: metric alerts evaluate platform or custom metrics against static or dynamic thresholds, good for near-real-time conditions like CPU or error counts; log alerts run Kusto queries on a schedule against Log Analytics, suited to complex conditions such as pod restart counts or specific log patterns; activity log alerts trigger on control-plane events like resource deletion; and Prometheus alert rules for managed Prometheus metrics. Alert rules have severity, evaluation frequency, and a scope. Action groups are reusable sets of actions: email, SMS, voice, push notifications; webhooks, Logic Apps, Azure Functions, automation runbooks, ITSM tools, and event hubs. Integration with paging systems like PagerDuty or Teams is typical. For incident response: base paging alerts on user-impacting symptoms, such as availability and latency SLO burn, with cause-based signals as lower-severity warnings; use dynamic thresholds where baselines vary; route by severity to the right groups and on-call schedules; include runbook links and context in the alert; use alert processing rules to suppress during maintenance; and use automation for first-response remediation, such as restarting a service or scaling. Manage alerts as code, test them, review noise and missed incidents regularly, and track time to detect and acknowledge.

Likely follow-ups

- When do you use a log alert instead of a metric alert?
- What are alert processing rules for?

```
# Action Group (where to send alerts):
```

```
az monitor action-group create --resource-group rg --name ops-team \
```

```
  --action email ops ops-team@company.com \
```

```
  --action webhook pagerduty https://events.pagerduty.com/integration/xxx/enqueue
```

```
# Metric alert:
```

```
az monitor metrics alert create --name pod-oom --resource $AKS_ID \
```

```
  --condition "count kube_pod_container_status_last_terminated_reason where reason includes
OOMKilled > 0" \
```

```
  --action /subscriptions/.../actionGroups/ops-team
```

```
# Log alert (KQL):
```

```
az monitor scheduled-query create --resource-group rg --name high-errors \
```

certs.eknathalabs.com

```
  --condition-query "ContainerLog | where LogEntry contains "ERROR" | summarize count() > 10"
```

Action Groups define who gets notified and how (email, SMS, webhook, PagerDuty, Logic App, Azure Function). Reuse action groups across multiple alerts — update once, changes everywhere. Metric alerts fire within 1-5 minutes. Log alerts (KQL) fire within 5-15 minutes. Smart Groups automatically cluster related alerts to reduce noise. Alert processing rules can suppress alerts during maintenance windows without disabling them.

📚 **Official docs**

- Azure Monitor
- Google SRE Book: Managing Incidents
- Google SRE Workbook: Incident Response

Q24 AKS nodes are NotReady — how do you diagnose and recover?

Troubleshooting

🎙 **Spoken answer · ~85 s**

When AKS nodes go NotReady, the kubelet isn't reporting healthy to the control plane. Start with kubectl describe node to read conditions and events: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable, and messages from the kubelet. Identify the pattern: one node, a pool, or all nodes; if all, suspect the control plane connectivity, networking, or a recent change like an upgrade or policy. Common causes: resource exhaustion, where memory, disk, or PIDs run out and the kubelet or container runtime struggles; the node VM being unhealthy, stopped, or evicted, in the case of spot; kubelet or container runtime failure; network problems between node and API server, such as NSG, firewall, UDR, or DNS blocking required endpoints; certificate or time sync issues; disk full from logs or images; and failed extensions or scale set instance problems. Investigate using Container Insights, the VMSS instance view and boot diagnostics, and the Azure portal's diagnose and solve problems. You can access the node through a debug pod with host access, using kubectl debug node, to check kubelet logs and disk usage. Recover: cordon and drain the node, then reimage or replace it through the scale set, since nodes are cattle; scale the pool to let the autoscaler create a replacement.

Likely follow-ups

- How do you get onto an AKS node to inspect it?
- Why replace a bad node rather than repair it?

```
kubectl get nodes                          # which nodes are NotReady?
```

```
kubectl describe node mynode | tail -30    # Events section
```

```
kubectl get pods --all-namespaces --field-selector spec.nodeName=mynode  # pods on the node
```

```
# SSH to node (via AKS debug pod):
```

certs.eknathalabs.com

```
kubectl debug node/mynode -it --image=mcr.microsoft.com/cbl-mariner/busybox:2.0
```

```
chroot /host
```

```
journalctl -u kubelet --since "10 minutes ago" | tail -50
```

```
systemctl status kubelet
```

```
# Check node resource pressure:
```

```
kubectl describe node mynode | grep -A10 Conditions
```

Common causes: kubelet crash (check journalctl -u kubelet), disk pressure (check ephemeral storage, /var/log, container image cache), memory pressure (OOMKiller killed kubelet), network plugin failure (Azure CNI misconfiguration), or node unresponsive (Azure VM health issue). For persistent node failures: cordon, drain, and delete the node — the VMSS will provision a replacement automatically. AKS Node Auto-Repair detects and replaces unhealthy nodes automatically in managed node pools.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q25 What is Azure Policy — how do you enforce governance on AKS workloads?

IAM & EntraID

🎙 Spoken answer · ~84 s

Azure Policy evaluates resources against rules and can audit, deny, modify, or deploy configuration, enforcing governance at scale across management groups, subscriptions, and resource groups. Policies are grouped into initiatives, and assignments apply them to scopes, with exemptions for exceptions. For AKS, the Azure Policy add-on installs Gatekeeper, the OPA-based admission controller, and syncs Azure Policy definitions into constraint templates. Built-in policies cover common controls: no privileged containers, no host networking or host path mounts, allowed container registries only, require resource limits, disallow latest tags, enforce labels, require HTTPS ingress, restrict allowed capabilities, and the pod security baseline and restricted standards. At the Azure resource level, policies enforce things like requiring private clusters, Entra ID integration, Azure RBAC, disabled local accounts, diagnostics enabled, and approved regions and SKUs. Rollout approach: start in audit mode to see non-compliance without disruption, review results in the compliance dashboard, fix workloads, then switch to deny for critical rules, with namespace exclusions for system namespaces. Use remediation tasks with deployIfNotExists and modify effects to bring existing resources into compliance. Manage policies as code with versioning and testing, communicate changes to developers, and provide clear error messages. Complement with Defender for Containers for runtime and vulnerability coverage.

Likely follow-ups

- Why start new policies in audit mode?
- What does the Azure Policy add-on install on AKS?

```
# Assign built-in policy to subscription:
```

```
az policy assignment create --name require-pod-limits \
```

```
  --policy "/providers/Microsoft.Authorization/policyDefinitions/e345eecc-fa47-480f-9e88-
67dcc122b164" \
```

```
  --scope /subscriptions/SUB \
```

```
  --params "{"allowedContainerImagesRegex":{"value":"myregistry.azurecr.io/.*"}}"
```

```
# Enable Azure Policy add-on for AKS (OPA Gatekeeper):
```

```
az aks enable-addons --addons azure-policy --name prod-aks --resource-group rg
```

```
kubectl get constrainttemplate                    # Gatekeeper constraint templates
```

```
kubectl get k8srequiredlabels                      # custom constraints
```

certs.eknathalabs.com

Azure Policy for Kubernetes: uses OPA Gatekeeper under the hood to enforce policies at pod admission time. Built-in policies: require resource limits, require readiness/liveness probes, restrict privileged containers, require specific labels, allow only images from approved registries. Policy effects: Audit (log violations), Deny (block non-compliant deployments). Policies can be assigned at Management Group → Subscription → Resource Group scope — inherit hierarchically. Essential for SOC2, CIS Benchmark, and internal governance requirements.

📚 **Official docs**

- Azure Policy
- Azure Kubernetes Service (AKS)

Q26 What is Azure Spot VMs — how do you use them with AKS safely?

Compute

🎙 **Spoken answer · ~86 s**

Azure Spot VMs use spare capacity at deep discounts, up to around ninety percent, but Azure can evict them with short notice, thirty seconds, when it needs the capacity back or the price exceeds your maximum. On AKS, spot node pools are separate user pools backed by spot scale sets; AKS automatically taints them with a spot taint, so only pods that explicitly tolerate it are scheduled there. Use them for stateless, fault-tolerant, interruptible workloads: batch jobs, CI runners, dev and test environments, background workers, and the burst portion of scalable services. To use them safely: keep system pods and critical services on regular pools; add a tolerations entry and node affinity or selectors to target spot pools; configure multiple spot pools across different VM sizes and zones to reduce simultaneous eviction risk; set the eviction policy and a maximum price, typically the pay-as-you-go cap; run multiple replicas with topology spread and PodDisruptionBudgets; handle termination gracefully with short graceful shutdown periods, checkpoints, and idempotent job design; and use the cluster autoscaler with the priority expander to prefer spot and fall back to regular nodes. Avoid spot for databases, singletons, or latency-critical single points. Monitor eviction rates and adjust. The result is major savings with a resilient design.

Likely follow-ups

- Why does AKS taint spot node pools?
- How does the priority expander provide a fallback?

```
# Create spot node pool:
```

```
az aks nodepool add --resource-group rg --cluster-name prod-aks \
```

```
  --name spot --priority Spot --eviction-policy Delete \
```

```
  --spot-max-price -1 --node-vm-size Standard_D4s_v5
```

```
# Toleration for spot nodes:
```

certs.eknathalabs.com

```
tolerations:
- key: "kubernetes.azure.com/scalesetpriority"
  operator: "Equal"
  value: "spot"
  effect: "NoSchedule"
# Use node affinity to target spot nodes explicitly
```

Spot VMs: 60-80% cheaper, can be evicted with 30 seconds notice when Azure needs capacity. Design workloads for eviction: use Pod Disruption Budgets to ensure minimum replicas survive eviction, use spot for stateless horizontal workloads (batch processing, CI runners, ML training). Never run stateful services on spot without replication. Use multiple VM sizes in the spot pool to increase capacity availability. SPOT_INSTANCE_TERMINATION_NOTICE environment variable (via metadata service) gives 30 seconds warning — use a SIGTERM handler to checkpoint state.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q27 What is Azure Service Mesh (Istio add-on) — when do you enable it on AKS? Networking

🎙 Spoken answer · ~83 s

The Istio-based service mesh add-on for AKS is a managed installation of Istio, where Microsoft handles lifecycle, upgrades, and support for the control plane. A service mesh injects sidecar proxies, or in ambient mode, per-node components, which handle traffic between services. It provides mutual TLS for encrypted, authenticated service-to-service traffic; traffic management such as retries, timeouts, circuit breaking, canary and weighted routing, and fault injection; authorisation policies between services; and consistent metrics, logs, and traces for every call. Enable it when you need these capabilities across many services consistently without changing application code: zero-trust requirements, fine-grained traffic control for progressive delivery, or deep telemetry in a polyglot environment. Avoid or defer it when you have few services, no strong need for mTLS or advanced routing, or limited operational capacity, since a mesh adds resource overhead for proxies, latency per hop, control plane complexity, upgrade effort, and debugging difficulty. Lighter alternatives: network policies, Cilium-based features, ingress with TLS, and libraries for retries. If you adopt it: enable it for a few namespaces first, pilot with non-critical services, measure overhead, manage the ingress gateway, plan revision-based upgrades, and monitor the control plane. Ensure the team has the skills to operate it.

Likely follow-ups

- What does mTLS in a mesh give you?
- What overheads should you weigh before enabling the mesh?

```
# Enable Istio service mesh on AKS:
```

```
az aks mesh enable --resource-group rg --name prod-aks
```

```
az aks mesh enable-ingress-gateway --resource-group rg --name prod-aks --ingress-gateway-type external
```

```
# Enable sidecar injection for namespace:
```

```
kubectl label namespace production istio.io/rev=asm-1-18
```

```
# Traffic management:
```

```
kubectl apply -f virtual-service.yaml   # canary: 90/10 traffic split
```

```
kubectl apply -f destination-rule.yaml  # circuit breaker, mTLS
```

Service mesh adds: mTLS between services (zero-trust networking), traffic management (canary, circuit breaker, retries), and L7 observability (per-service golden signals, distributed traces). AKS Istio add-on: managed Istio (Azure updates the control plane). When to enable: you need mTLS for compliance, advanced

certs.eknathalabs.com

traffic management (canary without redeployment), or need L7 metrics between services without code instrumentation. Cost: every pod gets a sidecar proxy — 15-20% overhead in CPU/memory. Not worth it for small clusters or simple architectures.

📚 **Official docs**

- Istio documentation
- Azure Kubernetes Service (AKS)

Q28 How does Azure Backup work — how do you protect AKS data?

Storage

🎙 **Spoken answer · ~85 s**

Azure Backup protects workloads with policy-based backups stored in a Recovery Services vault or Backup vault, handling scheduling, retention, encryption, and restore. For AKS, Azure Backup for AKS, built on Velero-like technology, backs up cluster state, such as Kubernetes resources in selected namespaces, and persistent volume data using Azure Disk snapshots, to a Backup vault and a storage account. You install the backup extension in the cluster, grant trusted access, create a backup policy for schedule and retention, and configure backup instances; you can restore to the same or a different cluster, selecting namespaces or resources, which supports migration and disaster recovery. Important points: what you need to protect is the persistent data and the declarative configuration. Stateless workloads are better recreated from Git through GitOps rather than backed up. Databases should use their own native or managed backup, such as Azure SQL automated backups, because a disk snapshot may not be application-consistent. Use pre and post hooks for consistency where needed. Protect the backups: soft delete, immutability, and cross-region restore or geo-redundant storage for regional disasters, plus role separation so a compromised cluster can't delete backups. Test restores regularly, measure the recovery time, document runbooks, monitor job success, and align retention with compliance requirements.

Likely follow-ups

- Why are stateless workloads better recreated from Git than backed up?
- Why test restores regularly?

```
# Azure Backup for AKS (preview/GA depending on region):
```

```
az dataprotection backup-vault create --resource-group rg --vault-name myvault --storage-settings type=LocallyRedundant
```

```
az dataprotection backup-policy create --resource-group rg --vault-name myvault --policy file://aks-policy.json
```

```
# Velero (open-source, multi-cloud alternative):
```

```
velero install --provider azure --plugins velero/velero-plugin-for-microsoft-azure:v1.8 \
```

certs.eknathalabs.com

```
  --bucket velero-backups --secret-file ./credentials-velero
```

```
velero backup create prod-backup --include-namespaces production
```

```
velero restore create --from-backup prod-backup
```

AKS data protection: Azure Backup for AKS (persistent volume snapshots + Kubernetes resource backup), Velero (open-source, backs up K8s resources and PVs to Azure Blob). Backup strategy: take daily snapshots of PVs, backup Kubernetes manifests (or rely on GitOps — manifests are already in Git), test restore quarterly. For stateful workloads: the database backup policy (RTO/RPO) usually determines the backup frequency more than the Kubernetes backup policy.

📚 **Official docs**

- Azure Backup
- Azure Kubernetes Service (AKS)

Q29 How do you investigate high memory usage in an AKS cluster?

Troubleshooting

🎙 **Spoken answer · ~88 s**

I begin by establishing whether the memory pressure is at node level or pod level, and whether it's a leak, a limit misconfiguration, or genuine demand. Use kubectl top nodes and pods, and Container Insights or Prometheus dashboards to see memory usage trends per node, namespace, and pod. Check for OOMKilled in pod status and kernel OOM events on nodes, and node conditions for MemoryPressure and evictions. Compare usage to requests and limits: pods without requests can overcommit nodes, and limits too low cause OOM kills even when nodes have free memory. Look at the memory shape over time: a steadily rising line suggests a leak, a sawtooth suggests garbage collection, and spikes correlate with traffic or batch jobs. Identify top consumers, including system components and daemonsets; sidecars and agents can add up. For the application, use language-specific tools: heap dumps and profilers for Java or .NET, tracking of unbounded caches, connection leaks, and large payload buffering. In JVM containers, check heap settings relative to container limits, since misconfigured heaps lead to OOM kills. Remediate: fix the leak, tune caches and heap sizing, set correct requests and limits based on observed usage, use VPA recommendations, add nodes or larger VM sizes, spread pods using anti-affinity, and set alerts for memory saturation.

Likely follow-ups

- How do you tell a memory leak from normal garbage collection behaviour?
- Why can a JVM be OOM-killed even if the node has memory?

```
# Node-level memory:
```

```
kubectl top nodes                                  # memory usage %
```

certs.eknathalabs.com

```
kubectl describe node mynode | grep -A10 Allocated  # memory requests vs capacity
```

```
# Pod-level memory:
```

```
kubectl top pods -n prod --sort-by=memory      # top memory consumers
```

```
kubectl describe pod myapp-xxx | grep -A5 Limits  # configured limits
```

```
# OOMKill events:
```

```
kubectl get events -n prod | grep OOMKill
```

```
# Container Insights KQL:
```

```
KubePodInventory | join KubeNodeInventory on Computer | where OomKilled == true
```

```
# Node memory saturation (leaves no room for burst):
```

```
container_memory_working_set_bytes / container_spec_memory_limit_bytes > 0.9
```

Memory investigation path: identify top memory-consuming pods (kubectl top pods), compare actual usage against memory limits (if close to limit, OOMKill risk), check if any pods were recently OOMKilled (kubectl get events), and verify memory requests are accurate (over-requested memory causes node waste, under-requested causes OOMKill). Increase memory limits for legitimate high-memory workloads. For memory leaks: take heap dumps, analyse with application-specific profiling tools. Node memory pressure (allocatable - requests) must remain positive or new pods cannot schedule.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q30 Walk through diagnosing a production AKS performance issue reported by users.

Troubleshooting

🎙 Spoken answer · ~86 s

I diagnose a user-reported AKS performance problem by first scoping and then narrowing down layer by layer. Clarify the symptom: which user actions, which endpoints, since when, how many users, which regions, and what changed recently, like deployments or traffic. Check the golden signals at the entry point: ingress or Application Gateway metrics for latency, error rates, and backend response time, using percentiles. Use Application Insights or distributed traces to find the slow component. Then check each layer. Application pods: CPU throttling from limits, memory pressure and OOM kills, restarts, thread or connection pool exhaustion, and HPA behaviour; are there enough replicas, and are they all ready? Nodes: CPU, memory, disk, and network saturation, noisy neighbours, and node pressure conditions; check scheduling and pending pods. Dependencies: database performance, using query insights, locks, and DTU or CPU limits; external APIs; caches; and DNS latency from CoreDNS. Network: ingress path, NAT or firewall SNAT port exhaustion, cross-zone traffic, and network policies. Platform: Azure service health and throttling. Correlate with deployments and configuration changes. Mitigate quickly, by rolling back a bad release, scaling out, or shedding load, then fix the root cause, whether a query, a limit, or a bottleneck. Communicate status, confirm with metrics, and write a post-incident review.

Likely follow-ups

- What is SNAT port exhaustion and why does it cause latency?
- Why check CPU throttling when CPU usage looks low?

Principal-level systematic diagnosis — cover the full investigation flow.

certs.eknathalabs.com

```
Step 1: Quantify — error rate, P99 latency, affected users (Azure Monitor)
```

```
Step 2: Golden signals — which signal is anomalous? (Grafana or Insights)
```

```
Step 3: Recent changes — kubectl rollout history, recent CI/CD deployments
```

```
Step 4: Pod health — kubectl get pods -n prod (any Pending/CrashLoop?)
```

```
Step 5: Node health — kubectl get nodes (all Ready?)
```

```
Step 6: CPU throttling — container_cpu_cfs_throttled_seconds_total
```

```
Step 7: Database — RDS/Cosmos DB slow query logs, connection pool exhaustion
```

```
Step 8: Downstream — is a dependency (external API, cache) slow?
```

```
Step 9: Network — NSG blocking? Private Endpoint DNS resolution?
```

```
Step 10: Rollback if recent change — restore known-good state first
```

The most common production AKS performance issues: CPU throttling (invisible, silent), memory pressure causing pod eviction, HPA not scaling fast enough (scale-up latency), Cluster Autoscaler slow to provision nodes, database connection pool exhaustion, and cross-AZ data transfer causing latency. Rollback first if there was a recent deployment — restore the known-good state, then diagnose at leisure without user impact.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q31 How do you design hub-and-spoke network topology in Azure for multiple workloads?

Networking

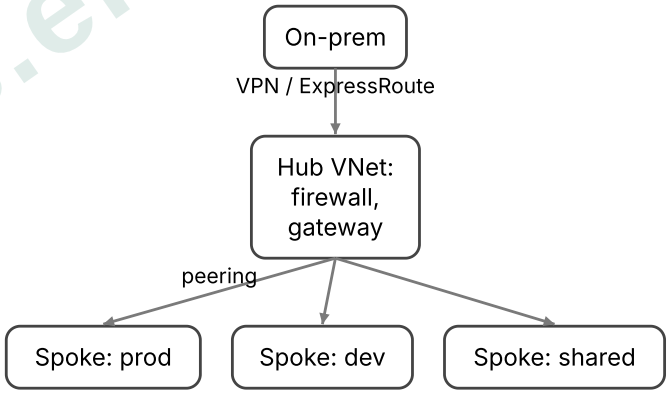
🎙 Spoken answer · ~83 s

Hub-and-spoke centralises shared services in a hub virtual network, with workloads isolated in spoke networks peered to the hub. The hub typically hosts Azure Firewall or a network virtual appliance, VPN or ExpressRoute gateways for on-premises connectivity, Azure Bastion, DNS resolvers, and shared services. Each spoke is a separate VNet, per workload or environment, often in its own subscription, peered to the hub; spokes don't peer with each other directly, so inter-spoke traffic flows through the hub firewall, which gives central inspection, logging, and policy. To route traffic through the firewall, apply user-defined routes in spoke subnets, with a default route to the firewall's private IP, and enable gateway transit so spokes can use the hub's gateways. Benefits: shared cost for expensive components, central security control, isolation between workloads, and a scalable pattern. Design points: plan non-overlapping address space; use private DNS zones linked to the hub or spokes, with a central resolver for private endpoints; decide on egress control and outbound internet through the firewall; use network security groups within spokes for segmentation; and monitor with flow logs and firewall logs. For very large or global estates, Azure Virtual WAN provides a managed hub-and-spoke with automated routing.

Likely follow-ups

- Why route inter-spoke traffic through the hub?
- When would you choose Virtual WAN?

🗺 Hub and spoke




```
# Hub VNet (shared services: firewall, VPN/ExpressRoute, DNS):
```

```
az network vnet create -g net-rg -n hub-vnet --address-prefix 10.0.0.0/16
```

```
# Spoke VNets (per workload/environment):
```

```
az network vnet create -g app-rg -n spoke-prod --address-prefix 10.1.0.0/16
```

certs.eknathalabs.com

```
# Peer spoke to hub (and hub to spoke):
az network vnet peering create -g app-rg --vnet-name spoke-prod -n to-hub \\
  --remote-vnet hub-vnet --allow-vnet-access --allow-forwarded-traffic
# Route spoke egress through hub firewall via UDR:
az network route-table route create -g net-rg --route-table-name spoke-rt \\
  -n default --address-prefix 0.0.0.0/0 --next-hop-type VirtualAppliance \\
  --next-hop-ip-address 10.0.1.4   # Azure Firewall private IP
```

Hub-and-spoke centralises shared services (Azure Firewall, VPN/ExpressRoute gateways, DNS) in a **hub** VNet, with workloads isolated in **spoke** VNets peered to it. Spokes do not peer to each other (transitive peering is not automatic) — inter-spoke traffic routes through the hub firewall via User-Defined Routes, giving a central inspection and policy point. This is the standard enterprise topology: it isolates blast radius per spoke, centralises egress control and logging, and scales by adding spokes.

Interview tip: the key facts are that VNet peering is *non-transitive* (spokes need the hub to talk, forced via UDR to the firewall) and that the hub centralises egress/inspection. Mentioning Azure Virtual WAN as the managed evolution of hub-and-spoke shows current knowledge.

> 📚 **Official docs**
>
> - Azure hub-spoke network topology

certs.eknathalabs.com

Q32 What is Azure Firewall vs NSG vs Application Gateway WAF — when do you use each?

Networking

🎙 Spoken answer · ~87 s

These three control traffic at different points and layers, and are often used together. Network Security Groups are basic, stateful, distributed layer three and four filters on subnets and network interfaces, allowing or denying by source, destination, port, and protocol, with service tags and application security groups. They're free, simple, and the baseline for segmentation inside a VNet, but have no application awareness or central management. Azure Firewall is a managed, central, stateful firewall service, with layer three to seven features: network and application rules with FQDN filtering, threat intelligence, TLS inspection and intrusion detection in the premium tier, and central policy. Use it in the hub for egress control and east-west inspection between spokes, with logging. Application Gateway with WAF operates at layer seven for inbound HTTP and HTTPS traffic to web applications, with TLS termination, path routing, and protection against OWASP threats like SQL injection and cross-site scripting. Use it as the entry point for web workloads. So: NSGs for micro-segmentation everywhere; Azure Firewall for centralised outbound and inter-network traffic control; WAF for inbound web application protection. A typical flow combines them: Front Door or Application Gateway WAF inbound, Azure Firewall in the hub for egress and inter-spoke traffic, NSGs on each subnet. Layered defence gives depth.

Likely follow-ups

- Why use both NSGs and Azure Firewall?
- Which service protects against SQL injection?

```
NSG:           L3/L4 stateful allow/deny on subnet/NIC (free, basic)
```

```
Azure Firewall: L3-L7 managed firewall, FQDN filtering, threat intel, central egress
```

```
App Gateway WAF: L7 web protection (OWASP), SQLi/XSS, for HTTP(S) apps
```

```
# NSG rule:
```

```
az network nsg rule create -g rg --nsg-name aks-nsg -n allow-https \\
```

```
  --priority 100 --destination-port-ranges 443 --access Allow --protocol Tcp
```

```
# Azure Firewall app rule (FQDN egress):
```

```
az network firewall application-rule create -g rg -f myfw --collection-name egress \\
```

```
  -n allow-mcr --protocols Https=443 --target-fqdns mcr.microsoft.com --action Allow
```

certs.eknathalabs.com

These operate at different layers and are complementary, not alternatives. **NSGs** are free, basic L3/L4 stateful filters attached to subnets/NICs — your first line for segment-to-segment rules. **Azure Firewall** is a managed, scalable L3–L7 firewall with FQDN filtering, threat intelligence, and centralised egress control (placed in the hub). **Application Gateway WAF** protects HTTP(S) apps from L7 attacks (OWASP rules: SQLi, XSS). A typical stack: NSGs for micro-segmentation, Azure Firewall for centralised egress in the hub, WAF for inbound web traffic.

Interview tip: frame them by layer and purpose — NSG (cheap L4 segmentation), Azure Firewall (managed L7 egress/inspection, FQDN rules), WAF (inbound web-app protection). Knowing they layer together (defence in depth) rather than compete is the senior framing.

📚 **Official docs**

- Azure Network security groups
- Azure Firewall
- Azure Application Gateway

Q33 How does DNS resolution work for Private Endpoints, and what breaks if it is misconfigured?

Networking

🎙 **Spoken answer · ~88 s**

Private Endpoints give a service a private IP in your VNet, but clients usually connect by name, so DNS must resolve the service's normal name to that private IP. The standard setup uses Azure Private DNS zones, such as privatelink.blob.core.windows.net, linked to the VNets that need resolution. When you create a private endpoint, a DNS record in that zone maps the resource name to its private IP; the public DNS name has a CNAME to the privatelink name, so with the private zone linked, clients inside resolve the private address, while others resolve the public one. For on-premises clients, you need a DNS forwarder or Azure DNS Private Resolver in Azure; on-premises DNS forwards the relevant zones to it, so the private zone is consulted. In hub-and-spoke designs, link the zones to the hub where the resolver lives. What breaks when misconfigured: the name resolves to the public IP, so traffic goes to the public endpoint and fails if public access is disabled, with errors that look like authorisation or connectivity failures; different networks resolve differently, causing intermittent failures; stale records after recreating endpoints; zones not linked to the right VNet; conflicting zones; and custom DNS servers not forwarding to Azure's resolver at 168.63.129.16. Troubleshoot with nslookup from the client, and verify.

Likely follow-ups

- What does a client resolve if the private DNS zone isn't linked to its VNet?
- How do on-premises clients resolve private endpoint names?

```
# Private Endpoint gives the PaaS service a private IP in your subnet.
```

```
# A Private DNS Zone maps the service FQDN to that private IP:
```

certs.eknathalabs.com

```
az network private-dns zone create -g rg -n privatelink.database.windows.net
```

```
az network private-dns link vnet create -g rg -z privatelink.database.windows.net \\
```

```
  -n vnet-link --virtual-network prod-vnet --registration-enabled false
```

```
# Verify resolution from inside the VNet:
```

```
nslookup mydb.database.windows.net   # MUST return 10.x.x.x (private), not public IP
```

A Private Endpoint creates a private IP for a PaaS service, but clients still connect using the public FQDN (e.g. `mydb.database.windows.net`). The magic is a **Private DNS Zone** (e.g. `privatelink.database.windows.net`) linked to your VNet: it overrides resolution so the FQDN returns the private IP instead of the public one. If the zone is missing or not linked to the VNet, the FQDN resolves to the public IP — and if the service has public access disabled, the connection fails entirely.

Interview tip: the classic failure is "Private Endpoint created, but the app still cannot connect" — almost always missing/unlinked Private DNS Zone, so the FQDN resolves public while public access is off. Always verify with `nslookup` from inside the VNet returning a private IP. This is one of the most common real-world Azure networking incidents.

> 📚 **Official docs**
>
> - RFC 1035: DNS implementation and specification
> - Cloudflare Learning: What is DNS?
> - Azure Private Endpoint

certs.eknathalabs.com

Q34 What is the difference between Azure Load Balancer, Application Gateway, and Front Door?

Networking

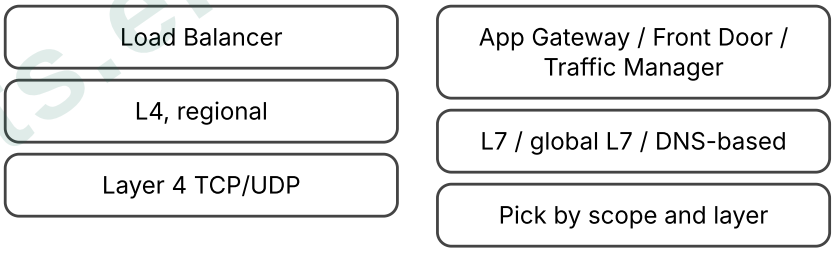
🎙 Spoken answer · ~86 s

The three differ in layer and scope. Azure Load Balancer operates at layer four, handling TCP and UDP, with very high throughput and low latency. It's regional, comes in public and internal flavours, and distributes flows with health probes. It doesn't understand HTTP. Use it for non-HTTP protocols, internal tiers, and as the basis for AKS service type LoadBalancer. A Gateway Load Balancer variant chains network virtual appliances. Application Gateway is a regional layer-seven load balancer for HTTP and HTTPS, with URL and host-based routing, SSL termination, cookie affinity, rewrite rules, autoscaling, and an optional web application firewall. Use it as the entry point for web applications within a region, and as an AKS ingress. Azure Front Door is a global layer-seven service at the Microsoft edge, with anycast, global load balancing across regions, caching, acceleration, TLS offload, WAF, and health-based failover. Use it for globally distributed web applications. Also Traffic Manager is DNS-based global routing, for any protocol, but without proxying. Selection: non-HTTP or raw performance, Load Balancer; regional web routing with WAF, Application Gateway; global web with edge acceleration and failover, Front Door. They compose: Front Door to regional Application Gateways to Load Balancer-backed pods or VMs. Consider cost, features, latency, and operational overhead.

Likely follow-ups

- How does Front Door differ from Traffic Manager?
- When is Azure Load Balancer sufficient?

🗺 Azure load balancers




```
Azure Load Balancer:  L4 (TCP/UDP), regional, ultra-low latency, no app awareness
```

```
Application Gateway:   L7 (HTTP/S), regional, WAF, URL routing, SSL termination
```

```
Front Door:            L7, GLOBAL, CDN + edge WAF, multi-region routing/failover
```

```
# Common enterprise chain:
```

```
# Front Door (global edge) -> App Gateway (regional WAF) -> AKS internal LB -> pods
```

certs.eknathalabs.com

They differ by OSI layer and scope. **Load Balancer** is L4 (TCP/UDP), regional, extremely fast, with no HTTP awareness — use for non-HTTP or raw performance. **Application Gateway** is L7, regional, with WAF, path/host routing, and SSL termination — use for HTTP apps needing web features in one region. **Front Door** is L7 but **global**: a CDN with edge PoPs that routes users to the nearest healthy region, with edge WAF and global failover — use for a worldwide user base or multi-region HA. They compose: Front Door for global routing/failover into per-region App Gateways.

Interview tip: the discriminators are *layer* (L4 vs L7) and *scope* (regional vs global). "Global users / multi-region failover" → Front Door; "regional web app with WAF" → App Gateway; "raw TCP/low latency" → Load Balancer. Naming the Front Door → App Gateway → AKS chain shows real architecture experience.

📚 **Official docs**

- Cloudflare Learning: Load balancing
- Azure Load Balancer
- Azure Application Gateway

Q35 How do you connect on-premises networks to Azure — VPN Gateway vs ExpressRoute?

Networking

🎙 **Spoken answer · ~88 s**

There are two main ways to link on-premises networks to Azure. VPN Gateway creates encrypted IPsec and IKE tunnels over the public internet, either site-to-site from a network, or point-to-site for individual clients. It is quick to set up, inexpensive, and flexible, with bandwidth depending on the gateway SKU, up to a few gigabits, and latency that varies with the internet path. It supports active-active gateways and zone-redundant SKUs for resilience, and BGP for dynamic routing. ExpressRoute provides a private, dedicated connection through a connectivity provider that doesn't traverse the public internet: predictable low latency, higher bandwidth up to one hundred gigabits, higher reliability with an SLA, and access to Microsoft services via private and Microsoft peering. It involves provider lead time and higher monthly cost. Choose VPN for smaller workloads, quick starts, development environments, remote users, or as a backup; choose ExpressRoute for production hybrid workloads needing consistent performance, large data transfers, or compliance reasons. Best practice for critical systems: ExpressRoute as primary, with redundant circuits in different peering locations, and VPN as failover over the same hub; ExpressRoute Global Reach connects on-premises sites through Microsoft's backbone. Terminate both in the hub of a hub-and-spoke or Virtual WAN, and plan address space and BGP routes carefully. Monitor the connection health.

Likely follow-ups

- Why keep a VPN as backup for ExpressRoute?
- What does ExpressRoute give that VPN cannot?

```
# Site-to-Site VPN (over public internet, IPsec):
```

certs.eknathalabs.com

```
az network vnet-gateway create -g rg -n vpn-gw --vnet hub-vnet \\
```

```
  --gateway-type Vpn --vpn-type RouteBased --sku VpnGw2
```

```
# ExpressRoute (private dedicated circuit via a provider):
```

```
az network express-route create -g rg -n er-circuit \\
```

```
  --bandwidth 1000 --provider Equinix --peering-location "Silicon Valley" --sku-tier
Standard
```

VPN Gateway connects on-prem to Azure over the public internet via encrypted IPsec tunnels — quick to set up, lower cost, but subject to internet variability and lower throughput. **ExpressRoute** is a private, dedicated circuit through a connectivity provider that bypasses the public internet entirely — consistent low latency, high bandwidth (up to 100 Gbps), and an SLA. Use VPN for dev/test, smaller workloads, or as a backup path; use ExpressRoute for production, latency-sensitive, high-throughput, or compliance-driven connectivity. A common HA design uses ExpressRoute primary with VPN as failover.

Interview tip: the trade-off is internet-based (VPN: cheap, fast to deploy, variable) vs private circuit (ExpressRoute: consistent, high-bandwidth, SLA, more cost/lead-time). Mentioning ExpressRoute-primary + VPN-failover as a resilient hybrid pattern is a strong senior detail.

📚 **Official docs**

- Cloudflare Learning: What is a VPN?
- Azure ExpressRoute
- Azure VPN Gateway

certs.eknathalabs.com

Q36 What are Azure Management Groups and how do you structure subscriptions at enterprise scale?

IAM & EntraID

🎙 Spoken answer · ~89 s

Management groups sit above subscriptions in a hierarchy, up to six levels deep under the tenant root group, allowing governance to be applied once and inherited by everything beneath. You can assign Azure Policy and RBAC at a management group and it flows to all child subscriptions. At enterprise scale, the structure follows Microsoft's Cloud Adoption Framework and the Azure landing zone model: a top-level organisation group, with child groups for platform, such as identity, management, and connectivity; landing zones, split into corporate and online, or by workload sensitivity; sandbox for experimentation with relaxed policy and budgets; and decommissioned for retired subscriptions. Subscriptions are the unit of scale, billing, and isolation: use separate subscriptions per workload or environment, such as production and non-production, because they provide limits, access boundaries, and cost separation. Apply policies at the right level: security baselines and allowed regions at the top, workload-specific rules lower. Use RBAC with groups at management group or subscription scope, with PIM for privileged roles. Avoid deep or overly complex hierarchies organised by organisational chart, which changes; design around policy and compliance needs instead. Automate subscription vending so new landing zones are created consistently with networking, policies, and monitoring baked in. Keep the hierarchy documented, and use infrastructure as code for management groups and assignments.

Likely follow-ups

- Why use separate subscriptions per workload or environment?
- What is subscription vending?

```
# Management Group hierarchy (governance applied top-down):
```

```
az account management-group create --name Corp --display-name Corporate
```

```
az account management-group create --name Prod --parent Corp
```

```
az account management-group create --name NonProd --parent Corp
```

```
# Move a subscription under a management group:
```

```
az account management-group subscription add --name Prod --subscription SUB_ID
```

```
# Policies/RBAC assigned at MG level inherit to all child subscriptions
```

Management Groups sit above subscriptions, forming a hierarchy (Tenant Root → Management Groups → Subscriptions → Resource Groups) where Azure Policy and RBAC assignments inherit downward. The enterprise pattern (Cloud Adoption Framework "landing zones") groups subscriptions by purpose — e.g.

certs.eknathalabs.com

separate Platform, Prod, and NonProd management groups — so governance (allowed regions, required tags, security baselines) is defined once at the group level and automatically applies to every subscription beneath it. Subscriptions become units of isolation, billing, and scale limits.

Interview tip: the key insight is inheritance — assign policy/RBAC at the management-group level so it cascades, rather than per-subscription. Referencing the Azure landing-zone / Cloud Adoption Framework structure (platform vs workload subscriptions under management groups) signals enterprise governance maturity.

📚 **Official docs**

- Azure management groups
- RHEL 9 documentation

Q37 What is Conditional Access in Entra ID and how does it strengthen security?

IAM & EntraID

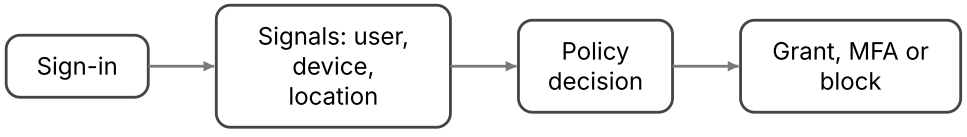
🎙 **Spoken answer · ~88 s**

Conditional Access is the policy engine in Entra ID that evaluates signals at sign-in time and decides whether to grant access, require additional controls, or block. It works as if-then rules. The signals include user or group, application, device state and compliance, location or network, client app, and sign-in or user risk from Identity Protection. The decisions: block access; grant with requirements such as multi-factor authentication, a compliant or hybrid-joined device, an approved client app, terms of use, or a password change. Session controls can limit what a user does inside an app, restrict persistent sessions, and enforce sign-in frequency. How it strengthens security: it enforces zero-trust principles by verifying explicitly rather than trusting network location. Typical policies: require MFA for all users, and always for administrators; block legacy authentication, which can't do MFA; require compliant devices for sensitive apps; block or step up for risky sign-ins; restrict access from untrusted countries; and require MFA to register security info. Rollout practice: start in report-only mode to see the impact, exclude break-glass emergency accounts that are monitored closely, apply to groups, use named locations, and test with the what-if tool. Combine with PIM for privileged roles, and monitor sign-in logs. Keep policies simple and documented, and avoid lockouts by designing carefully.

Likely follow-ups

- Why block legacy authentication?
- What is report-only mode for?

🗺 **Conditional Access**



certs.eknathalabs.com

```
# Conditional Access policy intent (configured in Entra ID):
#   IF user accesses [Azure Management]
#   AND sign-in risk is medium/high OR location is outside corp network
#   THEN require MFA AND a compliant/managed device
#   ELSE block
# Common controls: require MFA, require compliant device, block legacy auth,
#   restrict by location/IP, require approved client app
```

Conditional Access is Entra ID's policy engine that evaluates signals at sign-in — user/group, application, device state, location, and real-time risk — and enforces controls like requiring MFA, demanding a compliant managed device, or blocking access. Instead of "always require MFA," it applies controls contextually: low-risk sign-in from a managed device on the corporate network passes seamlessly, while a risky sign-in from an unknown location is challenged or blocked. Blocking legacy authentication protocols (which bypass MFA) is one of the highest-impact policies.

Interview tip: frame it as risk-based, context-aware access — the foundation of Zero Trust on Azure. The single most impactful policy to mention is *blocking legacy auth*, since it closes the gap attackers use to bypass MFA. Pairing Conditional Access with Identity Protection (risk signals) is the complete answer.

> 📚 **Official docs**
>
> - Microsoft Entra ID: What is Microsoft Entra ID?
> - Microsoft Entra: Conditional Access

certs.eknathalabs.com

Q38 How do Azure RBAC and Kubernetes RBAC interact on AKS?

IAM & EntraID

🎙 Spoken answer · ~89 s

AKS has two authorisation layers, and how they combine depends on configuration. Azure RBAC controls access to the Azure resource, the cluster, through the control plane: who can list cluster credentials, upgrade the cluster, scale node pools, or delete it. Kubernetes RBAC controls access to objects inside the cluster through the Kubernetes API: who can get pods, create deployments, or read secrets in a namespace, using Roles, ClusterRoles, and bindings. By default these are separate: Azure roles like Contributor manage the resource but don't automatically grant Kubernetes data access, although some roles allow retrieving admin credentials, which effectively grants cluster-admin. Entra ID integration lets users authenticate to the Kubernetes API with their Entra identities, and you can map Entra groups to Kubernetes RoleBindings. Alternatively, Azure RBAC for Kubernetes authorisation lets you manage Kubernetes permissions through Azure role assignments, using built-in roles such as Azure Kubernetes Service RBAC Reader, Writer, Admin, and Cluster Admin, scoped to the cluster or a namespace, giving one place to manage access with Azure tooling, PIM, and auditing. Best practice: enable Entra ID integration and Azure RBAC for Kubernetes, disable local accounts so no static admin credentials exist, use groups, apply least privilege and namespace scoping, restrict who can fetch credentials, use PIM for elevated access, and audit with logs.

Likely follow-ups

- Why disable local accounts on AKS?
- What does Azure RBAC for Kubernetes authorisation offer?

```
# Two RBAC layers on AKS:
```

```
# 1. Azure RBAC: who can manage the CLUSTER resource (az aks ...)
```

```
az role assignment create --assignee user@co.com \\
```

```
  --role "Azure Kubernetes Service Cluster User Role" --scope $AKS_ID
```

```
# 2. Kubernetes RBAC: what you can do INSIDE the cluster (kubectl)
```

```
# Option A: AKS-managed Entra integration maps Entra groups -> K8s RBAC
```

```
az aks update -g rg -n prod-aks --enable-azure-rbac   # use Azure RBAC for K8s authz too
```

```
# Then assign K8s-scoped Azure roles:
```

```
az role assignment create --assignee $GROUP_ID \\
```

```
  --role "Azure Kubernetes Service RBAC Reader" --scope $AKS_ID/namespaces/prod
```

certs.eknathalabs.com

AKS has two distinct authorization layers. **Azure RBAC** controls operations on the cluster *resource* (creating/scaling the cluster, getting credentials). **Kubernetes RBAC** controls actions *inside* the cluster (which pods/secrets a user can read). With Entra integration, users authenticate via Entra ID and their group membership maps to Kubernetes Roles/RoleBindings. Enabling "Azure RBAC for Kubernetes Authorization" lets you manage even in-cluster permissions through Azure role assignments scoped to namespaces — unifying both layers under Entra/Azure RBAC.

Interview tip: the crisp distinction is "Azure RBAC = permission to manage the cluster object; Kubernetes RBAC = permission to act on objects within the cluster." Note that getting a kubeconfig (Cluster User role) does *not* grant in-cluster permissions — K8s RBAC still gates that. Mapping Entra groups to namespaces is the production pattern.

📚 **Official docs**

- Kubernetes: RBAC
- Azure RBAC overview
- Azure Kubernetes Service (AKS)

Q39 What is Privileged Identity Management (PIM) and why use just-in-time access?

IAM & EntraID

🎙 **Spoken answer · ~88 s**

Privileged Identity Management, part of Entra ID governance, provides just-in-time, time-bound, and approval-based activation of privileged roles, for both Entra directory roles and Azure resource roles. Instead of users holding standing permanent admin rights, they are made eligible; when they need the access, they activate the role for a limited duration, such as one to eight hours, optionally needing MFA, a justification, a ticket number, and approval from designated approvers. After the time expires, the permission is removed automatically. Why it matters: standing privileged access is a major risk because a compromised account immediately yields powerful rights, and administrators rarely need them continuously. JIT dramatically shrinks the exposure window and the attack surface, forces deliberate use, and creates an audit trail of who activated what and why. PIM also offers access reviews to periodically recertify who is eligible, alerts for risky patterns such as roles being activated outside normal behaviour or too many permanent assignments, and the ability to require approval for the most sensitive roles. Implementation: inventory privileged roles, convert permanent assignments to eligible, configure activation settings by sensitivity, keep one or two monitored emergency break-glass accounts outside the process, set up reviews, and monitor. For Azure resources, use PIM on subscription or management group scope roles like Owner.

Likely follow-ups

- What is a break-glass account and why exclude it?
- How does JIT access reduce risk from compromised accounts?

```
# PIM makes privileged roles ELIGIBLE rather than permanently ACTIVE:
```

certs.eknathalabs.com

```
# - user is 'eligible' for Owner/Contributor, not standing-assigned
```

```
# - to use it: activate the role (time-boxed, e.g. 4 hours)
```

```
# - activation can require: MFA, justification, approval, ticket number
```

```
# Eligible assignment via Graph/PIM (conceptual):
```

```
#   role: Contributor  scope: prod-sub  type: Eligible  maxDuration: PT4H
```

PIM implements just-in-time, time-bound privileged access. Instead of users holding standing admin rights (a constant attack surface), they are made *eligible* for a role and must **activate** it on demand — with optional MFA, business justification, approval workflow, and an automatic expiry. This shrinks the window during which privileged credentials are exploitable, creates an audit trail of every elevation, and enforces least privilege over time. It is central to limiting the blast radius of a compromised admin account.

Interview tip: the security principle is "no standing privilege." PIM converts permanent admin into time-boxed, justified, approved, audited elevation. Mentioning that activation can require approval + MFA + a ticket, and that everything is logged, demonstrates you understand privileged-access security, not just RBAC basics.

> 📚 **Official docs**
>
> - Microsoft Entra: Privileged Identity Management

certs.eknathalabs.com

Q40 How do you architect a private (API-server-isolated) AKS cluster and still allow CI/CD access?

Networking

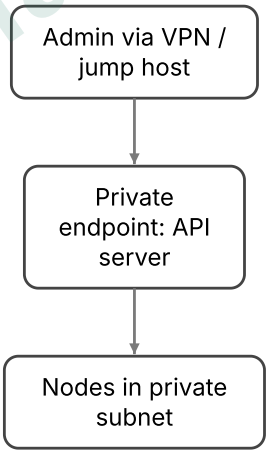
🎙 Spoken answer · ~83 s

A private AKS cluster exposes the API server only through a private endpoint in your VNet, so it has no public IP; the control plane and nodes communicate over private networking. That removes internet exposure of the API, but means CI/CD and administrators must reach it from inside the network. Options. Self-hosted build agents or runners placed in the VNet or a peered network, such as Azure DevOps self-hosted agents or GitHub runners on VMs or in the cluster itself, with network access to the API server's private IP. Access from on-premises or admin machines through VPN or ExpressRoute, with DNS configured so the cluster's private FQDN resolves to the private endpoint, via a private DNS zone linked to the right VNets and forwarders. An Azure Bastion or jump host in the network for administrators. The az aks command invoke feature lets you run kubectl or helm commands against the cluster through the Azure control plane API, without direct network connectivity, using Azure RBAC, which works well for pipelines using hosted agents. GitOps avoids the problem: Flux or ArgoCD inside the cluster pulls from Git, and CI only updates the repository, so no inbound access to the API is needed.

Likely follow-ups

- How does az aks command invoke avoid network access?
- Why does GitOps fit private clusters well?

🗺 Private AKS




```
# Private cluster: API server gets a private IP, no public endpoint
```

```
az aks create -g rg -n prod-aks --enable-private-cluster \\
```

```
  --private-dns-zone system --enable-managed-identity
```

```
# CI/CD reaches the private API server via one of:
```

```
#  - self-hosted runners inside the VNet (or a peered VNet)
```

certs.eknathalabs.com

```
#  - 'az aks command invoke' (runs kubectl in an AKS-managed pod, no network line-of-sight)
```

```
az aks command invoke -g rg -n prod-aks --command "kubectl get pods -n prod"
```

```
#  - private endpoint + private DNS for the API server FQDN
```

A **private AKS cluster** places the Kubernetes API server on a private IP with no public endpoint, so management traffic never traverses the internet. The challenge is that CI/CD and operators now lack a direct line of sight to the API server. Solutions: run self-hosted CI runners inside the VNet (or a peered/hub VNet), use `az aks command invoke` (which executes your kubectl/helm inside an AKS-managed pod that already has cluster access, requiring no network path), or wire up a Private Endpoint with Private DNS for the API server FQDN so VPN/ExpressRoute-connected clients resolve it privately.

Interview tip: private cluster = no public API endpoint, which forces a decision about how pipelines reach it. The elegant, low-infrastructure answer is `az aks command invoke` (no VNet line-of-sight needed); the scalable answer is in-VNet self-hosted runners. Knowing the Private DNS requirement for the API FQDN rounds it out.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q41 How do you secure secrets end-to-end on Azure: Key Vault, rotation, and access?

IAM & EntraID

🎙 Spoken answer · ~83 s

End-to-end secret security means controlling creation, storage, access, rotation, and auditing. Store secrets in Key Vault, not in code, pipeline variables, or container images, with a vault per application and environment to limit blast radius. Use the RBAC permission model for access, with least-privilege roles such as Key Vault Secrets User for reading, assigned to managed identities or workload identities rather than users or shared service principals. Network: disable public access or restrict it, and use private endpoints, with firewall rules. Protection: enable soft delete and purge protection so secrets can't be permanently destroyed accidentally or maliciously, and use a premium vault with HSM-backed keys where required. Rotation: prefer eliminating secrets altogether through managed identities and Entra authentication to databases and storage. Where secrets remain, set expiry dates, rotate automatically using Event Grid notifications from Key Vault near expiry to trigger a Function that generates a new value and updates the target; some services support built-in automatic rotation. Applications should fetch secrets at runtime, via the CSI driver on AKS or the SDK, and handle rotation without restart where possible. Audit: send Key Vault diagnostic logs to Log Analytics, alert on unusual access or failed attempts, and review access regularly.

Likely follow-ups

- Why prefer managed identities over rotating secrets?
- What do soft delete and purge protection prevent?

```
az keyvault create -g rg -n myvault --enable-rbac-authorization true
```

```
# RBAC (preferred over access policies):
```

```
az role assignment create --assignee $MI_OBJECT_ID \\
```

```
  --role "Key Vault Secrets User" --scope $VAULT_ID
```

```
# Store + rotate:
```

```
az keyvault secret set --vault-name myvault -n db-password --value <v>
```

```
# Auto-rotation policy / event-grid trigger to rotate and update consumers
```

```
# Consume from AKS via CSI driver or Workload Identity (no creds in pod)
```

End-to-end secret security on Azure: store secrets/keys/certs in **Key Vault** with RBAC authorization (granular, auditable — preferred over legacy access policies), grant access to **Managed Identities / Workload Identity** rather than shared credentials, enable **soft-delete and purge protection** so secrets cannot

certs.eknathalabs.com

be permanently lost or maliciously deleted, and automate **rotation** (Key Vault rotation policies or Event Grid-triggered functions that rotate the credential and update dependents). Pods consume secrets via the CSI driver or ESO with no credentials embedded. All access is logged for audit.

Interview tip: hit the full lifecycle — storage (Key Vault, RBAC, purge protection), access (Managed Identity, no shared secrets), rotation (automated, not manual), and consumption (CSI/Workload Identity, nothing in pod specs). Calling out RBAC authorization over access policies, and soft-delete/purge protection, shows current best practice.

📚 **Official docs**

- Azure Key Vault
- HashiCorp Vault documentation

Q42 What causes SNAT port exhaustion on AKS and how do you fix it?

Networking

🎙 **Spoken answer · ~85 s**

SNAT, source network address translation, lets resources without a public IP make outbound connections by sharing a public IP, with each connection using a port from a limited pool. On AKS, outbound traffic through the standard load balancer gets a finite number of SNAT ports per node, by default roughly a thousand per backend instance for the typical allocation. When a node opens many concurrent outbound connections to the same destination IP and port, such as an API or database, ports are consumed, and new connections fail or time out. Ports stay allocated for a time after connections close, so rapid short-lived connections without reuse worsen it. Symptoms: intermittent timeouts or connection failures to external services, errors under load, and metrics showing SNAT connection failures on the load balancer. Fixes. Reduce demand: use connection pooling and keep-alive, reuse HTTP clients, and avoid creating a connection per request. Add capacity: use a NAT Gateway for outbound, which provides up to sixty-four thousand ports per public IP and supports multiple IPs, scaling to hundreds of thousands; or assign more outbound public IPs or a public IP prefix, and tune allocated ports. Avoid internet egress where possible: use private endpoints and service endpoints for Azure services.

Likely follow-ups

- Why does connection reuse reduce SNAT usage?
- How does a NAT Gateway help with port exhaustion?

```
# Symptom: outbound connections intermittently fail/timeout under load
```

```
# Diagnose via Azure Monitor / LB metrics: SNAT port utilization, failed connections
```

```
# Fixes:
```

```
# 1. Increase allocated outbound SNAT ports (Standard LB outbound rule):
```

certs.eknathalabs.com

```
az aks update -g rg -n prod-aks --load-balancer-outbound-ports 2048
```

```
# 2. Add more outbound public IPs (more SNAT ports total):
```

```
az aks update -g rg -n prod-aks --load-balancer-managed-outbound-ip-count 4
```

```
# 3. Use a NAT Gateway for the node subnet (far larger SNAT scale):
```

```
az network nat gateway create -g rg -n aks-nat --public-ip-addresses natip
```

Azure performs **SNAT** (source NAT) for pod egress through the cluster's load balancer, and each public IP provides a limited pool of SNAT ports. Workloads making many concurrent outbound connections (chatty microservices, lots of external API calls) can exhaust the port pool, causing intermittent connection failures and timeouts that are hard to diagnose. Fixes: allocate more SNAT ports per node, add more outbound public IPs (each adds ports), or — the most scalable — attach a **NAT Gateway** to the node subnet, which provides vastly more SNAT ports and better handles bursty egress.

Interview tip: SNAT exhaustion is a classic, hard-to-spot AKS production issue — intermittent outbound failures under load with healthy pods. The senior answer names the symptom, points to LB SNAT metrics, and recommends **NAT Gateway** as the durable fix (plus connection reuse/pooling in the app to reduce port churn).

> 📚 **Official docs**
>
> - RFC 3022: Traditional NAT
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q43 How do you implement Workload Identity on AKS and why is it better than pod-managed identity?

IAM & EntraID

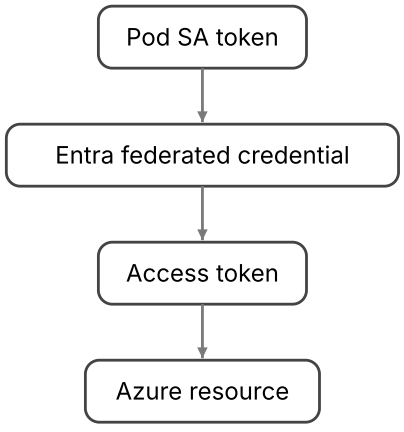
🎙 Spoken answer · ~85 s

Microsoft Entra Workload ID lets a pod authenticate to Azure services as an Entra identity without secrets, using OIDC federation. How it works: the AKS cluster is an OIDC issuer, publishing keys; you create a user-assigned managed identity, or an app registration, and a federated identity credential that trusts tokens issued by the cluster for a specific Kubernetes service account in a namespace. A pod using that service account, annotated with the client ID and labelled for workload identity, receives a projected service account token. The Azure SDK, through the identity libraries, exchanges that token with Entra ID for an access token for the target resource, like Key Vault, Storage, or SQL, and you grant the identity RBAC roles. Setup: enable the OIDC issuer and workload identity add-on on the cluster, create the identity and federation, annotate the service account, and label pods. It's better than the older pod-managed identity, which relied on a node-level NMI component intercepting instance metadata calls, with security weaknesses, scale limits, and complexity, and is deprecated. Workload ID offers per-service-account scoping, no privileged interceptors, works across clusters and clouds through standard OIDC, supports multi-tenant scenarios, and scales better. Apply least privilege, one identity per workload, and monitor sign-in logs.

Likely follow-ups

- What is a federated identity credential?
- Why was pod-managed identity deprecated?

🗺 Workload Identity




```
# 1. Enable OIDC issuer + workload identity on the cluster:
```

```
az aks update -g rg -n prod-aks --enable-oidc-issuer --enable-workload-identity
```

```
# 2. Create a managed identity and federate it to a K8s service account:
```

```
az identity create -g rg -n app-mi
```

certs.eknathalabs.com

```
az identity federated-credential create -g rg -n app-fc --identity-name app-mi \\
  --issuer $(az aks show -g rg -n prod-aks --query oidcIssuerProfile.issuerUrl -o tsv) \\
  --subject system:serviceaccount:prod:app-sa
# 3. Annotate the K8s service account; pod uses it, gets tokens automatically
```

Workload Identity federates a Kubernetes **service account** with an Entra ID managed identity via the cluster's OIDC issuer. Pods using that service account receive short-lived, automatically-rotated Entra tokens to access Azure resources (Key Vault, Storage, SQL) — with no secrets stored anywhere. It replaces the deprecated **pod-managed identity (aad-pod-identity)**, which relied on a node-level component intercepting metadata requests and had scaling, latency, and security drawbacks. Workload Identity is the current standard: cleaner (native OIDC federation), more secure (no intercepting daemon), and scalable.

Interview tip: say explicitly that Workload Identity *replaces* the deprecated aad-pod-identity, and explain the mechanism: OIDC federation maps a K8s service account to an Entra identity, pods get auto-rotating tokens, zero stored credentials. This is the most current AKS identity answer and a strong signal you track the platform's evolution.

📚 **Official docs**

- Azure: Managed identities
- Azure Kubernetes Service (AKS)

Q44 Explain Azure CNI vs kubenet vs CNI Overlay for AKS networking.

Networking

🎙 **Spoken answer · ~81 s**

These are AKS networking models differing in how pods get IP addresses. With kubenet, nodes get IPs from the VNet subnet, but pods get addresses from a separate private CIDR; traffic leaving a node is NATed, and user-defined routes direct pod traffic between nodes. It conserves VNet addresses and is simple, but has limits: a route table with a maximum number of nodes, extra hops, no direct pod reachability from outside the cluster, no support for some features like Windows nodes and virtual nodes, and it is being retired. Azure CNI, the traditional model, assigns every pod a real IP from the VNet subnet, so pods are directly routable, integrate with NSGs, private endpoints, and on-premises networks, and perform well, but consume many VNet IPs, so subnet planning is critical, and scaling is limited by address space. Azure CNI Overlay gives pods IPs from a private overlay CIDR, not the VNet, while nodes use VNet addresses; it scales to very large clusters, conserves VNet space, and avoids route table limits, with pod traffic NATed when leaving the cluster, so pods aren't directly addressable from outside. Azure CNI with Cilium adds eBPF-based networking and policy.

Likely follow-ups

- Why is Overlay the usual recommendation for new clusters?
- What limits does kubenet have?

cons.eknathalabs.com

```
Azure CNI (standard): every pod gets a real VNet IP (fast, NSG-controllable, IP-hungry)
```

```
kubenet:              pods get internal IPs, UDR-routed (IP-efficient, more limits)
[legacy]
```

```
Azure CNI Overlay:    pods on an overlay network, nodes use VNet IPs (scales huge, IP-
efficient)
```

```
# Overlay (recommended for large clusters):
```

```
az aks create -g rg -n prod-aks --network-plugin azure --network-plugin-mode overlay \\
```

```
  --pod-cidr 10.244.0.0/16
```

The choice trades VNet integration against IP consumption. **Azure CNI (standard)** gives every pod a routable VNet IP — best network performance and NSG-level pod control, but consumes VNet address space fast (nodes × max-pods), risking IP exhaustion. **kubenet** conserves IPs (pods get internal addresses, UDR-routed) but is legacy with feature limits. **Azure CNI Overlay** is the modern recommendation for large clusters: pods run on a separate overlay CIDR while nodes use VNet IPs — giving near-CNI performance with massive scale and minimal VNet IP usage. Cilium-powered CNI adds eBPF dataplane and network policy.

Interview tip: the deciding factor is IP address planning at scale. "Large cluster, limited VNet IP space" → Azure CNI Overlay (or Cilium). "Need every pod directly addressable in the VNet" → standard Azure CNI. Calling kubenet legacy and recommending Overlay/Cilium for new large clusters reflects current AKS guidance.

> 📚 **Official docs**
>
> - Kubernetes: Cluster networking model
> - AKS: Network concepts
> - Kustomize reference

certs.eknathalabs.com

Q45 How do you set up cross-tenant or B2B access securely in Entra ID?

IAM & EntraID

🎙 Spoken answer · ~87 s

Cross-tenant and B2B collaboration should balance enablement with control. Entra B2B lets external users, from other Entra tenants or other identity providers, be invited as guests into your tenant, using their own credentials, so you don't manage their passwords. Secure it with several controls. Cross-tenant access settings: define inbound and outbound policies per partner organisation, specifying which users, groups, and applications can collaborate, and control whether you trust the partner's MFA and device claims, so guests aren't asked for MFA twice but only when the partner enforces it. Default-deny: restrict collaboration to an allow list of approved tenants where practical. Conditional Access: require MFA, compliant devices, or terms of use for guest users, scope policies to guest and external user types, and block risky sign-ins. Least privilege: guests get limited directory permissions by default, grant resource access through group membership and Azure RBAC with narrow roles, and avoid giving guests privileged roles. Governance: use entitlement management access packages with expirations and approval workflows, and access reviews to recertify and remove stale guests. Restrict who can invite. Monitor sign-in logs and audit logs for guest activity. For tenant-to-tenant data sharing, such as multi-tenant applications, use app consent controls and workload identity federation rather than shared secrets. Document the partner relationships.

Likely follow-ups

- Why scope Conditional Access policies to guest users?
- How do access reviews limit guest sprawl?

```
# Invite an external (guest) user via B2B collaboration:
```

```
az ad user invite --invited-user-email partner@vendor.com \\
```

```
  --invite-redirect-url https://portal.azure.com
```

```
# Cross-tenant access settings control what guests can do:
```

```
#  - inbound: which external tenants/users may be invited
```

```
#  - outbound: which external resources your users may access
```

```
#  - trust MFA/device claims from the home tenant (avoid double-MFA)
```

```
# Entitlement management: access packages with approval + expiry for guests
```

Entra **B2B collaboration** invites external users as **guest** identities in your tenant; they authenticate with their home-tenant credentials but are governed by your policies. **Cross-tenant access settings** let you control inbound/outbound collaboration per partner tenant and optionally trust the partner's MFA/device claims (avoiding double prompts). For governance at scale, **Entitlement Management** bundles resources into

certs.eknathalabs.com

access packages with approval workflows and automatic expiry/recertification, so guest access does not linger. Conditional Access still applies to guests, so you can require MFA and restrict what external identities reach.

Interview tip: the secure pattern is B2B guests (not shared accounts) + cross-tenant access settings (scoped trust, optional MFA pass-through) + Entitlement Management (time-bound, approved, recertified access) + Conditional Access on guests. Emphasising expiry/recertification shows you think about guest-access sprawl, a real governance risk.

📚 **Official docs**

- Microsoft Entra ID: What is Microsoft Entra ID?

Q46 How do you design egress control and force-tunneling for AKS in a regulated environment?

Networking

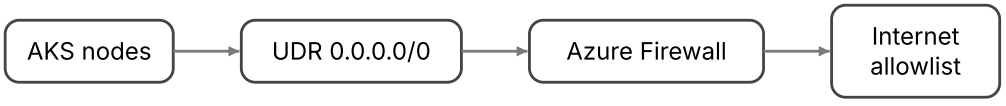
🎙 **Spoken answer · ~87 s**

In a regulated environment, outbound traffic from AKS must be inspected, restricted, and logged. The standard design is hub-and-spoke: the cluster's subnets use a user-defined route table sending the default route, zero-zero-zero-zero slash zero, to Azure Firewall in the hub, a pattern called forced tunnelling, so all egress passes through central controls. Configure AKS with outbound type userDefinedRouting, so it doesn't create its own public load balancer egress. The firewall must allow the required AKS control plane and platform endpoints, using the AzureKubernetesService FQDN tag for the necessary Microsoft endpoints, plus NTP and registry access; everything else is denied by default. Then allow-list application destinations by FQDN in application rules, and use network rules for non-HTTP traffic. Use private clusters and private endpoints for Azure services such as ACR, Key Vault, and storage, so that traffic never traverses the internet; mirror needed images into ACR. Enable threat intelligence and, on the premium tier, TLS inspection and IDPS where required. Log everything to Log Analytics, with alerts on denied traffic. Add Kubernetes network policies for east-west control, and DNS proxy for FQDN rules. Watch for asymmetric routing when traffic returns, and SNAT capacity on the firewall. Test thoroughly, because blocking required endpoints breaks nodes. Document exceptions and review rules regularly.

Likely follow-ups

- What is forced tunnelling in this context?
- Why must the firewall allow the AKS FQDN tag?

🗺 **Egress control**



certs.eknathalabs.com

```
# Route all node/pod egress through Azure Firewall in the hub:
az network route-table route create -g rg --route-table-name aks-rt \\
  -n to-fw --address-prefix 0.0.0.0/0 --next-hop-type VirtualAppliance \\
  --next-hop-ip-address 10.0.1.4
# Associate UDR to the node subnet; set AKS outbound type:
az aks create -g rg -n prod-aks --outbound-type userDefinedRouting ...
# Allow required AKS egress FQDNs on the firewall (mcr.microsoft.com, etc.)
```

In regulated environments, you must control and inspect all outbound traffic. Set the AKS **outbound type to userDefinedRouting** and use a User-Defined Route to force `0.0.0.0/0` egress through **Azure Firewall** in the hub, where you allow only required destinations (by FQDN/tag) and log everything. AKS itself needs specific egress FQDNs (the Microsoft Container Registry, AKS API, Ubuntu/security repos, etc.), so the firewall ruleset must permit those or the cluster breaks. This gives a single, auditable egress chokepoint that satisfies compliance requirements for traffic inspection and data-exfiltration prevention.

Interview tip: the mechanism is `outboundType=userDefinedRouting` + UDR → Azure Firewall + an allow-list of required AKS FQDNs. The gotcha to mention: lock down egress *and* remember AKS has mandatory egress endpoints — over-tight rules silently break image pulls and node bootstrap. Centralised, logged egress is the compliance story.

> 📚 **Official docs**
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q47 What is Defender for Cloud and how does it improve AKS/Azure security posture?

IAM & EntraID

🎙 Spoken answer · ~85 s

Microsoft Defender for Cloud is a cloud security posture management and workload protection platform. Cloud security posture management continuously assesses resources against benchmarks like the Microsoft cloud security benchmark and regulatory standards, producing a secure score and prioritised recommendations, such as enabling private endpoints, fixing exposed management ports, or enabling encryption, with attack path analysis showing how a threat actor could chain weaknesses to reach critical assets. Workload protection comes through Defender plans. For AKS, Defender for Containers provides: image vulnerability scanning in registries and running images; runtime threat detection, using a sensor on nodes to detect suspicious behaviour like cryptominers, reverse shells, and privilege escalation; Kubernetes API audit log analysis for suspicious operations; Kubernetes-specific posture recommendations and admission control integration through Azure Policy; and discovery of exposed workloads. How it improves posture: you get visibility across subscriptions and multi-cloud, actionable prioritised remediation guidance, automated compliance reporting, and alerts you can send to Sentinel or ticketing. Practical rollout: enable the foundational CSPM everywhere, and enable Defender plans where workloads justify cost; assign owners to recommendations; automate remediation with policy and workflow automation; integrate alerts into incident response; track the secure score over time. It complements, rather than replaces, good design, least privilege, and patching.

Likely follow-ups

- What does Defender for Containers detect at runtime?
- What is attack path analysis?

```
# Enable Defender plans (CSPM + workload protection):
```

```
az security pricing create -n Containers --tier Standard
```

```
az security pricing create -n VirtualMachines --tier Standard
```

```
# Defender for Containers provides:
```

```
#  - image vulnerability scanning (in ACR + at runtime)
```

```
#  - K8s runtime threat detection (suspicious exec, crypto-mining)
```

```
#  - secure-score recommendations + regulatory compliance dashboards
```

Microsoft Defender for Cloud combines posture management (CSPM) with workload protection (CWPP). For posture, it continuously assesses configuration against best practice and regulatory standards (CIS, PCI, SOC2), producing a **Secure Score** and prioritized remediation. **Defender for Containers** adds AKS-specific

certs.eknathalabs.com

protection: scanning images in ACR and at runtime for vulnerabilities, detecting runtime threats (suspicious `kubectl exec`, crypto-mining, privilege escalation), and surfacing Kubernetes misconfigurations. It turns security from periodic audits into continuous monitoring with actionable, ranked recommendations.

Interview tip: split it into *posture* (CSPM: Secure Score, compliance dashboards, config recommendations) and *protection* (CWPP: image scanning, runtime threat detection). For AKS specifically, name Defender for Containers' image scanning + runtime detection. Tying it to compliance frameworks (CIS/PCI/SOC2) connects security to audit requirements interviewers care about.

📚 **Official docs**

- Microsoft Defender for Cloud
- Azure Kubernetes Service (AKS)

Q48 How do you achieve multi-region active-active networking on Azure?

Networking

🎙 **Spoken answer · ~87 s**

Active-active across regions needs global traffic distribution, regional networks that mirror each other, and a data strategy. For traffic, use Azure Front Door for HTTP applications, with origin groups in each region, latency-based routing, health probes, and automatic failover; for non-HTTP, use Traffic Manager for DNS-based routing or a cross-region load balancer. Each region has its own hub-and-spoke network with a firewall, and applications deployed identically through infrastructure as code. Connect the regions with global VNet peering or Virtual WAN so that replication and management traffic flows privately, with non-overlapping address spaces. Use private endpoints and private DNS zones carefully, linked to each region's VNets, with region-specific records so each region resolves its local endpoints. ExpressRoute circuits in both regions provide on-premises connectivity with redundancy. The hard part is data: choose data services with multi-region capabilities, such as Cosmos DB with multi-region writes, Azure SQL failover groups, geo-redundant storage, and accept the consistency trade-offs. Design applications to be stateless, tolerate replication lag, and route a user consistently to a home region where needed. Plan capacity so that one region can absorb the other's load. Test failover regularly with game days, monitor each region's health independently, and automate. Weigh the substantial cost and complexity against requirements; active-passive is often adequate.

Likely follow-ups

- How does Front Door provide failover between regions?
- Why must each region be able to absorb the other's load?

certs.eknathalabs.com

```
# Global L7 entry with health-based routing + failover:
```

```
az afd profile create -g rg --profile-name global-fd --sku Premium_AzureFrontDoor
```

```
# Origins in each region (regional App Gateway / AKS LB), health-probed
```

```
# Data tier: geo-distributed (Cosmos DB multi-region writes) or paired-region replication
```

```
# Per-region VNets (no overlapping CIDRs) for future peering/global VNet peering
```

```
# DNS: Front Door for HTTP; Traffic Manager for non-HTTP/DNS-level failover
```

Active-active multi-region means each region runs a full stack serving live traffic, with global routing distributing users and failing over automatically. **Azure Front Door** (or Traffic Manager for non-HTTP) provides the global entry point, health-probing each region and routing to the nearest healthy one. Each region has its own VNet (non-overlapping CIDRs so they can be peered) and regional ingress (App Gateway/AKS). The hard part is **data**: use a globally-distributed store (Cosmos DB with multi-region writes) or carefully designed cross-region replication, accepting the consistency trade-offs that active-active writes imply.

Interview tip: separate the easy part (stateless compute replicated per region behind Front Door) from the hard part (multi-region *data* consistency). Mention Front Door for global HTTP routing/failover, non-overlapping VNet CIDRs, and that the real design challenge is the data tier (Cosmos multi-region writes vs replication latency/consistency). That framing reads as principal-level.

> 📚 **Official docs**
>
> - Google Cloud: Multi-region deployment archetypes

certs.eknathalabs.com

Q49 How do you perform a zero-downtime AKS cluster upgrade in production?

Containers & AKS

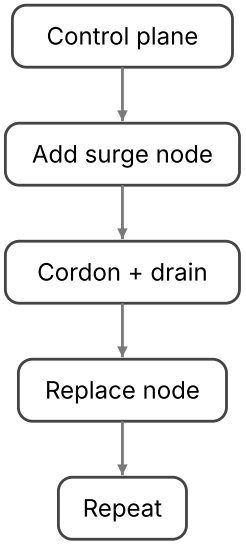
🎙 Spoken answer · ~89 s

Zero-downtime AKS upgrades depend on preparation and the surge mechanism. Preparation: check the Kubernetes version support policy and release notes; find deprecated or removed APIs in your manifests and Helm charts, using tools like kubent or Pluto and the AKS deprecation insights; verify add-ons, ingress controllers, CSI drivers, and operators support the target version; ensure workloads run multiple replicas across zones with PodDisruptionBudgets, proper readiness probes, and graceful termination; confirm quotas and subnet IP space for surge nodes. Test the upgrade in a staging cluster with representative workloads, and back up important state. Execution: upgrade the control plane first, which is highly available and doesn't interrupt running workloads, then upgrade the node pools. For node pools, AKS uses surge: it creates extra nodes up to max-surge, cordons and drains old nodes respecting PDBs, and replaces them progressively, with the max-surge setting trading speed against capacity. Upgrade the system pool, then user pools, optionally one at a time. Use maintenance windows and auto-upgrade channels with care; for production, consider the blue-green cluster approach for major changes: build a new cluster, shift traffic gradually via Front Door or DNS, then decommission the old. Monitor workloads and run smoke tests throughout, and keep a rollback path, since you can't downgrade the control plane. Document and rehearse.

Likely follow-ups

- What does max-surge control?
- When would you use a blue-green cluster instead of an in-place upgrade?

🗺 AKS upgrade




```
# 1. Upgrade the control plane first:
```

```
az aks upgrade -g rg -n prod-aks --control-plane-only --kubernetes-version 1.30
```

certs.eknathalabs.com

```
# 2. Upgrade node pools with surge (extra nodes added before draining old):
```

```
az aks nodepool update -g rg --cluster-name prod-aks -n workers \\
```

```
  --max-surge 33%   # 33% extra capacity during rollout
```

```
az aks nodepool upgrade -g rg --cluster-name prod-aks -n workers --kubernetes-version 1.30
```

```
# Ensure PodDisruptionBudgets protect minimum replicas during node drains
```

AKS upgrades the **control plane** and **node pools** separately. Upgrade the control plane first (it stays available with AZ-backed SLA), then roll node pools. Node-pool upgrade **cordons and drains** each node, rescheduling pods elsewhere — so zero downtime depends on two things: **max-surge** (adds extra nodes before draining, avoiding capacity dips) and **PodDisruptionBudgets** (guarantee a minimum number of replicas stay running during evictions). Without PDBs, a drain can take all replicas of a service down at once; without surge, you lose capacity mid-rollout.

Interview tip: the two pillars of zero-downtime upgrades are **max-surge** (capacity during the rollout) and **PDBs** (availability during drains), plus readiness probes so traffic only hits ready pods. Mention upgrading control plane before nodes, and validating in a non-prod cluster first. Blue/green node pools are the advanced option for risk-averse upgrades.

> 📚 **Official docs**
>
> - Kubernetes: Upgrading kubeadm clusters
> - AKS: Best practices
> - AKS: Upgrade a cluster

certs.eknathalabs.com

Q50 How do you implement a canary or blue-green deployment on AKS?

Containers & AKS

🎙 Spoken answer · ~87 s

On AKS, canary and blue-green releases are achieved by controlling traffic between versions. Blue-green: run two complete deployments, blue as current and green as new, each behind its own Service or labelled selector; test green internally, then switch the production Service selector or ingress backend to green, with an instant rollback by switching back. It costs double capacity during the switch, and needs backward-compatible database changes. Canary: send a small share of traffic to the new version and increase gradually. Implementation options: with an ingress controller like NGINX, use canary annotations for weight or header-based routing; with Application Gateway, weighted backend pools; with a service mesh such as the Istio add-on, VirtualService weights; or use Argo Rollouts or Flagger, which automate the process: they manage ReplicaSets, shift traffic in steps, analyse metrics from Prometheus or Azure Monitor against thresholds like error rate and latency, and automatically promote or roll back. Plain Kubernetes rolling updates with maxSurge and maxUnavailable are the simpler baseline. Requirements: good observability labelled by version, readiness probes, automated analysis, and defined abort criteria. Use feature flags to decouple release from deploy. Keep sessions and data compatible across versions, clean up the old version after bake time, and integrate into the pipeline with approvals for production.

Likely follow-ups

- How does Argo Rollouts automate canary analysis?
- Why must database changes be backward compatible for blue-green?

```
# Blue-green: two deployments, switch Service selector
```

```
kubectl apply -f green-deploy.yaml         # new version alongside blue
```

```
kubectl patch service myapp -p '{"spec":{"selector":{"version":"green"}}}'  # cutover
```

```
# Canary with Istio (AKS mesh add-on) weighted routing:
```

```
#   VirtualService: 90% -> v1 subset, 10% -> v2 subset, shift gradually
```

```
# Progressive delivery (automated): Argo Rollouts / Flagger
```

```
#   analyze golden-signal metrics, auto-promote or auto-rollback
```

Blue-green runs two full versions; you flip the Service selector (or ingress) from blue to green for an instant cutover, with instant rollback by flipping back — simple, but doubles resources momentarily and shifts 100% at once. **Canary** shifts a small traffic percentage to the new version (10% → 50% → 100%), limiting blast radius and letting you observe before full rollout — best done with a mesh (Istio) or ingress weighting. **Progressive delivery** tools (Argo Rollouts, Flagger) automate canary analysis: they watch error rate/latency and auto-promote on success or auto-rollback on regression.

certs.eknathalabs.com

Interview tip: blue-green = instant switch + easy rollback but all-or-nothing; canary = gradual, metric-observed, smaller blast radius. The senior addition is **automated** progressive delivery (Argo Rollouts/Flagger) that gates promotion on golden-signal metrics and rolls back automatically — turning safe deploys into the default rather than a manual ritual.

📚 **Official docs**

- Google SRE Workbook: Canarying Releases
- Argo Rollouts
- Martin Fowler: BlueGreenDeployment

Q51 What are resource requests and limits, and how do they affect scheduling and QoS on AKS?

Containers & AKS

🎙 **Spoken answer · ~77 s**

Requests are the resources a container is guaranteed, used by the scheduler to place pods: a pod is scheduled onto a node only if the node's unallocated capacity covers the sum of the requests. Limits are the maximum a container may use. For CPU, exceeding the limit results in throttling; for memory, exceeding it results in the container being OOM-killed. Together they determine the Quality of Service class. Guaranteed pods have requests equal to limits for all containers in CPU and memory; they get the highest priority and are evicted last. Burstable pods have at least one request or limit set but not equal; they're evicted after BestEffort ones under pressure. BestEffort pods have no requests or limits; they're evicted first and are risky in production. Effects on AKS: missing requests lead to overcommitted nodes, noisy neighbours, and unpredictable performance; requests that are too high waste money and cause unschedulable pods; limits that are too low cause throttling latency or OOM kills. Autoscaling relies on requests, since HPA's CPU utilisation is relative to them, and the cluster autoscaler adds nodes based on pending requests.

Likely follow-ups

- Which QoS class is evicted first?
- Why can CPU limits hurt latency-sensitive services?

```
resources:
```

```
  requests: {cpu: 250m, memory: 256Mi}   # used for SCHEDULING + guarantee
```

```
  limits:   {cpu: 500m, memory: 512Mi}   # hard cap (CPU throttled, memory OOMKilled)
```

```
# QoS classes derived from requests/limits:
```

```
#  Guaranteed: requests == limits (highest priority, last evicted)
```

```
#  Burstable:  requests < limits
```

certs.eknathalabs.com

```
#  BestEffort: none set (first evicted under pressure)
```

Requests are what the scheduler reserves — a pod is placed only where its requested CPU/memory fits, and requests act as the guaranteed floor. **Limits** are the hard ceiling: exceeding the CPU limit causes **throttling** (silent slowdowns), exceeding the memory limit causes **OOMKill**. The relationship sets the pod's **QoS class**: *Guaranteed* (requests=limits) pods are evicted last; *Burstable* next; *BestEffort* (nothing set) first. Accurate requests are critical: too high wastes node capacity (nothing else schedules), too low risks OOMKill and noisy-neighbour contention.

Interview tip: nail the effects — requests drive scheduling + QoS + eviction order; CPU limit → throttling, memory limit → OOMKill. The common production mistake is mis-set requests (over-request wastes money, under-request causes evictions/OOM). Mentioning that CPU throttling is *silent* (no events, just latency) is a strong real-world detail.

📚 **Official docs**

- Kubernetes: Resource management for Pods & containers
- Kubernetes: Pod Quality of Service classes
- Azure Kubernetes Service (AKS)

Q52 How do you choose between AKS, Azure Container Apps, and Azure Container Instances?

Compute

🎙 **Spoken answer · ~86 s**

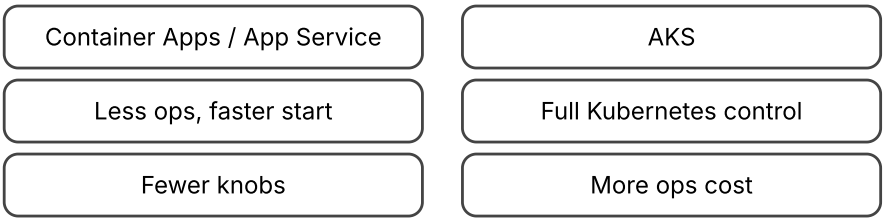
The three offer different levels of abstraction. AKS is managed Kubernetes: full control, the entire Kubernetes ecosystem, advanced networking and scheduling, operators, service meshes, and portability, but you operate the data plane, upgrades, and cluster design, requiring skills and a platform team. Azure Container Apps is a serverless container platform built on Kubernetes, with KEDA, Dapr, and Envoy included: you deploy containers and configure scaling, ingress, revisions, and traffic splitting without managing clusters, with scale to zero, per-second billing, and built-in microservices features. It suits microservices, APIs, background workers, and event-driven apps where you don't need direct Kubernetes API access. Azure Container Instances runs individual containers or small groups on demand, with fast startup and per-second billing, without orchestration, scaling, or load balancing built in. It suits short-lived tasks, build jobs, simple batch work, burst capacity, and quick tests; it's not meant for complex long-running multi-service applications. Choose AKS when you need Kubernetes features, custom controllers, complex networking, or have existing Kubernetes investment and expertise; Container Apps when you want the productivity of serverless containers for microservices with minimal operations; ACI for simple isolated tasks. They can be combined, for example ACI as virtual nodes bursting from AKS. Consider team skills, workload complexity, cost model, and portability.

Likely follow-ups

- When is Container Apps a better fit than AKS?
- What is ACI best suited for?

certs.eknathalabs.com

🗺 **Compute choice**




```
ACI:            single containers, fast start, per-second billing, no orchestration
```

```
Container Apps: serverless containers on managed K8s+KEDA+Dapr, scale-to-zero
```

```
AKS:            full Kubernetes, maximum control + ecosystem, you run the platform
```

```
# ACI as AKS virtual-node burst target (Virtual Kubelet):
```

```
az aks enable-addons --addons virtual-node -g rg -n prod-aks --subnet-name aci-subnet
```

It is a control-vs-simplicity spectrum. **ACI** runs individual containers with no orchestration — great for short tasks, burst, or CI jobs (per-second billing, fast start), and can back AKS virtual nodes for burst capacity. **Container Apps** is serverless containers on a managed Kubernetes+KEDA+Dapr foundation: scale-to-zero, built-in ingress, revisions/canary, microservices features — without operating a cluster. **AKS** gives full Kubernetes: maximum control, the entire ecosystem (operators, CRDs, mesh), and the responsibility of running the platform. Choose by how much Kubernetes control you actually need.

Interview tip: "do you need to *run Kubernetes* or just *run containers*?" ACI for one-off/burst, Container Apps for serverless microservices without cluster ops, AKS when you need full K8s control/ecosystem. For a platform team standardising on K8s tooling, AKS; for product teams wanting velocity without platform overhead, Container Apps. That decision framing is what they want.

📚 **Official docs**

- Azure Container Apps
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q53 How does the AKS Cluster Autoscaler work and how do you tune it?

Containers & AKS

🎙 Spoken answer · ~87 s

The Cluster Autoscaler adjusts the number of nodes in each autoscaling-enabled node pool. It watches for pods that are pending because no node has enough resources, and scales the pool up, within its configured minimum and maximum, by increasing the underlying scale set's capacity; it scales down when nodes are underutilised, below a threshold of requested resources for a period, and their pods can be moved elsewhere, then drains and removes the node. It decides using resource requests, not actual usage, so accurate requests are essential. You enable it per node pool with min and max counts, and tune cluster-level settings through the autoscaler profile: scan interval, scale-down delay after add, scale-down unneeded time, utilisation threshold, expander strategy such as random, least-waste, or priority, balance-similar-node-groups for zone balance, and max graceful termination. Things that block scale-down: pods without a controller, pods with local storage, restrictive PodDisruptionBudgets, node affinity constraints, and system pods; annotations can mark pods as safe or unsafe to evict. Tuning for faster scale-up: smaller scan intervals, over-provisioning with low-priority placeholder pods to keep spare capacity, and appropriate max-surge. Pair it with HPA for pods. Consider node auto-provisioning, Karpenter-based, for more flexible VM selection. Monitor pending pods, scale events, and cost, and test scaling under load.

Likely follow-ups

- Why does the autoscaler depend on resource requests?
- What blocks a node from scaling down?

```
az aks nodepool update -g rg --cluster-name prod-aks -n workers \\
  --enable-cluster-autoscaler --min-count 3 --max-count 20
# Tunable behaviour (autoscaler profile):
az aks update -g rg -n prod-aks --cluster-autoscaler-profile \\
  scale-down-delay-after-add=10m scale-down-unneeded-time=10m \\
  scale-down-utilization-threshold=0.5
```

The **Cluster Autoscaler** watches for pods that cannot be scheduled due to insufficient resources and adds nodes (scaling the underlying VMSS) up to `max-count`; it removes nodes that stay underutilized long enough, down to `min-count`. It scales on **pending pods** (unschedulable due to requests), not on raw utilization — which is why accurate pod *requests* are essential. Tuning levers control scale-down aggressiveness: how long to wait after adding a node, how long a node must be unneeded, and the utilization threshold below which a node is a removal candidate.

certs.eknathalabs.com

Interview tip: the crucial insight is that it scales on *unschedulable pods* driven by resource *requests*, not actual usage — so wrong requests break autoscaling. Pair it mentally with HPA/KEDA (scale pods) → Cluster Autoscaler (scale nodes). Mention scale-down tuning and that node provisioning has latency (minutes), which is why surge/headroom matters for spiky load.

📚 **Official docs**

- Kubernetes Autoscaler: Cluster Autoscaler
- AKS: Cluster autoscaler
- Azure Kubernetes Service (AKS)

Q54 What is the difference between availability sets, availability zones, and region pairs? Compute

🎙 **Spoken answer · ~85 s**

These are Azure's resilience constructs at different scales. An availability set spreads VMs within a single datacentre across fault domains, which are separate racks with independent power and network, and update domains, which are groups rebooted separately during planned maintenance. It protects against rack-level failures and maintenance, with a lower SLA than zones, and doesn't protect from datacentre loss. Availability zones are physically separate locations within a region, each with independent power, cooling, and networking; deploying across three zones protects from a datacentre failure, giving a higher SLA for VMs, and zone-redundant services like zone-redundant storage, load balancers, and databases replicate across zones synchronously with low latency. Region pairs are two regions in the same geography, typically hundreds of miles apart, that Azure pairs for disaster recovery: geo-redundant storage replicates to the pair, platform updates roll out to one region of a pair at a time, and in a wide outage one region of each pair is prioritised for recovery. Design guidance: use zones for high availability within a region, since they handle most failure scenarios with low latency; use region pairs, or any other region, for disaster recovery, with replicated data and tested failover. Availability sets are mostly legacy for regions without zones.

Likely follow-ups

- What does an availability zone protect against that an availability set does not?
- What are fault domains and update domains?

```
Availability Set:  spreads VMs across fault/update domains WITHIN a datacenter
```

```
Availability Zones: physically separate datacenters within a region (AZ1/AZ2/AZ3)
```

```
Region Pairs:       two regions paired for geo-redundancy + sequential updates
```

```
# AKS across zones:
```

```
az aks create -g rg -n prod-aks --zones 1 2 3 --node-count 3
```

cors.EknathaLabs.com

These are increasing levels of fault isolation. **Availability Sets** spread VMs across fault and update domains *within a single datacenter* — protecting against rack/hardware and maintenance failures, but not a datacenter outage. **Availability Zones** are physically separate datacenters within a region (independent power/cooling/network) — protecting against a whole-datacenter failure; spreading AKS nodes across zones survives a zone outage. **Region Pairs** are two regions linked for geo-redundancy, with platform updates rolled sequentially (never both at once) and prioritized recovery — protecting against a regional disaster.

Interview tip: map each to the failure it survives — Availability Set (rack/host within a DC), Availability Zone (whole datacenter within a region), Region Pair (entire region). For production AKS, spread node pools across **zones**; for DR, replicate to the **paired region**. Knowing AZs are physically distinct DCs (not just logical) is the differentiator.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q55 How do you secure the software supply chain for AKS container images?

Containers & AKS

🎙 **Spoken answer · ~85 s**

Securing the supply chain for AKS images means controlling what goes into images and ensuring only trusted ones run. Build: use minimal, trusted base images, pin versions, and rebuild regularly to pick up patches; scan source and dependencies in CI; generate a software bill of materials; and build on isolated, ephemeral agents with least-privilege credentials, ideally workload identity rather than stored secrets. Registry: use Azure Container Registry, private, with private endpoints, and disable anonymous pull and admin user; attach to AKS through managed identity with the AcrPull role; enable Defender for Containers to scan images on push and continuously, flagging new vulnerabilities in stored and running images; use content trust or, preferably, sign images with Notation or Cosign, and attach attestations. Use ACR tasks to rebuild when base images update, and retention policies. Deployment: enforce admission policies so only images from approved registries, signed by trusted keys, and passing scans can run, using Azure Policy for Kubernetes with Gatekeeper, Ratify for signature verification, or Kyverno. Reference images by digest rather than mutable tags. Runtime: run as non-root, with read-only filesystems and dropped capabilities, and monitor for runtime threats. Mirror external images into your registry, rather than pulling from public registries at deploy time.

Likely follow-ups

- Why deploy by digest instead of tag?
- How do admission policies enforce signed images?

certs.eknathalabs.com

🗺 **Supply chain**




```
# Scan images in ACR (Defender) + in CI before push:
```

```
az acr task create -n scan --registry myacr --cmd "trivy image \$IMG" ...
```

```
# Sign images (Notation / cosign) and store signatures in ACR:
```

```
notation sign myacr.azurecr.io/app:v1
```

```
# Enforce: only signed images from approved registries (Azure Policy / ratify +
Gatekeeper):
```

```
kubectl get constrainttemplate   # e.g. allow only myacr.azurecr.io/*, signed
```

Supply-chain security for AKS spans build, registry, and admission. **Scan** images for vulnerabilities in CI and continuously in ACR (Defender for Containers). **Sign** images (Notation/cosign) so their provenance can be verified, storing signatures alongside the image in ACR. **Enforce** at the cluster with Azure Policy / Ratify + Gatekeeper: admit only images from approved registries that are signed and pass vulnerability gates, blocking anything else. Pull from ACR using **Workload Identity** (no registry credentials in pods). This closes the gap between "image built" and "image running."

Interview tip: structure it as scan → sign → enforce-at-admission, plus credential-less ACR pulls via managed identity. Naming Ratify/Gatekeeper (or Azure Policy) to *block unsigned/unapproved images at admission* is the detail that shows you can make the policy preventive, not advisory — the same SLSA-style thinking interviewers probe for.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q56 How do you design for graceful shutdown and pod termination on AKS?

Compute

🎙 Spoken answer · ~88 s

Graceful termination ensures in-flight work finishes and no requests are dropped when a pod is stopped, during rollouts, scale-downs, node drains, or upgrades. The sequence: Kubernetes marks the pod Terminating and removes it from Service endpoints; in parallel, runs the preStop hook; sends SIGTERM to the container; waits for the termination grace period, thirty seconds by default; then sends SIGKILL. The race is that endpoint removal propagates to load balancers and ingress controllers asynchronously, so traffic can still arrive after SIGTERM. Design for it. In the application: handle SIGTERM by stopping accepting new connections, finishing in-flight requests, closing database connections and flushing buffers, then exiting; make sure the process is PID one or that the init system forwards signals, since shells in entrypoints can swallow them. Add a preStop hook with a short sleep, five to fifteen seconds, to let endpoint removal propagate before shutdown begins. Set terminationGracePeriodSeconds longer than the maximum request or job duration plus the preStop delay. Use readiness probes that start failing on shutdown. Define PodDisruptionBudgets so drains and upgrades don't remove too many pods at once, and run multiple replicas with anti-affinity across zones. For background workers, checkpoint progress and make jobs idempotent. Behind Application Gateway or Azure load balancers, also tune connection draining settings.

Likely follow-ups

- Why add a preStop sleep?
- Why must the application handle SIGTERM explicitly?

```
# Pod termination sequence:
#  1. Pod marked Terminating; removed from Service Endpoints
#  2. preStop hook runs (e.g. sleep to drain in-flight)
#  3. SIGTERM sent to container; app should finish in-flight, stop accepting new
#  4. After terminationGracePeriodSeconds, SIGKILL
lifecycle: {preStop: {exec: {command: ["sh","-c","sleep 10"]}}}
terminationGracePeriodSeconds: 30
```

When a pod is terminated (scale-down, rollout, node drain), Kubernetes removes it from Service Endpoints and sends **SIGTERM**, then **SIGKILL** after `terminationGracePeriodSeconds`. For zero dropped requests, the app must catch SIGTERM, stop accepting new connections, and finish in-flight work. A common race: endpoint removal and SIGTERM are near-simultaneous, but load balancers/kube-proxy take a moment to stop routing — so a short **preStop sleep** (or app-level drain) avoids sending traffic to a pod that already began shutting down. Set the grace period longer than the worst-case request duration.

certs.eknathalabs.com

Interview tip: the sequence (Endpoint removal → preStop → SIGTERM → grace → SIGKILL) plus the classic race (LB still routing briefly after termination starts) is the answer. The fix — a preStop drain delay and a SIGTERM handler — demonstrates you have debugged real "occasional 502s during deploy" issues, which is exactly the production signal.

📚 **Official docs**

- Kubernetes: Pod lifecycle & termination
- Azure Kubernetes Service (AKS)

Q57 What are taints, tolerations, node selectors, and affinity — and when do you use each?

Containers & AKS

🎙 **Spoken answer · ~89 s**

These scheduling controls decide where pods run. Node selectors are the simplest: a pod specifies labels that a node must have, such as a pool name or disk type; use them for straightforward hard placement. Node affinity is a richer version: required rules act as hard constraints, preferred rules are soft with weights, and it supports expressions like In and NotIn; use it when you need flexible or soft preferences, such as preferring spot nodes. Pod affinity and anti-affinity place pods relative to other pods: affinity co-locates pods, for example a cache near its client, while anti-affinity spreads replicas across nodes or zones for resilience; topology spread constraints do spread more precisely. Taints are applied to nodes to repel pods; tolerations on pods let them be scheduled onto tainted nodes. Taints say who is not welcome, while tolerations alone don't attract pods. Use taints to reserve nodes: GPU pools, spot pools, or system pools, with the CriticalAddonsOnly taint. A dedicated pool typically combines a taint to keep other workloads off and a node selector or affinity to pull the intended workload on. On AKS, system pods run on the system pool; user workloads go on user pools. Combine with priority classes and resource requests. Avoid over-constraining scheduling, which causes pending pods, and test.

Likely follow-ups

- Why do dedicated pools need both a taint and a selector?
- What do topology spread constraints add over anti-affinity?

```
# Taint a node pool so only tolerating pods land there (e.g. GPU/spot):
```

```
az aks nodepool add ... --node-taints sku=gpu:NoSchedule
```

```
tolerations: [{key: sku, operator: Equal, value: gpu, effect: NoSchedule}]
```

```
# nodeSelector / nodeAffinity to ATTRACT pods to nodes:
```

```
nodeSelector: {agentpool: gpunodes}
```

certs.eknathalabs.com

```
# podAntiAffinity to spread replicas across nodes/zones (HA):
```

```
#   topologyKey: topology.kubernetes.io/zone
```

Taints repel pods from a node pool unless they carry a matching **toleration** — used to reserve special pools (GPU, spot, system) for the right workloads. **Node selectors / node affinity** attract pods *to* nodes with certain labels (e.g. schedule onto the GPU pool). The two work together: taint the GPU pool so general pods stay off it, and use affinity/selector so GPU pods target it. **Pod anti-affinity** spreads replicas across nodes/zones for HA (so one node/zone failure does not take all replicas), while pod affinity co-locates related pods. Topology spread constraints are the modern way to balance across zones.

Interview tip: the mental model is "taints repel, affinity/selectors attract." Reserve specialized pools with taint+toleration *and* target them with affinity. For HA, use pod anti-affinity / topology spread across zones so replicas survive a zone loss. Giving the GPU/spot pool example and the zone-spread example shows practical scheduling experience.

📚 **Official docs**

- Kubernetes: Taints and tolerations
- Kubernetes: Assigning Pods to Nodes

Q58 How do you right-size AKS node pools and pods to balance cost and reliability?

Compute

🎙 **Spoken answer · ~83 s**

Right-sizing balances cost against reliability, based on measured data. Start at the pod level: observe actual CPU and memory usage over a representative period, including peaks, using Container Insights, Prometheus, or the Vertical Pod Autoscaler in recommendation mode, and set requests near typical usage with enough headroom, and memory limits above the peak; avoid wildly inflated requests, which waste capacity, and requests that are too low, which cause noisy neighbours and instability. Then the node level: choose VM sizes that bin-pack your pods well; larger nodes have less overhead from system components and reduce fragmentation, but a node failure affects more pods and scaling is coarser; smaller nodes give finer scaling but more overhead. Use multiple node pools for distinct profiles, such as memory-optimised, compute-optimised, spot, and GPU. Enable the cluster autoscaler or node auto-provisioning, and HPA, so capacity follows demand. Keep reliability protections: minimum replicas across zones, PodDisruptionBudgets, and headroom for failure, such as capacity to survive losing a zone. Use spot for tolerant workloads and reserved instances or savings plans for the baseline. Review regularly because workloads change; use the AKS cost analysis, Azure Advisor, and tools like Kubecost to find over-provisioned workloads and idle resources.

Likely follow-ups

- Why set requests from observed usage rather than guesses?
- What is the trade-off between large and small nodes?

certs.eknathalabs.com

```
# Observe actual usage to set requests (Container Insights / Prometheus):
```

```
kubectl top pods --all-namespaces --sort-by=memory
```

```
# VPA in recommendation mode suggests requests/limits:
```

```
kubectl get vpa myapp -o yaml   # target recommendations
```

```
# Match node VM size to pod shape (avoid stranded capacity):
```

```
#  many small pods -> general-purpose; memory-heavy -> E-series; CPU-heavy -> F-series
```

Right-sizing has two layers. At the **pod** level, set requests from observed usage (Container Insights/Prometheus, or VPA in recommendation mode): over-requesting strands node capacity (you pay for unused reservation and block scheduling), under-requesting causes OOMKills and contention. At the **node** level, pick VM sizes that match the workload shape so requested resources pack efficiently — mismatched node sizes leave "stranded" CPU or memory that nothing can use. Combine with Cluster Autoscaler bounds, spot pools for tolerant workloads, and bin-packing awareness to minimize idle capacity while keeping enough headroom for bursts and node failures.

Interview tip: tie cost to correctness — accurate *requests* are the lever for both reliability (no OOM/contention) and cost (no stranded capacity). Mention VPA recommendations to set requests from real data, matching node SKU to pod shape to avoid stranded resources, and keeping failure/burst headroom. This connects FinOps to reliability, a strong platform-engineering angle.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q59 How do liveness, readiness, and startup probes differ, and what happens when they are misconfigured?

Containers & AKS

🎙 Spoken answer · ~73 s

The three probes tell Kubernetes different things. The startup probe checks whether the application has finished starting; until it succeeds, the other probes are disabled, which protects slow-starting applications from being killed prematurely. The readiness probe checks whether the container is ready to receive traffic; if it fails, the pod is removed from Service endpoints but not restarted. The liveness probe checks whether the container is still healthy; if it fails repeatedly, the kubelet restarts the container. Misconfiguration causes real outages. A liveness probe that is too aggressive, or that checks dependencies like a database, can restart healthy pods during a slowdown, creating restart storms and amplifying the incident. A liveness probe without a startup probe can kill applications that take longer to boot, causing crash loops. A readiness probe that's too lax sends traffic to unready pods, causing errors during deploys; one that's too strict or checks downstream dependencies can remove all pods from rotation and cause a total outage. A missing readiness probe means rolling updates proceed before the new pods can serve.

Likely follow-ups

- Why shouldn't liveness probes check external dependencies?
- What problem does a startup probe solve?

```
livenessProbe:  {httpGet: {path: /healthz, port: 8080}, periodSeconds: 10}
```

```
readinessProbe: {httpGet: {path: /ready,   port: 8080}, periodSeconds: 5}
```

```
startupProbe:   {httpGet: {path: /healthz, port: 8080}, failureThreshold: 30,
periodSeconds: 5}
```

```
# liveness fail -> container RESTARTED
```

```
# readiness fail -> pod removed from Service Endpoints (no traffic), NOT restarted
```

```
# startup running -> liveness/readiness suppressed until startup passes
```

Liveness answers "is this container healthy?" — failure causes a **restart**. **Readiness** answers "can it serve traffic now?" — failure removes the pod from Service Endpoints (no traffic) but does **not** restart it. **Startup** protects slow-starting apps: while it runs, liveness/readiness are suppressed, so a slow boot is not mistaken for a crash. Misconfiguration is a top production cause of outages: a too-aggressive liveness probe restarts healthy-but-busy pods (CrashLoop under load); a liveness probe that should be readiness restarts pods that are merely warming up; missing readiness sends traffic to not-yet-ready pods (errors during rollout).

Interview tip: "liveness restarts, readiness gates traffic, startup buys boot time." The classic bug to cite: using liveness where readiness was needed, so heavy load makes the probe fail and Kubernetes restarts perfectly healthy pods — a self-inflicted CrashLoop. Recommending startup probes for slow apps and

certs.eknathalabs.com

conservative liveness thresholds shows hard-won experience.

📚 **Official docs**

- Kubernetes: Liveness, readiness and startup probes

Q60 How do you run stateful workloads (databases) on AKS, and when should you not? Compute

🎙 **Spoken answer · ~90 s**

Running databases on AKS is possible with the right tools and discipline. You need persistent storage, using StatefulSets with Azure Disk volumes through the CSI driver, ideally Premium SSD v2 or ultra for performance, and zone-aware placement since disks are zonal; operators that manage the lifecycle, such as CloudNativePG for PostgreSQL, which handles replication, failover, backups, and upgrades; anti-affinity and PodDisruptionBudgets to spread replicas across zones and protect against simultaneous disruption; dedicated node pools with taints for isolation and predictable performance; careful resource requests with the Guaranteed QoS class; backup and restore tested regularly, with point-in-time recovery to object storage; monitoring of replication lag, storage, and performance; and graceful handling of node upgrades. Considerations: the operational burden is significant, including upgrades, tuning, security patching, and incident response. When not to: if the team lacks the expertise, if the database is a critical system of record and a managed service meets requirements, or if you need advanced features such as automatic failover across regions, point-in-time recovery, and compliance certifications that managed offerings give. Azure SQL, PostgreSQL Flexible Server, and Cosmos DB take over the heavy lifting and usually provide better availability for less effort. Run databases on Kubernetes when you need portability, specific versions or extensions, or cost reasons, and have the skills to operate them.

Likely follow-ups

- What does a database operator automate?
- Why prefer managed databases for most teams?

```
# StatefulSet + persistent volumes for stable identity/storage:
#  - volumeClaimTemplates -> per-pod PVC (managed-premium / Ultra disk)
#  - stable network IDs (pod-0, pod-1) for clustering
# Operators manage complex stateful systems (e.g. Postgres, Kafka operators)
# BUT: managed PaaS is usually better:
#  Azure SQL / PostgreSQL Flexible Server / Cosmos DB -> Azure handles HA, backup, patching
```

certs.eknathalabs.com

Kubernetes *can* run stateful workloads via **StatefulSets** (stable pod identity, ordered rollout, per-pod persistent volumes) and **operators** that automate clustering, failover, and backups. But for most databases, **managed PaaS** (Azure SQL, PostgreSQL Flexible Server, Cosmos DB) is the better choice: Azure handles HA, patching, backups, and point-in-time restore, removing enormous operational burden and risk. Run databases on AKS only when you have a strong reason — a mature operator, specific version/extension needs, cost at scale, or portability requirements — and the team to operate them. Otherwise, keep stateful data in managed services and AKS for stateless compute.

Interview tip: the senior answer is "you *can* with StatefulSets + operators, but usually *shouldn't* — use managed PaaS." Articulate the trade-off: operators give control/portability but you own HA/backup/patching; PaaS offloads all of that. Knowing when *not* to put a DB on Kubernetes is a stronger signal than knowing how to.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q61 What is KEDA and how does it enable event-driven and scale-to-zero workloads on AKS?

Containers & AKS

🎙 **Spoken answer · ~83 s**

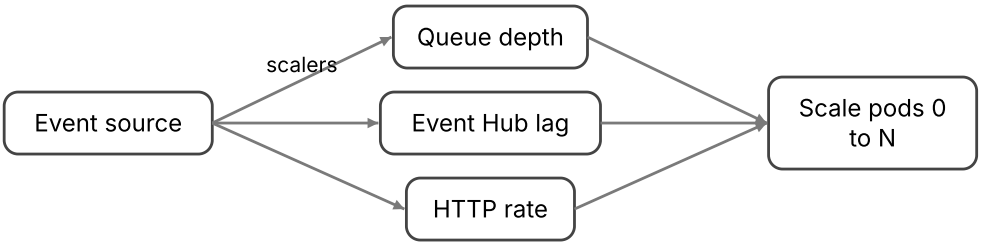
KEDA, Kubernetes Event-driven Autoscaling, scales workloads based on the number of events waiting to be processed, rather than just CPU or memory. It works alongside the HPA: KEDA acts as a metrics adapter and a controller; you define a ScaledObject, or ScaledJob for batch, which references a scaler and a target deployment. Scalers exist for many sources: Azure Service Bus queue length, Event Hubs lag, Storage queues, Kafka lag, Prometheus queries, cron schedules, HTTP request rates, and databases. KEDA can scale from zero replicas up when events arrive and back to zero when idle, which the plain HPA cannot do, saving cost for intermittent workloads. It's available as a managed AKS add-on. Authentication to event sources uses TriggerAuthentication, preferably with workload identity. Use cases: queue consumers whose backlog drives replicas, event-driven microservices, batch processors that run only when there's work, and scheduled scaling. Tuning: set polling interval, cooldown period, minimum and maximum replicas, and the target value per replica, such as messages per pod. Combine with the cluster autoscaler so new pods get nodes, and design consumers for idempotency and graceful shutdown since pods scale down. Consider cold-start latency when scaling from zero. Monitor lag and scaling decisions.

Likely follow-ups

- How does KEDA scale to zero when the HPA cannot?
- What is a ScaledJob used for?

certs.eknathalabs.com

🗺 **KEDA event scaling**




```
az aks update -g rg -n prod-aks --enable-keda
```

```
apiVersion: keda.sh/v1alpha1
```

```
kind: ScaledObject
```

```
spec:
```

```
  scaleTargetRef: {name: worker}
```

```
  minReplicaCount: 0          # scale to ZERO when idle
```

```
  maxReplicaCount: 100
```

```
  triggers:
```

```
  - type: azure-servicebus
```

```
    metadata: {queueName: jobs, messageCount: "5"}
```

KEDA (Kubernetes Event-Driven Autoscaling) extends the HPA to scale workloads on **event sources** — Service Bus/Storage Queue depth, Event Hub lag, Kafka, Prometheus metrics, cron schedules, HTTP traffic, and dozens more — rather than just CPU/memory. Critically, it supports **scale-to-zero**: when there are no events (empty queue), pods drop to zero, and the first event scales them back up. This brings serverless economics (pay only when working) to standard Kubernetes workloads, letting platform teams standardise on AKS tooling instead of splitting between Functions and K8s. It is the managed Azure equivalent of running KEDA yourself.

Interview tip: KEDA = "HPA, but on any event source, with scale-to-zero." The scale-to-zero capability (impossible with native HPA) is the headline. Frame it as the reason a platform team can keep event-driven and batch workloads on AKS rather than Azure Functions — one toolchain, same observability, same RBAC — which is the standardisation argument interviewers like.

📚 **Official docs**

- KEDA documentation
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q62 How do you handle GPU workloads (ML/AI) on AKS efficiently?

Compute

🎙 Spoken answer · ~90 s

GPU workloads on AKS need dedicated infrastructure and careful sharing. Create GPU node pools using N-series VM sizes such as NC or ND families; AKS installs the NVIDIA device plugin, or you install the GPU operator, so pods can request the nvidia.com/gpu resource. Taint the GPU pool so only GPU workloads schedule there, and add tolerations and node selectors to those pods. Efficiency matters because GPUs are expensive. Use the cluster autoscaler, with a minimum of zero, so GPU nodes exist only while jobs run; use spot GPU nodes for fault-tolerant training with checkpointing. Share GPUs where workloads are small: Multi-Instance GPU on supported hardware partitions a GPU into isolated slices, and time-slicing allows oversubscription for inference with less isolation. Right-size by matching GPU type to workload: smaller GPUs for inference, larger for training. Optimise the pipeline: keep data close to compute, use fast storage, batch inference requests, and use optimised runtimes and quantised models. Use job schedulers like Kueue or Volcano for queuing, fair sharing, and gang scheduling of distributed training. Monitor GPU utilisation and memory with DCGM exporter in Prometheus; low utilisation signals waste. Pre-pull large images to reduce start time, and watch driver and CUDA version compatibility. Consider quotas, since GPU capacity in a region can be limited, and request them early.

Likely follow-ups

- How does Multi-Instance GPU improve utilisation?
- Why scale GPU pools to zero when idle?

```
# GPU node pool, scale-to-zero when idle, tainted for GPU pods only:
```

```
az aks nodepool add -g rg --cluster-name prod-aks -n gpu \\
```

```
  --node-vm-size Standard_NC6s_v3 --node-taints nvidia.com/gpu=true:NoSchedule \\
```

```
  --enable-cluster-autoscaler --min-count 0 --max-count 8
```

```
# NVIDIA device plugin exposes GPUs; pod requests them:
```

```
resources: {limits: {nvidia.com/gpu: 1}}
```

```
# Consider spot GPU pools + checkpointing for training jobs
```

GPU efficiency on AKS centres on not paying for idle accelerators. Put GPUs in a **dedicated node pool** that is **tainted** (so only GPU workloads land there) and autoscales from **zero** — nodes spin up when GPU jobs queue and back down to zero when idle, since GPU VMs are very expensive. The NVIDIA device plugin advertises GPUs so pods request them via `nvidia.com/gpu` limits. For training, use **spot** GPU pools with checkpointing (huge savings, tolerate eviction) and consider GPU sharing/MIG for inference to pack multiple workloads onto one GPU.

certs.eknathalabs.com

Interview tip: the cost lever is scale-to-zero on a tainted GPU pool (idle GPUs are the budget killer), plus spot + checkpointing for training and MIG/time-slicing for inference packing. Mentioning the NVIDIA device plugin and taints (keep non-GPU pods off pricey nodes) shows you have actually run GPU workloads, not just read about them.

📚 **Official docs**

- Kubernetes: Scheduling GPUs
- Azure Kubernetes Service (AKS)

Q63 How do you implement network policies to restrict pod-to-pod traffic on AKS? Containers & AKS

🎙 **Spoken answer · ~84 s**

Kubernetes network policies control which pods can communicate, by default everything can talk to everything, so policies enable segmentation and least privilege. A NetworkPolicy selects pods and defines allowed ingress and egress rules based on pod labels, namespace labels, and IP blocks, plus ports. Once a pod is selected by any policy for a direction, all traffic in that direction not explicitly allowed is denied. On AKS, you need a network policy engine enabled at cluster creation: Azure Network Policy Manager, Calico, or Cilium, the last providing richer features like layer-seven rules and FQDN policies with eBPF. The usual approach: start with a default-deny policy in each namespace for ingress and egress, then add explicit allows: from the ingress controller's namespace to the frontend, from the frontend to the API on its port, from the API to the database, and egress to DNS, which is often forgotten and breaks name resolution when egress is denied. Use namespace labels for cross-namespace rules. Roll out safely: test in non-production, use observability tools such as Hubble with Cilium or flow logs to understand actual traffic before enforcing, and apply progressively. Manage policies as code, alongside application manifests, review changes, and test connectivity in CI.

Likely follow-ups

- Why must you allow DNS egress when denying egress by default?
- How does Cilium extend standard network policies?

```
# Enable a network policy engine at cluster creation (Azure NPM or Cilium):
```

```
az aks create -g rg -n prod-aks --network-plugin azure --network-policy cilium
```

```
# Default-deny ingress in a namespace, then allow specific flows:
```

```
kind: NetworkPolicy
```

```
spec:
```

```
  podSelector: {matchLabels: {app: api}}
```

certs.eknathalabs.com

```
  policyTypes: [Ingress]
```

```
  ingress:
```

```
  - from: [{podSelector: {matchLabels: {app: frontend}}}]
```

```
    ports: [{port: 8080}]
```

By default all pods can talk to all pods. **Network Policies** implement micro-segmentation — but only if the cluster runs a policy engine (Azure NPM or, increasingly, **Cilium**/eBPF), chosen at cluster creation. The strong pattern is **default-deny** per namespace, then explicit allow rules for legitimate flows (e.g. frontend→api on port 8080), so a compromised pod cannot freely move laterally. This is zero-trust networking inside the cluster, complementing NSGs (which operate at subnet level, not pod level). Cilium adds richer L7-aware policy and better performance.

Interview tip: two key facts — (1) network policy requires a policy engine enabled at cluster creation (you cannot add it trivially later), and (2) the secure design is default-deny + explicit allows for lateral-movement containment. Recommending Cilium (eBPF, L7-aware) for new clusters reflects current AKS direction and signals depth beyond basic NetworkPolicy YAML.

> 📚 **Official docs**
>
> - Kubernetes: Network Policies
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q64 What is Bicep and how does it compare to ARM templates and Terraform on Azure?

Compute

🎙 Spoken answer · ~88 s

Bicep is Azure's domain-specific language for declaring Azure resources, compiling to ARM templates. Compared with raw ARM JSON templates, Bicep is far more concise and readable, has cleaner syntax for parameters, variables, loops, and conditions, modules for reuse, automatic dependency handling, strong typing and editor tooling with validation and IntelliSense, and no need to hand-write JSON. It's a transparent abstraction: day-one support for new Azure resource types and API versions, since it maps directly to ARM. Deployments run through Azure Resource Manager with what-if previews, incremental mode, deployment stacks for lifecycle management, and no state file to store since Azure holds the deployment state. Terraform uses HCL with a provider model; it supports multiple clouds and many SaaS providers, has a mature module ecosystem, a plan and apply workflow, and requires managing state, typically in Azure Storage with locking. The azurerm provider can lag behind new Azure features, though azapi helps. Choose Bicep for Azure-only organisations that want first-class Azure support, no state management, and a tight fit with Azure tooling and policy. Choose Terraform for multi-cloud, multi-provider scenarios, or when the team already has Terraform skills and a large module estate. Whichever tool, use modules, parameterise environments, run what-if or plan in pull requests, apply least-privilege pipelines, and detect drift.

Likely follow-ups

- What advantage does Bicep have over raw ARM JSON?
- Why might Terraform lag on new Azure features?

```
// main.bicep — concise, typed DSL transpiled to ARM JSON
resource aks 'Microsoft.ContainerService/managedClusters@2024-02-01' = {
  name: 'prod-aks'  location: location
  properties: { dnsPrefix: 'prod'  agentPoolProfiles: [...] }
}
az deployment group create -g rg --template-file main.bicep
```

ARM templates are Azure's native JSON IaC — powerful but verbose and hard to read. **Bicep** is a concise, typed DSL that transpiles to ARM JSON: same engine and full day-one resource coverage, far better authoring (modules, type safety, simpler syntax), and no state file (Azure Resource Manager is the source of truth). **Terraform** is multi-cloud, has a rich provider ecosystem and explicit state management, and is preferred when you span clouds or want a single IaC tool everywhere. Rule of thumb: Bicep for Azure-only shops wanting native tooling and instant new-feature support; Terraform for multi-cloud or existing-Terraform organisations.

certs.eknathalabs.com

Interview tip: Bicep = "ARM made usable," Azure-native, stateless (ARM is the truth), day-one resource support; Terraform = multi-cloud, stateful, broad ecosystem. The decision driver is "Azure-only vs multi-cloud" and existing tooling/skills. Noting Bicep's immediate support for new Azure features (vs Terraform provider lag) and its lack of a state file to manage is the nuanced comparison.

📚 **Official docs**

- Bicep overview
- Azure Resource Manager overview

Q65 How do you debug intermittent image pull failures (ImagePullBackOff) on AKS?

Containers & AKS

🎙 **Spoken answer · ~86 s**

Intermittent ImagePullBackOff means the kubelet sometimes can't fetch an image. First read the pod events with kubectl describe pod for the exact error: unauthorized, not found, timeouts, or too many requests. Authentication: for ACR, check that the AKS kubelet managed identity has the AcrPull role on the registry, that the attachment is intact, and that role propagation isn't pending; for other registries check image pull secrets exist in the right namespace and haven't expired. Image reference: verify the registry, repository, and tag; a tag pushed late or deleted by retention policy, or a wrong architecture, would fail; use digests. Network: if the registry is private, check private endpoint and DNS resolution from the node, such as the privatelink zone linked to the right VNet; check firewall or NSG rules and UDR forcing traffic through a firewall that blocks the registry or its data endpoints and blob storage; SNAT port exhaustion on egress causes intermittent timeouts. Rate limits: Docker Hub throttling anonymous pulls causes intermittent failures; mirror images to ACR or use a pull-through cache. Node issues: disk full from images, so check node disk and garbage collection. Registry availability and throttling: ACR tier limits on concurrent pulls under mass scale-out; use geo-replication and a higher tier.

Likely follow-ups

- How do you check that the node can pull from ACR?
- Why can mass scale-out cause pull failures?

```
kubectl describe pod myapp-xxx | grep -A10 Events   # exact pull error
```

```
# Common causes & checks:
```

```
#  - auth: AKS identity lacks AcrPull on the registry
```

```
az aks check-acr -g rg -n prod-aks --acr myacr.azurecr.io
```

```
#  - wrong tag/digest or image deleted from ACR
```

```
#  - registry throttling / rate limits (Docker Hub anonymous pulls)
```

certs.eknathalabs.com

```
#  - private registry not reachable (Private Endpoint / firewall egress)
```

```
#  - node disk pressure evicting image cache
```

`ImagePullBackOff` means the kubelet cannot pull the image; `kubectl describe pod` Events shows the precise reason. Common causes: **authorization** (the AKS managed identity lacks the `AcrPull` role — verify with `az aks check-acr`), a **wrong/missing tag or digest**, **registry throttling** (notably anonymous Docker Hub rate limits — mirror to ACR), the **private registry being unreachable** (missing Private Endpoint/DNS or egress firewall rule blocking it), or **node disk pressure** evicting the image cache. Intermittent (not constant) failures point toward throttling, transient network/egress issues, or a flaky registry rather than a static misconfiguration.

Interview tip: lead with "read the describe Events for the exact error," then enumerate auth (AcrPull / `az aks check-acr`), tag/digest, rate-limiting (Docker Hub → mirror to ACR), private-registry reachability, and disk pressure. The *intermittent* qualifier specifically implicates throttling or transient egress — calling that out shows diagnostic precision.

> 📚 **Official docs**
>
> - Kubernetes: Debug Pods
> - Kubernetes: Images (pull policy, registries)
> - Kubernetes: Pull an image from a private registry

certs.eknathalabs.com

Q66 How do you plan and execute a migration of VMs/workloads from on-prem to Azure?

Compute

🎙 Spoken answer · ~83 s

A migration should be planned in phases. Assess and discover: inventory servers, applications, dependencies, performance, and licensing using Azure Migrate, which provides discovery, dependency analysis, sizing recommendations, and cost estimates; classify workloads by the six Rs: rehost, replatform, refactor, rearchitect, rebuild, retire. Plan: design the landing zone with networking, identity, governance, and security following the Cloud Adoption Framework; decide on connectivity, such as ExpressRoute or VPN; create a wave plan, grouping applications by dependency and risk, starting with low-risk candidates; define success criteria, rollback plans, and communication. Prepare: set up the landing zone with infrastructure as code, replicate identity, configure monitoring and backup, and handle compliance. Migrate: use Azure Migrate server migration for replicating VMs with minimal downtime, Database Migration Service for databases, and Data Box or AzCopy for bulk data. Test first with a test migration in an isolated network, validate functionality and performance, then plan the cutover window with final sync, DNS changes, and verification. Post-migration: optimise by right-sizing, using reserved instances, adopting managed services, enabling backup and monitoring, and decommissioning on-premises systems after a stabilisation period. Manage risks: dependencies, latency between on-premises and cloud during a hybrid period, licensing, and skills. Train the team and document runbooks.

Likely follow-ups

- What are the six Rs of migration?
- Why use waves to group applications?

```
# Assess: discover + dependency-map + right-size (Azure Migrate)
#   Azure Migrate appliance inventories on-prem, suggests target SKUs + cost
# Choose a strategy per workload (the 5/6 R's):
#   Rehost (lift-shift VMs)  | Replatform (e.g. -> AKS/PaaS) | Refactor
#   Repurchase (SaaS) | Retire | Retain
# Migrate: Azure Migrate replication -> test failover -> cutover
```

A disciplined migration runs Assess → Plan → Migrate → Optimize. **Assess** with Azure Migrate: discover servers, map dependencies (so you do not break inter-app communication), and right-size target SKUs with cost estimates. **Plan** a strategy per workload using the "R's": *Rehost* (lift-and-shift VMs — fastest), *Replatform* (move to PaaS/AKS for managed benefits), *Refactor* (re-architect cloud-native), *Repurchase*

certs.eknathalabs.com

(SaaS), *Retire*, *Retain*. **Migrate** via replication, then **test failover** in Azure before the real **cutover** to minimize downtime. **Optimize** afterward for cost and reliability. Group workloads into waves by dependency so related systems move together.

Interview tip: structure the answer as Assess (Azure Migrate: discovery, dependency mapping, right-sizing) → strategy per workload (the R's) → wave-based migration with test failover before cutover → post-migration optimization. Emphasising *dependency mapping* and *migration waves* (move interdependent apps together) is what separates a real migration plan from "just move the VMs."

📚 **Start here (official docs)**

- Microsoft Azure documentation

Q67 Pods are stuck in Pending on AKS. Walk through the diagnosis.

Troubleshooting

🎙 **Spoken answer · ~83 s**

Pending means the scheduler can't place the pod. Start with kubectl describe pod and read the Events section, which usually states the reason. Insufficient CPU or memory: no node has enough unallocated capacity for the pod's requests; check requests, node allocatable, and whether the cluster autoscaler is enabled and able to scale, or has reached its maximum; look for autoscaler events saying why it did not scale, such as quota limits, subnet IP exhaustion, or an unsatisfiable constraint. Taints and tolerations: nodes are tainted and the pod lacks tolerations. Node selectors, affinity, or anti-affinity: constraints match no node, or topology spread cannot be satisfied. Volume issues: a PersistentVolumeClaim is unbound, a storage class is missing, or the disk is in a different zone from the available nodes, a common problem with zonal Azure Disks. Resource quota or LimitRange violations at namespace level. Pod count or IP limits: maximum pods per node reached, or with Azure CNI, the subnet has no free IPs. Scheduler problems: priority and preemption, or custom schedulers. Regional capacity or VM quota: the node pool cannot add nodes because the subscription's core quota for that VM size is exhausted, visible in the scale set activity log.

Likely follow-ups

- Why can zonal disks leave a pod pending?
- Where do you see why the autoscaler did not add a node?

```
kubectl describe pod myapp-xxx | grep -A15 Events   # scheduler's reason is here
```

```
# Common reasons & checks:
```

```
#  - Insufficient cpu/memory: no node fits the requests
```

```
kubectl describe nodes | grep -A5 Allocated
```

```
#  - No node matches nodeSelector/affinity/taints
```

certs.eknathalabs.com

```
#  - PVC unbound (storage class / disk quota)
```

```
kubectl get pvc
```

```
#  - Cluster Autoscaler at max-count or failing to add nodes
```

```
kubectl -n kube-system logs -l app=cluster-autoscaler | tail
```

A **Pending** pod means the scheduler cannot place it — and `kubectl describe pod` Events states exactly why. The usual causes: **insufficient resources** (no node satisfies the pod's CPU/memory *requests* — and the Cluster Autoscaler is at max-count or cannot add nodes), **scheduling constraints** (nodeSelector/affinity/taints with no matching node), or an **unbound PVC** (storage class issue, disk quota, or zone mismatch between pod and volume). Follow the Events message to the specific cause rather than guessing.

Interview tip: always "read the scheduler's reason in describe Events first." Then map: insufficient resources → requests vs node capacity + autoscaler state; unschedulable due to affinity/taints → node labels/taints; volume not bound → PVC/storage class/zone. Mentioning the autoscaler-at-max and PVC-zone-affinity cases shows real AKS troubleshooting depth.

> 📚 **Official docs**
>
> - Kubernetes: Debug Pods
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q68 Users report intermittent 502/504 errors from an AKS app behind Application Gateway. How do you diagnose?

Troubleshooting

🎙 Spoken answer · ~87 s

Intermittent 502 and 504 errors behind Application Gateway need me to locate which side is failing. A 502 means the gateway received an invalid response or couldn't connect to the backend; a 504 means the backend didn't respond within the timeout. Start with Application Gateway metrics and diagnostic logs: backend health, failed requests, backend response time, and the access log fields such as the server status and timing. Check backend health probes: if probes fail intermittently, the pool marks pods unhealthy, so check the probe path, port, interval, and expected status codes, and whether the pod responds fast enough; custom probes should hit a lightweight health endpoint and use the right host header. Check pods: restarts, readiness failures, resource limits and CPU throttling causing slow responses, and recent deployments; with AGIC, pods that terminate might still receive traffic before the gateway updates, since AGIC updates via Azure Resource Manager with a delay, causing 502s during rollouts; fix with preStop sleep, graceful shutdown, and connection draining. Check timeouts: the gateway's request timeout, defaulting to twenty seconds, versus slow backend operations; align keep-alive settings between the gateway and the application to avoid reset connections. Check NSGs, network policies, and subnet sizing, and WAF rules blocking legitimate requests, shown in WAF logs.

Likely follow-ups

- Why do rollouts with AGIC cause 502 errors?
- How do probe misconfigurations create intermittent failures?

```
# Layered check: client -> Front Door/AppGW -> ingress -> Service -> pod
```

```
# 1. App Gateway backend health:
```

```
az network application-gateway show-backend-health -g rg -n appgw
```

```
# 2. Endpoints populated + pods Ready:
```

```
kubectl get endpoints myapp ; kubectl get pods -l app=myapp
```

```
# 3. Probe mismatch (AppGW health probe path/port vs app)
```

```
# 4. Pod restarts / OOM during traffic:
```

```
kubectl get events --sort-by=.lastTimestamp | grep -i -E 'oom|killed|restart'
```

```
# 5. Timeouts: AppGW request-timeout < app response time
```

502/504s behind App Gateway point to the gateway being unable to get a healthy response from the backend. Work the path top-down: check **App Gateway backend health** (unhealthy backends → probe misconfiguration: wrong path/port/host, or app health endpoint failing), confirm Kubernetes **Endpoints** are

certs.eknathalabs.com

populated and pods are **Ready** (empty Endpoints = readiness failing = 502s), look for **pod restarts/OOM** coinciding with errors (capacity issue under load), and check **timeouts** (504 when the app's response time exceeds App Gateway's request-timeout). "Intermittent" usually means capacity/scaling lag or a subset of unhealthy pods, not a total misconfiguration.

Interview tip: demonstrate layered diagnosis (AppGW health → Endpoints/readiness → pod restarts/OOM → timeouts) rather than jumping to one cause. 502 = backend unreachable/unhealthy (often readiness/probe), 504 = backend too slow (timeout). Tying "intermittent" to scaling lag or a few bad pods, and checking AppGW backend health first, is the senior move.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- Azure Application Gateway
- Azure Kubernetes Service (AKS)

Q69 What are the golden signals and how do you implement them for an AKS service?

Monitoring

🎙 **Spoken answer · ~88 s**

The four golden signals are latency, traffic, errors, and saturation. Latency is the time to serve requests, measured separately for successes and failures, using percentiles such as p95 and p99. Traffic is the demand on the system, such as requests per second. Errors are the rate of failed requests, explicit like HTTP 500s, or implicit like wrong content or slow responses beyond a threshold. Saturation is how full the service is: CPU, memory, connection pools, queue depth, and disk. For an AKS service, implement them like this. Instrument the application with OpenTelemetry or Application Insights to emit request counts, durations as histograms, and status codes, and expose Prometheus metrics; managed Prometheus scrapes them. Get edge-level signals from the ingress controller or Application Gateway metrics for requests, response codes, and backend latency. Saturation comes from Container Insights or Prometheus: CPU throttling, memory versus limits, pod restarts, HPA headroom, and thread or connection pool usage, plus dependencies' saturation like database DTU. Build Grafana dashboards per service showing the four signals, with deployment markers. Define SLOs from the latency and error signals, and alert on burn rates rather than raw thresholds. Add distributed tracing to locate where latency originates, and synthetic availability tests against key endpoints. Keep labels low-cardinality, and review dashboards after incidents.

Likely follow-ups

- Why track latency for successes and failures separately?
- How does saturation differ from utilisation?

```
# Four golden signals (per service):
```

```
#  Latency:     histogram_quantile(0.99, http_request_duration_seconds_bucket)
```

certs.eknathalabs.com

```
#  Traffic:     sum(rate(http_requests_total[5m]))
```

```
#  Errors:      sum(rate(http_requests_total{code=~"5.."}[5m])) / total
```

```
#  Saturation:  container CPU throttling, memory working set vs limit
```

```
# Managed Prometheus scrapes; Managed Grafana dashboards; alert on SLO burn
```

The four **golden signals** — latency, traffic, errors, saturation — are the minimal set that characterise a service's health. On AKS, instrument the app to expose Prometheus metrics (request duration histograms, request counts by status), scrape them with **Azure Managed Prometheus**, and visualise/alert via **Managed Grafana**. Latency uses high percentiles (P99, not average — averages hide tail pain); errors as a ratio of failed to total requests; saturation from CPU throttling and memory-vs-limit. Alert on **SLO burn rate** (you are consuming the error budget too fast) rather than raw thresholds, to reduce noise and catch real degradation.

Interview tip: name all four signals and how each is measured (P99 latency, request rate, 5xx ratio, throttling/memory-pressure for saturation). The senior addition is alerting on **SLO burn rate** not static thresholds — it ties monitoring to user-facing reliability and cuts alert fatigue. Mentioning Managed Prometheus + Managed Grafana keeps it Azure-specific.

> 📚 **Official docs**
>
> - Google SRE Book: Monitoring Distributed Systems
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q70 An AKS deployment rollout is stuck (not progressing). How do you investigate and recover?

Troubleshooting

🎙 Spoken answer · ~87 s

A stuck rollout means the new ReplicaSet isn't becoming available. Check kubectl rollout status and describe the deployment: conditions such as ProgressDeadlineExceeded show the rollout exceeded its deadline. Then see the new ReplicaSet's pods: kubectl get pods and describe them. Common causes: new pods are Pending, due to insufficient resources, quota, or scheduling constraints; ImagePullBackOff from a wrong tag or registry authentication; CrashLoopBackOff because of an application or configuration error; readiness probes never passing, so pods never count as available, while old pods stay because of maxUnavailable; a PodDisruptionBudget or surge constraints blocking progress; missing ConfigMaps or Secrets, or Key Vault CSI mount failures; admission webhooks or policies rejecting pods, visible in ReplicaSet events; resource quotas preventing new pods; or a PersistentVolume attached to the old pod and not releasable for the new one. Read events and logs of the new pods to find which. Recover: if the release is bad, roll back with kubectl rollout undo, which restores the previous ReplicaSet, and then fix forward in the pipeline; if the problem is capacity, add nodes or quota; if it's configuration, correct it and re-apply. To avoid recurrence, use sensible progressDeadlineSeconds, readiness probes, automated pipeline checks that wait for rollout success and roll back on failure, and staging verification.

Likely follow-ups

- What does ProgressDeadlineExceeded mean?
- How do maxSurge and maxUnavailable affect a stuck rollout?

```
kubectl rollout status deployment/myapp -n prod   # shows it's stuck
```

```
kubectl get pods -l app=myapp                      # new ReplicaSet pods state?
```

```
kubectl describe deployment myapp | grep -A5 Conditions   # ProgressDeadlineExceeded?
```

```
kubectl describe pod <new-pod>                     # why new pods won't go Ready
```

```
# Recover:
```

```
kubectl rollout undo deployment/myapp              # roll back to last good ReplicaSet
```

```
kubectl rollout history deployment/myapp
```

A stuck rollout means the new ReplicaSet's pods are not becoming Ready, so the Deployment will not progress (and after `progressDeadlineSeconds` reports `ProgressDeadlineExceeded`). Investigate the new pods: are they **Pending** (scheduling/resources), **CrashLoopBackOff** (app failing on the new version), or **not passing readiness** (bad config, failing dependency)? `kubectl describe` on the deployment (Conditions) and

certs.eknathalabs.com

the new pods pinpoints it. The rolling-update strategy intentionally protects you — it will not tear down old healthy pods until new ones are Ready — so the old version keeps serving. Recover quickly with `kubectl rollout undo` to the previous ReplicaSet, then debug the new version offline.

Interview tip: the reassuring fact to state: a rolling update will not kill the old version until new pods are Ready, so a stuck rollout usually is *not* an outage — buy time with `rollout undo`, then diagnose. Identifying the new-pod failure mode (Pending vs CrashLoop vs readiness) and knowing `maxUnavailable`/`maxSurge` govern the rollout shows command of deployments.

📚 **Official docs**

- Kubernetes: Deployments
- Azure Kubernetes Service (AKS)

Q71 How do you write effective KQL queries in Log Analytics for AKS incident investigation?

Monitoring

🎙 **Spoken answer · ~82 s**

KQL, the Kusto Query Language, queries Log Analytics tables in a pipeline of operators. For AKS investigation, know the main tables: ContainerLogV2 for container logs, KubePodInventory for pod state, KubeEvents for Kubernetes events, KubeNodeInventory, InsightsMetrics and Perf for metrics, and AzureDiagnostics or resource-specific tables for control plane logs such as the audit log. Effective habits: always filter by time range first, then by cluster, namespace, or pod, to keep queries fast and cheap; use where to filter early, project to select only needed columns, summarize with bin to aggregate over time, and render timechart for visualisation. Examples: find errors, filtering ContainerLogV2 for a namespace and a log message containing error, summarising counts by pod over five-minute bins; restart analysis, using KubePodInventory to find pods with high restart counts; node memory trends from Perf by joining on computer; Kubernetes warnings from KubeEvents filtered by reason such as BackOff, FailedScheduling, or Evicted; and join KubePodInventory to logs to correlate. Use parse_json for structured logs, extend for computed fields, and let and materialize for reuse. In an incident, start broad to find the time window and affected components, then narrow to messages around the first occurrence, then correlate with deployments.

Likely follow-ups

- Which tables hold container logs and Kubernetes events?
- Why filter on time first in KQL?

```
// Error spike by container over time:
```

```
ContainerLogV2 | where LogMessage has 'ERROR'
```

```
| summarize count() by bin(TimeGenerated, 5m), ContainerName | render timechart
```

certs.eknathalabs.com

```
// OOMKilled pods:
KubePodInventory | where ContainerStatusReason == 'OOMKilled'
| project TimeGenerated, Namespace, Name, Node=Computer
// Correlate node + pod:
KubeNodeInventory | join kind=inner KubePodInventory on Computer
```

KQL (Kusto Query Language) is the query engine for Log Analytics. Effective incident KQL follows a pattern: **filter** early (time range + `where` on the relevant table/field), **summarize** with `bin()` for time-bucketed aggregation, `render` as a timechart to spot the onset, and **join** tables (e.g. `KubePodInventory` with `KubeNodeInventory`) to correlate pod issues with node conditions. Key AKS tables: `ContainerLogV2` (app logs), `KubePodInventory`/`KubeNodeInventory` (object state), `KubeEvents` (cluster events). Use `has`/`contains` for log text and `summarize ... by bin(TimeGenerated, 5m)` to see when an issue began.

Interview tip: the method matters more than memorising tables — filter→summarize-by-bin→render to find onset, then join to correlate (pod↔node). Naming the AKS tables (`ContainerLogV2`, `KubePodInventory`, `KubeEvents`) and the OOMKilled query shows you have actually investigated AKS incidents in Log Analytics, not just read docs.

> 📚 **Official docs**
>
> - Azure Monitor
> - Kusto Query Language (KQL)
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q72 A node has disk pressure and is evicting pods. How do you diagnose and resolve it?

Troubleshooting

🎙 Spoken answer · ~88 s

Disk pressure means the node's disk space or inodes fell below the kubelet's eviction thresholds, so the kubelet evicts pods and marks the node with DiskPressure. Diagnose with kubectl describe node for conditions and eviction events, then inspect the node: use kubectl debug node or a privileged debug pod to run df and du, and check inode usage with df -i. Typical culprits: container logs growing without rotation, especially verbose applications; unused images and layers accumulating, because garbage collection hasn't reclaimed them; ephemeral storage use by pods writing to emptyDir or the container filesystem; core dumps; and large temp files. On AKS the OS disk size and type matter, since small OS disks fill quickly; consider ephemeral OS disks sized adequately. Resolve immediately by cordoning the node, draining or letting evicted pods reschedule, cleaning up images with crictl rmi --prune, and removing large logs, or simply replace the node by reimaging, since nodes are disposable. Fix the cause: set ephemeral-storage requests and limits on pods so one pod can't fill the node; configure log rotation, and limit application log verbosity; send logs to stdout and ship them off the node; use larger OS disks; tune kubelet image garbage collection thresholds; and use persistent volumes for data instead of node-local storage.

Likely follow-ups

- How do ephemeral-storage limits protect a node?
- What is the quickest way to recover a node with a full disk?

```
kubectl describe node mynode | grep -A5 Conditions   # DiskPressure=True
```

```
kubectl get events | grep -i evict
```

```
# Inspect node disk (via debug pod):
```

```
kubectl debug node/mynode -it --image=mcr.microsoft.com/cbl-mariner/busybox:2.0
```

```
chroot /host; df -h; du -sh /var/lib/containerd/* /var/log/*
```

```
# Common culprits: image cache bloat, runaway container logs, ephemeral PV growth
```

```
# kubelet GC settings: imageGCHighThresholdPercent, evictionHard nodefs
```

DiskPressure on a node triggers kubelet eviction of pods to reclaim space, causing disruption. Diagnose by confirming the condition (`describe node` → Conditions) and inspecting what filled the disk via a debug pod: usually **image cache bloat** (`/var/lib/containerd`), **runaway container logs** (apps logging excessively, no rotation), or **ephemeral storage** growth (emptyDir/temp files). Resolve immediately by reclaiming space (kubelet image GC, log rotation), and durably by setting log limits, sizing node OS/ephemeral disks adequately, capping ephemeral-storage requests/limits, and tuning kubelet GC thresholds. Excessive app logging is a frequent root cause.

certs.eknathalabs.com

Interview tip: name the eviction mechanism (DiskPressure → kubelet evicts), then the top culprits: image cache, container logs without rotation, ephemeral storage. Durable fixes (log rotation/limits, ephemeral-storage limits, bigger/managed OS disk, kubelet GC tuning) beyond the immediate cleanup show you fix root cause, not just symptoms.

📚 **Official docs**

- Kubernetes: Node-pressure eviction

Q73 How do you set up SLOs, error budgets, and burn-rate alerting on Azure?

Monitoring

🎙 **Spoken answer · ~83 s**

Start with the user experience: choose the critical journeys, such as login and checkout, and define SLIs as ratios of good events to total, for instance the proportion of requests that succeed and complete within a latency threshold. On Azure, SLIs come from Application Insights request telemetry, Application Gateway or Front Door metrics, Prometheus metrics from the ingress, and synthetic availability tests. Set SLOs based on user needs and current performance, such as ninety-nine point nine percent availability over thirty days, with an error budget of the remaining point one percent. Compute the budget burn rate, the speed at which the budget is consumed relative to the allowable pace. Implement with KQL queries in Log Analytics or PromQL in managed Prometheus, as recording rules computing the error ratio over multiple windows. Alert on burn rate with multiple windows: a fast alert, for example a fourteen times burn over one hour with a five-minute confirmation window, pages someone; a slower alert, six times over six hours, creates urgent tickets; and a slow burn over days creates tickets. This catches both sudden and gradual degradation with few false positives. Route through Azure Monitor action groups. Build dashboards showing budget remaining.

Likely follow-ups

- Why use multiple windows for burn-rate alerts?
- What happens operationally when the error budget is exhausted?

```
# Define SLI (e.g. availability = good requests / total) and SLO (99.9% / 30d)
```

```
# Error budget = 1 - SLO = 0.1% of requests may fail over the window
```

```
# Burn-rate alert (Managed Prometheus): fast burn = page, slow burn = ticket
```

```
// fast burn (2% of budget in 1h):
```

```
(1 - (sum(rate(good[5m]))/sum(rate(total[5m])))) > 14.4 * (1-0.999)
```

```
# Track budget consumption on a Managed Grafana SLO dashboard
```

cors.eknathalabs.com

An **SLO** is a target on a service-level indicator (e.g. 99.9% of requests succeed over 30 days). The **error budget** is the allowable failure (0.1% here) — a concrete budget you can "spend." **Burn-rate alerting** fires based on how fast you are consuming that budget: a fast burn (e.g. 2% of the monthly budget in an hour) pages immediately, while a slow burn raises a low-urgency ticket. Implement SLIs as Prometheus queries, compute burn rate, and alert via Managed Prometheus rules, visualising budget consumption in Managed Grafana. This replaces noisy static thresholds with alerts tied directly to user-facing reliability.

Interview tip: define the chain SLI → SLO → error budget → burn-rate alert, and contrast multi-window burn rates (fast burn pages, slow burn tickets) with naive static thresholds. The point interviewers want: alert on *budget burn*, not raw CPU/error spikes — it aligns paging with actual user impact and the error-budget policy governs release decisions.

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Google SRE Workbook: Implementing SLOs
- Google SRE Book: Embracing Risk (error budgets)

Q74 Cross-region or cross-AZ latency is degrading an AKS app. How do you investigate?

Troubleshooting

🎙 **Spoken answer · ~90 s**

Cross-region or cross-zone latency problems show up as slow calls between components that are placed far apart. First, quantify: use distributed traces to find which hops are slow and compare their durations to the network round trip time; check where each component runs by zone and region. Typical causes: chatty services making many sequential calls across zones, where each hop adds around a millisecond or two, multiplying under fan-out; a database in one region accessed by an application in another; pods scheduled across zones with no topology awareness; traffic hairpinning through a central firewall or via the hub; and cross-region dependencies like a cache or API. Measure with network tools: test latency between pods and between nodes in different zones, and check Network Watcher connection monitor to track latency over time. Fixes: co-locate chatty components in the same zone using topology-aware routing, which prefers endpoints in the same zone; use zone-aware replicas with enough capacity per zone; place read replicas or caches near the application; reduce call counts via batching, caching, or aggregating calls; use asynchronous communication where acceptable; ensure private endpoints and peering are used instead of public routing; and review the firewall path. Balance against resilience, since pinning to one zone reduces availability. Validate improvements with traces, and set latency alerts per path.

Likely follow-ups

- What is topology-aware routing?
- How do fan-out calls amplify small per-hop latency?

```
# Is traffic crossing zones/regions unnecessarily?
```

certs.eknathalabs.com

```
#  - pods + their DB/cache in the SAME zone? (topology-aware routing)
```

```
kubectl get pods -o wide   # node -> zone mapping
```

```
# Enable topology-aware hints so Services prefer same-zone endpoints:
```

```
#   service.kubernetes.io/topology-mode: Auto
```

```
# Check DB/cache region vs cluster region; Private Endpoint in correct region
```

```
# Measure: distributed traces (App Insights) to find the slow hop
```

Latency from topology usually means traffic is crossing zone or region boundaries it should not. Investigate where each hop lives: are pods and their **database/cache in the same zone/region**, or is the app in one region calling a data store in another? Use **topology-aware routing** (topology-aware hints) so Kubernetes Services prefer **same-zone** endpoints, reducing cross-AZ hops (which also cuts cross-AZ data-transfer cost). Verify PaaS dependencies (SQL, Cosmos, cache) are in the cluster's region with correctly-placed Private Endpoints. **Distributed tracing** (Application Insights) pinpoints exactly which hop is slow rather than guessing.

Interview tip: two angles — (1) is traffic needlessly crossing AZ/region (fix with topology-aware routing / co-locating data), and (2) measure the actual slow hop with distributed tracing rather than assuming. Noting that cross-AZ traffic costs money *and* latency, and that topology-aware hints keep traffic same-zone, connects performance to cost — a platform-level insight.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q75 How do you implement distributed tracing for microservices on AKS?

Monitoring

🎙 Spoken answer · ~88 s

Distributed tracing follows a request across microservices so you can see the path and timing. On AKS, the modern approach is OpenTelemetry. Instrument each service with the OpenTelemetry SDK or auto-instrumentation, which creates spans for incoming requests and outgoing calls to databases, HTTP services, and message queues, and propagates trace context using the W3C traceparent header across HTTP and message properties. Export spans to a collector: deploy the OpenTelemetry Collector in the cluster as a DaemonSet or deployment, which batches, samples, and enriches data and exports it to Azure Monitor Application Insights using the Azure Monitor exporter, giving the application map, transaction search, and end-to-end transaction views. Application Insights also supports codeless auto-instrumentation for some languages on AKS. Alternatives are Jaeger or Tempo with Grafana. Practical points: ensure context propagation through all hops, including asynchronous ones like Service Bus; add useful attributes such as tenant, version, and operation names; correlate logs by including the trace ID in structured logs so you can jump between traces and logs; use sampling to control volume and cost, such as head sampling for a percentage, with tail sampling in the collector to keep errors and slow traces; set service names and resource attributes for clear maps. Use traces to find bottlenecks, understand dependencies, and verify deployments.

Likely follow-ups

- What does trace context propagation require across message queues?
- What is tail sampling and why use it?

```
# Instrument with OpenTelemetry (vendor-neutral), export to App Insights:
#   OTEL SDK in app -> OTEL Collector (DaemonSet) -> Azure Monitor/App Insights
# Propagate trace context (W3C traceparent) across service calls
# Application Insights distributed trace view: end-to-end span waterfall
# Correlate traces with logs (trace_id) and metrics for full context
```

Distributed tracing follows a single request across service boundaries, revealing where latency and errors actually occur in a microservice call chain. The modern approach uses **OpenTelemetry** (vendor-neutral instrumentation) in each service, propagating **W3C trace context** (`traceparent` header) so spans link into one trace, exported via an OTel Collector to **Application Insights** / Azure Monitor. The App Insights trace view shows the end-to-end span waterfall — making it obvious that, say, a 2s request spent 1.8s in a downstream database call. Correlating `trace_id` across traces, logs, and metrics gives full-context debugging.

Interview tip: emphasise OpenTelemetry (avoid vendor lock-in) + W3C context propagation + export to App Insights, and the payoff: you find the *slow hop* in a microservice chain instead of guessing per-service. Mentioning trace↔log↔metric correlation via `trace_id` (the "three pillars of observability" tied together) is

certs.eknathalabs.com

the senior framing.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)
- Azure Kubernetes Service (AKS)

Q76 A Managed Identity / Workload Identity token is failing in a pod. How do you debug auth failures?

Troubleshooting

🎙 **Spoken answer · ~87 s**

When a managed or workload identity token fails in a pod, I trace the chain from the pod to the permission. For Workload Identity: check the service account has the correct azure.workload.identity/client-id annotation and the pod has the label azure.workload.identity/use set to true; verify the mutating webhook injected the environment variables and projected token volume, with kubectl describe pod, looking for AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_FEDERATED_TOKEN_FILE, and AZURE_AUTHORITY_HOST. Check the federated identity credential: the issuer must match the cluster's OIDC issuer URL exactly, and the subject must match system:serviceaccount:namespace:name, with the right audience; typos or a recreated cluster with a new issuer are common errors. Check that the identity has the needed Azure RBAC role at the right scope, and allow for propagation delays; confirm the target service allows the identity, for example Key Vault with RBAC enabled or database users created for the identity. Check the client library version supports workload identity, and the code uses DefaultAzureCredential or the right credential. Look at the exact error: AADSTS errors in the response identify issues such as invalid audience, no matching federated credential, or wrong tenant. Check network access to login.microsoftonline.com and to the resource, firewall rules, and private endpoints. For managed identity via IMDS on VMs, check the identity is assigned.

Likely follow-ups

- What must the federated credential's subject match?
- Where do AADSTS error codes help?

```
# Symptom: app gets 401/403 calling Key Vault/Storage/SQL
```

```
kubectl describe sa app-sa -n prod   # has azure.workload.identity/client-id annotation?
```

```
kubectl describe pod | grep -i 'azure.workload.identity/use'  # label present?
```

```
# Federated credential subject must match exactly:
```

```
#   system:serviceaccount:<ns>:<sa-name>
```

```
az identity federated-credential list -g rg --identity-name app-mi
```

certs.eknathalabs.com

```
# Target RBAC: does the identity have the role on the resource?
```

```
az role assignment list --assignee <client-id> --scope <resource-id>
```

Workload Identity auth failures (401/403 from Key Vault/Storage/SQL) usually trace to one of three mismatches. (1) **Service account wiring**: the K8s service account must have the `azure.workload.identity/client-id` annotation and the pod must use that SA with the `azure.workload.identity/use` label. (2) **Federated credential subject mismatch**: the federated credential's subject must *exactly* equal `system:serviceaccount:<namespace>:<sa-name>` — a namespace or name typo breaks token exchange. (3) **Missing target RBAC**: the managed identity authenticated fine but lacks the role (e.g. Key Vault Secrets User) on the specific resource. Check each in order.

Interview tip: structure it as authentication vs authorization. Auth (getting a token): SA annotation + pod label + *exact* federated-credential subject (`system:serviceaccount:ns:sa`). Authorization (using it): the identity needs the right Azure role on the target resource. The most common silent bug is the federated subject typo — calling that out demonstrates real Workload Identity debugging.

📚 **Official docs**

- Azure: Managed identities

Q77 How do you reduce alert fatigue and design an effective on-call alerting strategy on Azure?

Monitoring

🎙 **Spoken answer · ~85 s**

Alert fatigue comes from too many low-value alerts, so the principle is that a page should demand human action, right now, for something affecting users. Redesign around symptoms: alert on SLO burn rates for key journeys, such as error ratio and latency, instead of on every CPU spike. Keep cause-based alerts as dashboards or low-severity tickets unless they reliably precede impact. For every alert, require an owner, a severity, and a runbook, and ask whether the response can be automated, in which case automate instead of paging. Reduce noise technically: use appropriate evaluation windows and thresholds, dynamic thresholds for variable baselines, grouping and deduplication through alert processing rules, suppression during planned maintenance, and correlation so one failure produces one page rather than fifty. Route by severity through action groups: pages for critical, chat notifications for warnings, tickets for informational. Use Azure Monitor alerts with integrations to PagerDuty or Teams. Healthy on-call practice: balanced rotations with enough people, follow-the-sun if possible, handoff notes, time off after rough shifts, and fair compensation. Measure: pages per shift, percentage actionable, time to acknowledge and resolve, and repeat alerts. Hold regular alert reviews where every noisy alert is tuned or deleted, and after each incident ask what detection was missing.

Likely follow-ups

- How do you decide whether an alert should page or just create a ticket?
- What metrics tell you whether your alerting is healthy?

certs.eknathalabs.com

```
# Principles:
#  - alert on SYMPTOMS (user-facing SLO burn), not every cause
#  - every page must be ACTIONABLE and URGENT; else ticket/dashboard
#  - group related alerts (Azure Monitor Smart Groups)
#  - suppress during maintenance (alert processing rules)
#  - severity tiers -> action groups (P1 page, P3 email)
```

Alert fatigue comes from too many non-actionable, cause-based alerts. The fixes: alert on **symptoms** (user-facing SLO burn) rather than every internal cause, so one page corresponds to real impact instead of dozens of correlated cause-alerts; make every **page actionable and urgent** — if it is not, downgrade it to a ticket or dashboard; **group** related alerts (Azure Monitor Smart Groups cluster correlated alerts into one); **suppress** during known maintenance (alert processing rules) without disabling; and use **severity tiers** routed to different action groups (P1 pages PagerDuty, P3 emails). The test for any alert: "does a human need to do something now?"

Interview tip: the core principle is "page on symptoms (SLO burn), not causes," and "every page must be actionable+urgent." Add tactical Azure mechanisms (Smart Groups for de-duplication, alert processing rules for maintenance suppression, severity→action-group routing). Framing on-call health as a designed system, with the "actionable now?" test, is exactly the SRE maturity interviewers probe.

> 📚 **Official docs**
>
> - Google SRE Book: Practical Alerting
> - Google SRE Workbook: On-Call

certs.eknathalabs.com

Q78 An Azure SQL / database-backed AKS app is hitting connection pool exhaustion. How do you fix it?

Troubleshooting

🎙 Spoken answer · ~86 s

Connection pool exhaustion means the application is waiting for or can't get database connections, causing timeouts. Diagnose from both sides. Application: errors like timeout waiting for a connection from the pool; metrics on active, idle, and waiting connections; pool size settings; connection leaks, where code doesn't release connections, often from missing disposal or long transactions held across slow calls. Database: Azure SQL limits concurrent sessions and workers based on the service tier, so check sessions with dynamic management views, blocked sessions, long-running queries, and metrics like connection failed and worker percentage; with many pods each holding a pool, total connections multiply: pods times pool size can exceed the limit during scale-out. Fixes. Right-size pools: reduce per-instance pool size so the total across replicas fits within the limit with headroom; set sensible connection timeouts and lifetimes. Fix leaks and shorten transactions: always release connections promptly, avoid holding them during external calls. Optimise slow queries and indexes, since slow queries hold connections longer, and resolve blocking. Add pooling infrastructure where applicable, such as PgBouncer for PostgreSQL, or use a connection-efficient driver pattern. Scale the database tier if load genuinely exceeds capacity, and use read replicas for read traffic. Add retry logic with backoff for transient errors, and circuit breakers.

Likely follow-ups

- Why does autoscaling pods make pool exhaustion more likely?
- How do you find connection leaks?

```
# Symptom: 'timeout acquiring connection' / max pool size reached under load
```

```
# Check: pods x replicas x per-pod pool size vs DB max connections
```

```
#   e.g. 20 pods x 100 pool = 2000 > Azure SQL connection limit
```

```
# Fixes:
```

```
#  - right-size per-pod pool; ensure connections are RETURNED (no leaks)
```

```
#  - add a proxy/pooler (PgBouncer for PostgreSQL) to multiplex
```

```
#  - scale DB tier or use read replicas for read-heavy load
```

Connection pool exhaustion appears as "timeout acquiring connection" or max-pool-reached errors under load. The arithmetic is the key insight: total connections = replicas × per-pod pool size, which can blow past the database's connection ceiling as pods scale out (20 pods × 100-connection pools = 2000 connections — far above many tiers). Causes are over-sized per-pod pools, connection **leaks** (connections borrowed but never returned), or simply more replicas than the DB can serve. Fixes: right-size the per-pod pool relative to

certs.eknathalabs.com

replica count, fix leaks (always release connections), introduce a **connection pooler/proxy** (e.g. PgBouncer for PostgreSQL) to multiplex many app connections onto few DB connections, and scale the DB tier or add read replicas for read-heavy load.

Interview tip: lead with the math — replicas × pool-size vs DB max connections — because that scaling interaction is what surprises teams when pods autoscale. Then: right-size pools, fix leaks, add a pooler (PgBouncer) to multiplex, scale tier/read-replicas. The pooler-to-multiplex answer is the one that signals real database-at-scale experience.

📚 **Official docs**

- PgBouncer
- Azure Kubernetes Service (AKS)
- Azure SQL documentation

Q79 How do you monitor and control Azure costs in real time (FinOps on AKS)? Monitoring

🎙 **Spoken answer · ~85 s**

Real-time cost control on AKS needs visibility, attribution, and automatic guardrails. Visibility: Azure Cost Management shows spend by resource group, tag, and the AKS cost analysis add-on breaks cluster costs down by namespace, workload, and node pool, including idle costs; tools like Kubecost or OpenCost give pod-level allocation and recommendations. Attribution: tag Azure resources and label Kubernetes workloads consistently with team, environment, and application, so spend can be assigned to owners, with showback reports. Budgets and alerts: set Azure budgets per subscription, resource group, or tag with alert thresholds on actual and forecast spend, plus anomaly alerts for sudden jumps; send to the owners. Guardrails: Azure Policy to restrict VM sizes and regions; ResourceQuotas and LimitRanges per namespace; autoscaler maximums on node pools; separate node pools with caps for experiments; automatic scale-down of non-production at night. Optimisation levers: right-size requests, use spot pools, reservations or savings plans for baseline, efficient storage tiers, and log ingestion controls. Process: weekly or monthly reviews with teams, tracking unit costs such as cost per request or per tenant, and recording savings. Automate reports. Respond to anomalies with a runbook: identify the resource, check recent changes, and contain. FinOps works best when engineers see their own costs and are accountable.

Likely follow-ups

- How does the AKS cost analysis add-on attribute cost?
- Which guardrails stop runaway spend automatically?

```
# Cost visibility:
```

```
#  - Azure Cost Management: budgets, alerts, anomaly detection
```

```
#  - tags (team/env/cost-center) for showback/chargeback
```

certs.eknathalabs.com

```
#  - OpenCost / Kubecost on AKS for per-namespace/pod cost allocation
```

```
az consumption budget create --budget-name prod --amount 50000 --time-grain Monthly
```

```
# Controls: spot pools, autoscaler bounds, reservations/savings plans, rightsizing
```

FinOps on Azure combines visibility, allocation, and control. **Visibility**: Azure Cost Management for budgets, cost alerts, and anomaly detection; consistent **tagging** (team/env/cost-center) so spend maps to owners. For AKS specifically, shared cluster cost is opaque — tools like **OpenCost/Kubecost** allocate cost per namespace/pod/team, enabling showback/chargeback. **Control**: spot node pools for tolerant workloads, Cluster Autoscaler bounds and rightsized requests (the biggest waste source), and **Reservations/Savings Plans** for predictable baseline (40–72% off). The cycle is inform → allocate → optimize, made continuous with budgets and anomaly alerts.

Interview tip: structure as visibility (Cost Management + tagging + Kubecost/OpenCost for per-namespace AKS allocation) → control (spot, rightsizing, reservations/savings plans). The AKS-specific insight is that a shared cluster hides per-team cost until you add allocation tooling. Tying rightsized *requests* to both cost and reliability, and naming reservations vs savings plans, reads as platform/FinOps maturity.

> 📚 **Official docs**
>
> - Microsoft Cost Management
> - AWS Well-Architected: Cost optimization pillar
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q80 DNS resolution is failing intermittently inside AKS pods. How do you diagnose?

Troubleshooting

🎙 Spoken answer · ~85 s

Intermittent DNS failures in pods are usually CoreDNS capacity, network, or configuration issues. First reproduce and measure: from a debug pod, run repeated nslookup or dig queries against the cluster DNS service and an external name, noting failure rates, latency, and which names fail. Check CoreDNS: pod status, restarts, logs, and metrics; CPU throttling or too few replicas under load; scale replicas or autoscale them with the DNS autoscaler; check its configmap for forwarders and loops. Check node-level issues: the conntrack table full, or a known Linux race condition with UDP DNS in conntrack causing five-second delays, mitigated by NodeLocal DNSCache, which runs a caching agent on each node and uses TCP upstream; this reduces load and latency considerably. Check the ndots setting: the default of five causes many search-domain queries for external names, multiplying load; set ndots lower or use fully qualified names with a trailing dot. Check upstream: if CoreDNS forwards to Azure DNS or a custom DNS server, verify its health, throttling limits, and reachability; Azure DNS has a limit per VM on queries per second. Check network policies blocking DNS egress to kube-system, and NSG or firewall rules for custom DNS, plus private DNS zone links for private endpoints.

Likely follow-ups

- What does NodeLocal DNSCache solve?
- Why does a high ndots value inflate DNS traffic?

```
# Test from a pod:
```

```
kubectl run dnsutils --rm -it --image=registry.k8s.io/e2e-test-images/jessie-dnsutils:1.3
\\
```

```
  -- nslookup kubernetes.default
```

```
# Check CoreDNS health + load:
```

```
kubectl -n kube-system get pods -l k8s-app=kube-dns
```

```
kubectl -n kube-system logs -l k8s-app=kube-dns | tail
```

```
# Common causes: CoreDNS under-provisioned, ndots:5 search-domain amplification,
```

```
#   conntrack table full, upstream (Azure DNS / private zone) issues
```

Intermittent in-pod DNS failures are a classic AKS pain point. Diagnose by testing resolution from a debug pod and checking **CoreDNS** health/logs in `kube-system`. Common causes: CoreDNS **under-provisioned** for query volume (add replicas / enable autoscaling, or use **NodeLocal DNSCache** to offload), the **ndots:5** default causing search-domain amplification (each external lookup tries several suffixes first — fix with fully-

certs.eknathalabs.com

qualified names or tuned `dnsConfig`), **conntrack** table exhaustion on busy nodes dropping UDP DNS packets, or upstream issues resolving Private DNS Zones. Intermittency usually points to load/capacity (CoreDNS or conntrack) rather than a static misconfig.

Interview tip: the AKS-savvy answer names CoreDNS capacity (scale it / NodeLocal DNSCache), the `ndots:5` search-domain amplification (use FQDNs to cut lookups), and conntrack exhaustion dropping UDP under load. Recommending **NodeLocal DNSCache** as the durable fix for intermittent DNS at scale is a strong, specific signal.

📚 **Official docs**

- Kubernetes: Debugging DNS resolution
- Kubernetes: DNS for Services and Pods
- RFC 1035: DNS implementation and specification

Q81 How do you centralize logs across multiple AKS clusters and subscriptions? Monitoring

🎙 **Spoken answer · ~85 s**

Centralising logs across clusters and subscriptions starts with deciding the topology. Options: one Log Analytics workspace per environment or region shared by many clusters, which makes cross-cluster queries simple and keeps cost management central; or per-team workspaces for isolation, with cross-workspace queries when needed. Enable Container Insights or the Azure Monitor agent on each cluster with data collection rules that point to the chosen workspace, and send control plane diagnostic logs through diagnostic settings to the same place. Standardise on structured JSON logs with fields for cluster, namespace, service, and trace ID so queries work uniformly. For access control, use resource-context or table-level RBAC so teams see only their own logs. Control cost with data collection rules that filter namespaces and noisy containers, use the basic logs tier for verbose data, and set retention, archiving older data to storage. For multi-subscription estates, use Azure Lighthouse or a central management subscription, and policy to enforce diagnostic settings automatically on every new cluster. If you need search at scale or integration with other tools, stream through Event Hubs to Sentinel, Splunk, or Elastic. Query across clusters with KQL, dashboard through workbooks or Grafana, and alert centrally. Document the standard and test that new clusters are onboarded automatically.

Likely follow-ups

- When would you use separate workspaces per team?
- How does policy ensure every cluster sends logs?

certs.eknathalabs.com

```
# Container Insights -> Log Analytics workspace(s)
#  - one central workspace, or per-team workspaces + cross-workspace KQL:
union workspace('ws-a').ContainerLogV2, workspace('ws-b').ContainerLogV2
# Governance via Azure Policy: enforce Container Insights on all AKS
# Retention/cost: tiered (interactive vs archive), table-level retention
# Diagnostic settings route platform logs/metrics to the workspace too
```

Centralised logging across clusters/subscriptions uses **Log Analytics workspaces** fed by Container Insights and resource **diagnostic settings**. Two patterns: a single central workspace (simplest querying, but governance/cost concentration) or per-team/per-environment workspaces with **cross-workspace KQL** (`union workspace(...)` across two workspaces) for global queries while preserving access boundaries. Enforce coverage with **Azure Policy** (require Container Insights on every AKS so no cluster is unmonitored), and manage cost/retention with tiered storage (interactive vs cheap archive) and table-level retention. Diagnostic settings also route platform (control-plane) logs and metrics into the same workspace for correlation.

Interview tip: address the central-vs-federated workspace trade-off (single = easy queries; per-team + cross-workspace KQL = access isolation), enforcement via Azure Policy (no unmonitored clusters), and cost control via tiered retention. Mentioning cross-workspace `union` queries and policy-enforced coverage shows you think about observability at *fleet* scale, not one cluster.

> 📚 **Official docs**
>
> - RHEL 9 documentation
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q82 A scheduled job / CronJob on AKS is not running or is overlapping. How do you debug?

Troubleshooting

🎙 Spoken answer · ~86 s

For a CronJob that isn't running, check the basics first: kubectl get cronjob to see the schedule, last schedule time, and whether it is suspended. Verify the cron expression and time zone, since schedules are UTC unless a time zone field is set. Look at the jobs and pods it created and their events. If no jobs are created, causes include a suspended CronJob, a missed schedule with startingDeadlineSeconds exceeded, or more than one hundred missed schedules, which makes the controller stop. If jobs are created but pods don't run, treat it like any pod: pending due to resources, image pull errors, or failing mounts. If pods fail, read logs and check backoffLimit and activeDeadlineSeconds. For overlapping runs, the concurrencyPolicy decides: Allow permits concurrent runs, Forbid skips a new run if the previous is still running, and Replace cancels the old one. Set Forbid or Replace when overlap is harmful, and make jobs idempotent anyway, since a job may run twice or be missed. If runs are overlapping because jobs take longer than the interval, optimise the job, lengthen the schedule, or add locking. Set history limits to clean up finished jobs, resource requests and limits, and alerts on job failure or missing runs using kube-state-metrics.

Likely follow-ups

- What does concurrencyPolicy control?
- How do you alert on a job that silently stopped running?

```
kubectl get cronjob -n prod                      # LAST SCHEDULE, SUSPEND status
```

```
kubectl get jobs -n prod                         # did Jobs get created?
```

```
kubectl describe cronjob mycron -n prod          # events: missed schedules?
```

```
# Key fields:
```

```
#  concurrencyPolicy: Forbid|Allow|Replace   # overlap behaviour
```

```
#  startingDeadlineSeconds                    # missed-schedule tolerance
```

```
#  successfulJobsHistoryLimit / failedJobsHistoryLimit
```

CronJob issues fall into "not running" and "overlapping." For **not running**: check the CronJob is not `Suspend=true`, that its schedule (cron syntax, and the cluster's timezone assumptions) is correct, and the events for "missed schedule" — if the controller was down or busy past `startingDeadlineSeconds`, a run is skipped. For **overlapping**: the `concurrencyPolicy` governs behaviour — `Allow` (default) lets runs overlap (problematic for non-idempotent jobs), `Forbid` skips a new run if the previous is still going, `Replace` kills the old and starts fresh. Also check whether created Jobs themselves fail (image/permission/resource issues) versus the schedule never firing.

certs.eknathalabs.com

Interview tip: split the problem (not-firing vs overlapping) and map to fields: Suspend/schedule/startingDeadlineSeconds for missed runs; `concurrencyPolicy` (Forbid/Replace) for overlap of long or non-idempotent jobs. Distinguishing "schedule never created a Job" from "Job created but failed" is the diagnostic precision interviewers look for.

📚 **Official docs**

- Kubernetes: CronJob
- Azure Kubernetes Service (AKS)

Q83 How would you build a production-grade observability stack for AKS from scratch?

Monitoring

🎙 **Spoken answer · ~85 s**

I'd build it in layers around the signals and the people using them. Metrics: enable Azure Managed Prometheus for cluster and application metrics, with Managed Grafana for dashboards, plus Container Insights for node and container views. Logs: send container and control plane logs to a Log Analytics workspace using data collection rules, with filtering and retention controls, in structured JSON with correlation IDs. Traces: instrument services with OpenTelemetry, export through a collector to Application Insights, with sampling that keeps errors and slow traces. Link the signals by trace ID. On top, define SLIs and SLOs for key user journeys and alert on error budget burn rates, not raw resource thresholds. Add synthetic availability tests for critical paths and Azure Monitor alerts with action groups routing to on-call tooling. Provide standard dashboards: cluster health, node and pod resources, ingress golden signals, and per-service views, with deployment annotations. Cover platform health: node conditions, pending pods, restarts, autoscaler activity, and CoreDNS. Add security signals from Defender for Containers and audit logs. Manage everything as code with Bicep or Terraform and policy so every cluster gets the baseline automatically. Control cost from day one, and write runbooks linked from alerts. Review alert quality and gaps after incidents.

Likely follow-ups

- Which signals form the minimum viable observability stack?
- How do you tie metrics, logs, and traces together?

```
# Metrics:  Azure Managed Prometheus (scrape) + Managed Grafana (dashboards)
```

```
# Logs:     Container Insights -> Log Analytics (KQL), tiered retention
```

```
# Traces:   OpenTelemetry -> Application Insights (distributed tracing)
```

```
# Alerting: SLO burn-rate rules -> action groups (PagerDuty/email by severity)
```

```
# Correlate: trace_id across metrics/logs/traces; golden-signal dashboards
```

certs.eknathalabs.com

A complete stack covers the three pillars plus alerting and correlation. **Metrics**: Azure Managed Prometheus scrapes app and infra metrics (golden signals), visualised in Managed Grafana — zero-ops, PromQL-compatible. **Logs**: Container Insights ships to Log Analytics for KQL investigation, with tiered retention for cost. **Traces**: OpenTelemetry instrumentation exporting to Application Insights for end-to-end request tracing. **Alerting**: SLO burn-rate rules routed by severity to action groups (page vs ticket). The glue is **correlation** — shared `trace_id` linking a metric anomaly to the relevant logs and the exact slow trace — plus standardised golden-signal dashboards per service so any on-call engineer can navigate any service the same way.

Interview tip: present it as three pillars (metrics/logs/traces) + alerting + correlation, naming the managed Azure components (Managed Prometheus, Managed Grafana, Log Analytics, App Insights, OTel). The senior differentiators: alert on SLO burn (not thresholds), correlate via trace_id, and standardise golden-signal dashboards so the stack is usable under pressure — designing for the 3am incident, not just collecting data.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q84 How do you conduct a blameless post-incident review after an AKS production outage?

Troubleshooting

🎙 **Spoken answer · ~86 s**

A blameless review focuses on how the system and process allowed the failure, not on who made a mistake. Start soon after resolution, while memory is fresh, and assign a facilitator who wasn't the incident commander. Build a factual timeline from logs, alerts, chat, and deployment history: when the problem began, when it was detected, when responders engaged, what was tried, and when it was resolved. Capture impact in user and business terms: duration, affected customers, error budget consumed. Analyse causes using methods like five whys, looking for contributing factors across detection, design, process, and communication, not a single root cause. Ask why decisions made sense at the time given the information available. Examine what went well too: what worked in the response. Key questions: why wasn't it caught earlier, in testing or by alerts; why did it take as long as it did to detect and mitigate; what made diagnosis hard; and what controls failed. Produce action items that are specific, owned, prioritised, and dated, focusing on prevention, detection, and mitigation, with tracking to completion. Avoid vague items like be more careful. Share the written review widely, including the lessons, and review recurring themes across incidents. Psychological safety is essential, so leaders must model blamelessness.

Likely follow-ups

- Why avoid a single root cause conclusion?
- What makes an action item good?

```
# Structure:
```

certs.eknathalabs.com

```
#  1. Timeline: detection -> diagnosis -> mitigation -> resolution (with timestamps)
```

```
#  2. Impact: users affected, duration, SLO/error-budget burn
```

```
#  3. Root cause(s): the 'why' chain, not a person
```

```
#  4. What went well / what was hard (detection gaps, tooling gaps)
```

```
#  5. Action items: owned, dated, tracked to completion
```

A blameless post-incident review focuses on **systems and conditions**, not individual fault — because punishing people hides information, while psychological safety surfaces the real contributing factors. Build a precise **timeline** (detection → diagnosis → mitigation → resolution with timestamps), quantify **impact** (affected users, duration, error-budget burn), and trace **root cause(s)** as a chain of "why" (often multiple latent factors aligning, not a single cause). Capture what helped and what slowed response (detection gaps, missing runbooks, confusing dashboards), and turn findings into **owned, dated action items** tracked to completion. The output is organisational learning and concrete prevention, not blame.

Interview tip: emphasise *blameless* (systems not people — because blame suppresses the truth you need), a factual timeline, impact quantified against the error budget, multi-factor root cause (not "human error"), and trackable action items with owners. Mentioning that the error-budget burn from the incident informs release-pace decisions ties the postmortem into the broader SRE practice.

> 📚 **Official docs**
>
> - AKS: Best practices
> - Google SRE Book: Postmortem Culture
> - Google SRE Workbook: Postmortem Culture

certs.eknathalabs.com

Q85 How do you design high availability and disaster recovery for Azure SQL Database?

Databases

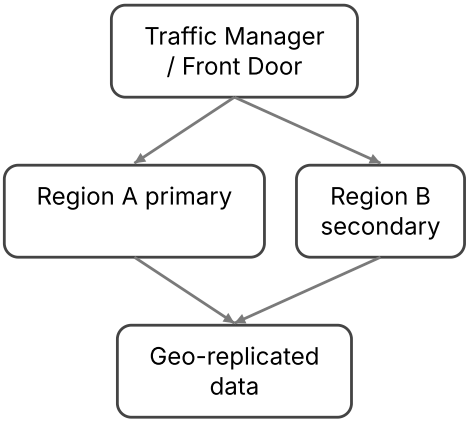
🎙 Spoken answer · ~88 s

Azure SQL Database offers high availability within a region and disaster recovery across regions. For HA: the service tiers include built-in replication. General Purpose uses remote storage with automatic failover to a spare compute node; Business Critical uses Always On-style local replicas on SSD with a readable secondary and fast failover; Hyperscale uses distributed storage and page servers, with replicas. Enable zone redundancy so replicas spread over availability zones, protecting against a datacentre failure with no data loss. The SLA reaches up to ninety-nine point nine nine five percent with zone redundancy. For DR: active geo-replication creates up to four readable asynchronous secondaries in other regions, and failover groups add a managed layer, with read-write and read-only listener endpoints that stay constant across failovers, so applications reconnect without configuration changes; policies can trigger automatic failover. Replication is asynchronous, so RPO is typically seconds, with RTO around a minute, and you must plan for potential small data loss. Backups provide another layer: automated full, differential, and log backups with point-in-time restore, long-term retention, and geo-redundant backup storage for regional restore. Design clients with retry logic for transient errors. Test failover regularly, monitor replication lag, and keep capacity in the secondary region. Define RTO and RPO per database to choose the right option.

Likely follow-ups

- How do failover group listener endpoints help applications?
- What is the difference between HA and DR here?

🗺 HA and DR




```
# In-region HA is built in (zone-redundant for Premium/Business Critical):
```

```
az sql db update -g rg -s srv -n mydb --zone-redundant true
```

```
# Cross-region DR options:
```

certs.eknathalabs.com

```
#  - Active geo-replication: readable secondaries in other regions
```

```
#  - Auto-failover groups: automatic failover + stable connection endpoint
```

```
az sql failover-group create -g rg -s primarysrv -n fg \\
```

```
  --partner-server secondarysrv --add-db mydb --failover-policy Automatic
```

Azure SQL provides layered resilience. **In-region HA** is automatic; higher tiers (Premium/Business Critical) add **zone redundancy**, surviving a datacenter (zone) failure transparently. For **regional DR**, **active geo-replication** creates readable secondary replicas in other regions (also useful for offloading reads), while **auto-failover groups** add automatic regional failover plus a stable listener endpoint — so the app connects to the failover-group name and Azure redirects to whichever region is primary, requiring no connection-string change on failover. Choose RPO/RTO targets, then match the tier and replication accordingly, and rehearse failover.

Interview tip: layer the answer — built-in HA → zone redundancy (DC failure) → geo-replication / auto-failover groups (region failure). The detail that lands: auto-failover groups give a *stable endpoint* so the app needs no change on failover, and you should define RPO/RTO and actually test failover. Tying tier choice to RPO/RTO shows you design DR to requirements, not by default.

> 📚 **Official docs**
>
> - AWS Disaster Recovery of workloads (whitepaper)
> - Azure SQL documentation

certs.eknathalabs.com

Q86 How do you perform a zero/low-downtime schema migration for a database backing an AKS app?

Databases

🎙 Spoken answer · ~88 s

The method is expand and contract, making every schema change backward compatible so that old and new application versions work against the same schema. Expand: add new columns as nullable or with defaults, new tables, and new indexes created online; do not rename or drop anything yet. Deploy the application version that writes to both old and new structures, or reads with fallback. Backfill existing data in small batches with throttling to avoid long locks and replication lag. Then migrate reads to the new structure, verify, and finally contract in a later release: remove the old columns and code once no running version uses them. Never combine a destructive change with the deployment that needs it. Practical safeguards: keep migrations versioned in source control, using tools like Flyway, Liquibase, or EF migrations; run them as a separate controlled pipeline step or init job before the rollout, not from every pod at startup, to avoid concurrent runs; test on production-like data to measure lock duration; use online index operations and avoid operations that take exclusive locks on large tables; set lock timeouts; and have a rollback plan, usually rolling the application back while the schema remains compatible. Take a backup or snapshot before risky changes. Monitor database performance and replication during migration.

Likely follow-ups

- Why must a destructive change never ship in the same release as the code that needs it?
- Why run migrations as a separate pipeline step?

```
# Expand/contract (parallel change) across multiple deploys:
#  1. EXPAND: add new column/table (nullable/default) — backward compatible
#  2. deploy app that writes BOTH old+new, reads old (or new)
#  3. BACKFILL existing rows in batches
#  4. switch app to read/write new only
#  5. CONTRACT: drop old column — only after no code uses it
```

Low-downtime schema changes use the **expand/contract** (parallel change) pattern, never a breaking change in one step. **Expand**: add the new structure backward-compatibly (nullable column, new table) so old code still works. Deploy code that writes to both old and new shapes. **Backfill** existing data in batches (avoiding long locks). Switch the app to use the new shape. Finally **contract**: remove the old structure — only once no running version references it. Each step is independently deployable and rollbackable, which is exactly what keeps a rolling AKS deploy (old and new pods coexisting) from breaking against the schema.

certs.eknathalabs.com

Interview tip: the key is decoupling schema changes from code changes across multiple deploys so old and new app versions both work during a rolling update. The trap to name: dropping a column in the same release that stops using it breaks rollback (old pods expect the column). Mentioning batched backfills to avoid lock contention shows production DB experience.

📚 **Official docs**

- gh-ost: GitHub's online schema migration
- Azure Kubernetes Service (AKS)
- Flyway documentation

Q87 When do you choose Cosmos DB vs Azure SQL vs PostgreSQL Flexible Server?

Databases

🎙 **Spoken answer · ~87 s**

The choice follows data model, access patterns, scale, and consistency needs. Azure SQL Database suits relational data with transactions, joins, constraints, and rich querying, using the SQL Server ecosystem, particularly for Microsoft-centred shops or migrations from SQL Server. PostgreSQL Flexible Server suits relational workloads where you want open-source PostgreSQL, its extensions like PostGIS and pgvector, portability, and lower licence concerns, with zone-redundant HA and read replicas. Both scale vertically and via read replicas, with horizontal write scaling being complex. Cosmos DB suits globally distributed, high-throughput, low-latency applications with flexible JSON schemas, known access patterns, and the need for multi-region writes or elastic scale: IoT, catalogues, personalisation, session state, and event stores. Trade-offs: Cosmos DB demands careful partition key design, charges by request units, has limited ad hoc joins and queries across partitions, and weaker relational integrity. Relational databases give flexible querying and strong transactional semantics for business data. Decision cues: complex relationships and reporting, relational; a massive scale of simple access patterns or global distribution, Cosmos DB. Consider team skills, ecosystem, cost predictability, compliance, and migration paths. Many systems use several, with a relational system of record and Cosmos DB for specific high-scale features. Start with the simplest option that meets requirements; relational databases are enough for most.

Likely follow-ups

- What makes partition key design critical in Cosmos DB?
- When is PostgreSQL Flexible Server preferable to Azure SQL?

```
Cosmos DB:        global distribution, NoSQL, horizontal scale, tunable consistency
```

```
Azure SQL:        relational, strong consistency, T-SQL, transactions, PaaS
```

```
PostgreSQL Flex:  open-source relational, extensions, in-VNet, cost-effective
```

```
# Decision drivers: data model, consistency, scale shape, global reach, portability
```

cons.eknathalabs.com

Match the store to the workload. **Cosmos DB**: globally distributed, horizontally scalable NoSQL with tunable consistency and single-digit-ms latency worldwide — for global apps, high write throughput, flexible schemas, where partitioning is designed well. **Azure SQL**: managed relational with strong consistency, rich T-SQL, and transactions — for relational workloads needing ACID and complex queries, especially Microsoft-stack apps. **PostgreSQL Flexible Server**: managed open-source Postgres with extensions (PostGIS, etc.), VNet integration, and cost efficiency — for relational workloads valuing portability/open-source and no cloud lock-in. The drivers are data model (relational vs document), consistency needs, scale shape (vertical vs global horizontal), and portability.

Interview tip: decide by data model + consistency + scale + portability, not popularity. "Global, high-throughput, flexible schema, partition-able" → Cosmos; "relational/ACID, Microsoft stack" → Azure SQL; "open-source relational, portable, cost-sensitive" → PostgreSQL Flexible Server. Flagging that Cosmos's partition-key design makes or breaks it is the senior caveat.

📚 **Official docs**

- Azure Cosmos DB
- Azure SQL documentation

certs.eknathalabs.com

Q88 How do you secure Azure Storage accounts for production?

Storage

🎙 **Spoken answer · ~84 s**

Securing storage accounts layers network, identity, data protection, and monitoring. Network: disable public network access or restrict it with firewall rules, and use private endpoints so traffic stays on the private network; set the minimum TLS version and require secure transfer. Identity: prefer Entra ID authentication with Azure RBAC over account keys and shared access signatures; disable shared key access where possible; use managed identities for applications; if SAS tokens are needed, use user delegation SAS, short expiry, narrow permissions, and stored access policies. Disable anonymous blob access at the account level. Encryption: data is encrypted at rest by default with Microsoft-managed keys; use customer-managed keys in Key Vault if required, and consider infrastructure encryption. Data protection: enable soft delete for blobs and containers, versioning, point-in-time restore, and immutability policies with legal holds or time-based retention for compliance; use resource locks to prevent account deletion. Redundancy choice supports resilience. Monitoring: enable diagnostic logs to Log Analytics, alert on unusual access or deletions, and enable Defender for Storage, which detects malware uploads and anomalous access. Governance: Azure Policy to enforce these settings across subscriptions, and regular access reviews. Rotate keys if keys remain in use. Together these controls reduce exposure and enable recovery.

Likely follow-ups

- Why disable shared key access?
- What does a user delegation SAS improve?

```
# Disable public access; require private connectivity:
```

```
az storage account update -g rg -n acct --public-network-access Disabled
```

```
az network private-endpoint create ... --group-id blob   # private endpoint + DNS
```

```
# Enforce: HTTPS only, min TLS 1.2, no shared-key (use Entra/RBAC):
```

```
az storage account update -g rg -n acct --https-only true --min-tls-version TLS1_2 \\
```

```
  --allow-shared-key-access false
```

```
# Access via Entra RBAC (Managed Identity) + SAS with short expiry where needed
```

Production storage hardening: **disable public network access** and reach the account via a **Private Endpoint** (with Private DNS) so traffic stays on the Azure backbone; **enforce HTTPS and TLS 1.2+**; and **disable shared-key access** in favour of **Entra ID RBAC** with Managed Identities — eliminating long-lived account keys that leak and never rotate. Where temporary delegated access is needed (e.g. uploads), use short-expiry, narrowly-scoped SAS tokens (ideally user-delegation SAS tied to Entra). Enable soft-delete/versioning for recovery and immutability/legal-hold where compliance requires. Encryption at rest is on by default (optionally with customer-managed keys).

Interview tip: the high-impact moves are disabling public access (→ Private Endpoint) and disabling shared-key access (→ Entra RBAC/Managed Identity), because account keys are the classic leak. Add TLS enforcement, short-lived user-delegation SAS, and soft-delete/versioning. Leading with "kill the account keys, use identity" signals modern Azure security thinking.

> 📚 **Official docs**
>
> - Azure Storage introduction

certs.eknathalabs.com

Q89 How do Azure Storage redundancy options (LRS/ZRS/GRS/GZRS) differ and how do you choose?

Storage

🎙 Spoken answer · ~83 s

Storage redundancy options define how many copies exist and where. Locally redundant storage keeps three copies within a single datacentre; it protects against drive and rack failures, is the cheapest, but not against a datacentre disaster. Zone-redundant storage keeps three synchronous copies across availability zones in one region, protecting against a zone outage with no data loss, and is the usual baseline for production. Geo-redundant storage keeps three copies locally in the primary region and asynchronously replicates to three in a paired secondary region, protecting against a regional disaster; data in the secondary isn't readable unless you use read-access GRS, and failover takes a manual or Microsoft-managed action with potential data loss because replication is asynchronous, typically with an RPO under fifteen minutes. Geo-zone-redundant storage combines both: ZRS in the primary and LRS in the secondary region, giving the highest durability and availability. Read-access variants allow reading from the secondary. Choose by requirements: the business impact of data loss and downtime, compliance needs, cost, and whether the application can use a secondary endpoint. Typical guidance: LRS for dev, test, or easily reproducible data; ZRS for production within a region; GZRS or RA-GZRS for critical data needing regional disaster protection.

Likely follow-ups

- Why isn't redundancy a substitute for backup?
- What does RA-GRS add?

```
LRS:  3 copies in ONE datacenter (cheapest; DC failure = data loss)
```

```
ZRS:  3 copies across availability ZONES in a region (survives a DC/zone loss)
```

```
GRS:  LRS + async copy to PAIRED REGION (regional DR; secondary not readable)
```

```
GZRS: ZRS + async copy to paired region (zone + region resilience)
```

```
RA-GRS / RA-GZRS: adds READ access to the secondary region
```

Redundancy tiers trade durability/availability against cost. **LRS** keeps three copies in one datacenter — cheapest, but a datacenter loss loses data. **ZRS** spreads copies across availability zones in the region — survives a zone/datacenter failure. **GRS** adds asynchronous replication to the **paired region** for regional disaster recovery (secondary is not readable unless you fail over). **GZRS** combines zone redundancy with cross-region replication — the most resilient. The **RA-** variants make the secondary region readable, useful for read scale or regional read-availability. Choose by required resilience: ZRS for in-region HA, G(Z)RS for regional DR, RA- when you need read access to the secondary.

Interview tip: map each to the failure it survives — LRS (none beyond disk), ZRS (zone/DC), GRS (region, no read), GZRS (zone+region), RA-* (adds secondary read). For production data you typically want at least ZRS (in-region resilience) and G(Z)RS for DR. Noting that GRS secondary is *not readable* until failover (unless RA-)

certs.eknathalabs.com

is the detail people miss.

📚 **Official docs**

- Azure Storage introduction
- Azure Storage redundancy

Q90 How do Azure Reservations, Savings Plans, and Spot pricing differ, and when do you use each? Cost

🎙 **Spoken answer · ~89 s**

All three reduce compute cost with different trade-offs. Reservations commit to a specific VM family, size, and region for one or three years in exchange for a discount of up to roughly sixty to seventy percent; they offer high savings but less flexibility, though instance size flexibility and exchanges help. They apply also to other resources such as databases, storage, and Cosmos DB capacity. Savings Plans commit to an hourly spend amount for one or three years, applying automatically across VM families, regions, and some services like App Service and Container Instances; discounts are somewhat lower than reservations, but flexibility is much greater, which suits changing workloads. Spot VMs use unused capacity at discounts up to about ninety percent, with eviction at short notice and no guarantee, suited to interruptible, fault-tolerant workloads. How to choose: analyse steady baseline usage with Azure Advisor and cost analysis; cover the predictable floor with reservations for stable, known configurations, or Savings Plans where workloads shift; use spot for batch, CI, stateless workers, and burst capacity, with fallback to regular capacity; keep pay-as-you-go for the variable remainder. Commit conservatively, around the minimum usage you're confident about, and ladder purchases to avoid renewal cliffs. Review utilisation, and consider planned architectural changes before committing. Combine with Hybrid Benefit for licence savings.

Likely follow-ups

- How much of your usage should you commit to?
- What is Azure Hybrid Benefit?

certs.eknathalabs.com

```
Reservations:  1/3-yr commit to SPECIFIC VM size/region -> up to ~72% off baseline
```

```
Savings Plans: 1/3-yr commit to hourly COMPUTE SPEND (flexible across SKUs/regions)
```

```
Spot:          spare capacity, ~60-90% off, evictable with short notice
```

```
PAYG:          on-demand, full price, no commitment
```

These are the core cost levers. **Reservations** commit to a specific VM family/region for 1–3 years — deepest discount (up to ~72%) but least flexible (locked to that SKU/region). **Savings Plans** commit to an hourly compute *spend* rather than a specific SKU — slightly less discount but flexible across instance sizes and regions, ideal when your baseline is steady but the exact VM mix shifts. **Spot** uses spare capacity at 60–90%

off but can be evicted on short notice — for fault-tolerant, interruptible workloads (batch, CI, ML training). The strategy: cover **predictable baseline** with reservations/savings plans, run **tolerant/bursty** workloads on spot, and leave only true variability on pay-as-you-go.

Interview tip: the decision frame is commitment vs flexibility vs interruptibility. Reservations (max discount, rigid SKU) for stable known shapes; Savings Plans (flexible spend commit) for steady-but-shifting compute; Spot for interruptible workloads. The portfolio answer — reservations/savings for baseline + spot for tolerant + PAYG for spikes — is what a FinOps-aware platform engineer says.

📚 **Official docs**

- AWS Savings Plans
- Azure Reservations

Q91 How do you handle read scaling and caching for a database-backed service on Azure?

Databases

🎙 **Spoken answer · ~83 s**

Read scaling and caching tackle the problem at different levels. First fix the basics: indexes, query tuning, and avoiding N+1 patterns. Then scale reads. Read replicas: Azure SQL Business Critical and Hyperscale offer readable secondaries; PostgreSQL Flexible Server supports read replicas; Cosmos DB scales reads through RUs and multi-region replication. Route read-only traffic to replicas with connection settings, such as ApplicationIntent equals ReadOnly for SQL, accepting replication lag and handling read-your-writes cases by reading from the primary right after a write. Caching: put Azure Cache for Redis in front of the database, using cache-aside: check the cache, load on miss, populate with a TTL. Cache expensive, frequently read, rarely changing data. Prevent stampedes with jittered TTLs and request coalescing, handle invalidation on writes, and make sure the application degrades gracefully if the cache is unavailable. Use HTTP caching and a CDN or Front Door for cacheable API responses. Consider materialised views or precomputed read models for heavy aggregations. Scale vertically when needed, and use connection pooling. Measure cache hit ratio, database load, replication lag, and latency to validate the gains. Combine layers: CDN, application cache, Redis, replicas, then primary, with sensible TTLs and clear consistency expectations per data type.

Likely follow-ups

- How do you handle read-your-writes with replicas?
- What does cache-aside look like in practice?

```
# Read replicas (offload reads):
```

```
#  Azure SQL: active geo-replication / named replicas (readable secondaries)
```

```
#  PostgreSQL Flexible Server: read replicas
```

certs.eknathalabs.com

```
# Cache hot data (Azure Cache for Redis):
```

```
az redis create -g rg -n mycache --sku Premium --vm-size P1
```

```
# Patterns: cache-aside, write-through; set TTLs; guard against stampede
```

Scale reads in two complementary ways. **Read replicas** offload read traffic from the primary: Azure SQL named replicas / geo-replication and PostgreSQL Flexible Server read replicas let read-heavy workloads (reporting, dashboards) hit secondaries while the primary handles writes — mind the small replication lag (eventual consistency on replicas). **Caching** with **Azure Cache for Redis** serves hot data from memory, dramatically cutting database load and latency. Common patterns: **cache-aside** (app checks cache, falls back to DB, populates cache) and write-through; always set sensible TTLs and protect against cache stampedes (e.g. request coalescing, jittered expiry) so a cold cache does not hammer the DB.

Interview tip: two levers — read replicas (offload reads, accept replication lag / eventual consistency on the replica) and Redis caching (cache-aside pattern, TTLs, stampede protection). Naming the replication-lag trade-off and cache-stampede mitigation shows you have run these at scale, not just enabled them.

> 📚 **Official docs**
>
> - Redis documentation

certs.eknathalabs.com

Q92 How do you implement lifecycle management and tiering for cost-efficient blob storage?

Storage

🎙 Spoken answer · ~88 s

Blob lifecycle management uses policies to automatically move or delete data based on age or access, reducing cost. Tiers: hot for frequent access with the highest storage cost and lowest access cost; cool for infrequent access, with a thirty-day minimum; cold for rarer access, with a ninety-day minimum; and archive, the cheapest storage but offline, requiring rehydration taking hours, with a one-hundred-and-eighty-day minimum. A lifecycle rule in the storage account defines filters by prefix, blob type, or index tags, and actions based on days since modification, creation, or last access: move to cool after thirty days, to cold or archive after ninety, delete after a year. It also handles snapshots and versions, expiring old ones, which prevents version sprawl. Last-access-time tracking, if enabled, enables policies based on actual use. Design points: know the access pattern before tiering, since early deletion fees and higher read and retrieval costs can negate savings if data is read often; avoid archiving data you may need quickly; small blobs may not benefit; transaction costs for moving many objects matter. Test with a subset, use Storage analytics and cost analysis to measure, and apply different rules per container for logs, backups, and media. Use immutability and legal holds where compliance requires, which override deletion. Review the policies regularly.

Likely follow-ups

- What penalties apply when data leaves a cool or archive tier early?
- How long does rehydration from archive take?

```
# Tiers: Hot (frequent), Cool (30+ days), Cold (90+), Archive (rare, hours to rehydrate)
```

```
# Lifecycle policy moves/expires blobs automatically by age:
```

```
#   if baseBlob > 30 days -> tierToCool
```

```
#   if baseBlob > 90 days -> tierToArchive
```

```
#   if baseBlob > 365 days -> delete
```

```
az storage account management-policy create --account-name acct --policy @policy.json
```

Blob storage has access tiers priced for different access patterns: **Hot** (frequent access, higher storage cost, lower access cost), **Cool** and **Cold** (infrequent, lower storage cost, higher access/retrieval cost, minimum retention periods), and **Archive** (rarely accessed, cheapest storage, but data must be *rehydrated* over hours before reading). **Lifecycle management policies** automate this: rules move blobs to cooler tiers as they age and delete them past a retention threshold — e.g. logs go Hot → Cool at 30 days → Archive at 90 → deleted at a year. This matches storage cost to actual access patterns automatically, a major lever for data-heavy workloads.

certs.eknathalabs.com

Interview tip: describe tiers by access pattern + the trade-off (cooler = cheaper storage but pricier/slower retrieval, Archive needs rehydration), then automate with lifecycle policies (age-based tier transitions + expiry). The Archive rehydration latency (hours) is the gotcha to mention — it disqualifies Archive for anything needing quick reads.

📚 **Start here (official docs)**

- Microsoft Azure documentation

Q93 How do you back up and restore data, and test DR, for production databases on Azure?

Databases

🎙 **Spoken answer · ~84 s**

Backup and DR start with RTO and RPO per system. Azure SQL provides automated backups: full weekly, differential, and log backups every few minutes, enabling point-in-time restore within the retention window, plus long-term retention for years, and geo-restore from geo-redundant backups. PostgreSQL Flexible Server has automated backups with point-in-time restore and optional geo-redundancy. Cosmos DB offers continuous backup with point-in-time restore or periodic backup. Storage uses soft delete, versioning, and snapshots. Azure Backup centralises policy-based protection for VMs, disks, and some databases, in vaults with soft delete, immutability, and multi-user authorisation for protection against ransomware or malicious deletion. For DR, use geo-replication or failover groups to a secondary region, and replicate infrastructure definitions, secrets, and images. The essential part is testing. Regularly restore backups into an isolated environment and validate data integrity and application function, measuring actual restore time against RTO. Rehearse failover with game days: fail over the database or whole application, verify connectivity, and fail back, documenting steps and timing. Automate runbooks with scripts and infrastructure as code. Protect backups: separate subscription or vault access, immutability, and restricted deletion rights. Monitor backup jobs and alert on failures, track restore test results as a metric, and review after each drill.

Likely follow-ups

- Why must restores be tested on a schedule?
- How do you protect backups from ransomware?

```
# Azure SQL: automated backups + Point-In-Time Restore (PITR) within retention
```

```
az sql db restore -g rg -s srv -n mydb --dest-name mydb-restored --time "2026-06-01T10:00:00Z"
```

```
# Long-term retention (LTR) for compliance (weekly/monthly/yearly)
```

```
# Cosmos DB: continuous backup + PITR; PostgreSQL: automated backups + PITR
```

```
# DR test: restore to a copy, validate, document RTO/RPO actually achieved
```

certs.eknathalabs.com

Managed Azure databases provide **automated backups** with **point-in-time restore** (PITR) within a configurable retention window — you can restore to any moment (e.g. just before a bad deploy or accidental delete). For compliance, **long-term retention** keeps weekly/monthly/yearly backups for years. Cosmos DB and PostgreSQL Flexible Server offer equivalent continuous-backup/PITR. The crucial discipline is **testing restores**: regularly restore to a copy, validate the data, and measure the *actual* RTO/RPO achieved against your targets — an untested backup is not a backup. Document the runbook so recovery under pressure is repeatable.

Interview tip: automated backups + PITR + LTR is the baseline; the senior point is "test the restore and measure real RTO/RPO" — untested backups fail when you need them. Mention restoring to a copy (never over the prod DB) for validation, and tying retention/LTR to compliance requirements. The "untested backup isn't a backup" line resonates.

📚 **Official docs**

- PostgreSQL: Continuous archiving and PITR

Q94 A team's Azure bill spiked unexpectedly. Walk through investigating and controlling it.

Cost

🎙 **Spoken answer · ~89 s**

A bill spike calls for speed and method. Open Cost Management cost analysis, view daily costs, and group by service, resource group, resource, location, and tag to find what changed and when. Compare against previous periods. Typical causes: autoscaling that ran away or a node pool left at a high maximum; new or oversized resources, such as GPU nodes; a forgotten test environment; increased data egress or inter-region traffic; Log Analytics ingestion spikes from verbose logging or a crash loop emitting logs; storage growth or snapshots; database tier changes; premium services enabled; or, in the worst case, compromised credentials running mining workloads. Cost Management's anomaly detection can flag unusual patterns. Use the Activity Log to see who created or changed resources around the spike date, and Kubernetes tools like cost analysis or Kubecost to see which namespace or workload on AKS drove it. Contain: scale down or delete the offending resources, fix the loop, and if security is suspected, rotate credentials and investigate. Then control: set budgets with alerts on actual and forecast spend, apply Azure Policy to restrict SKUs and regions, cap autoscaler maximums, use quotas, and configure log ingestion limits via daily caps on workspaces. Assign cost owners with tags, review weekly, and hold a short review documenting cause and prevention.

Likely follow-ups

- How do you find who created a resource that caused a spike?
- What controls cap runaway log ingestion?

```
# 1. Cost Management: cost analysis grouped by service / resource group / tag
```

```
#    -> what jumped, when, and which resource/team owns it
```

certs.eknathalabs.com

```
# 2. Cost anomaly alerts should have flagged it; check the spike onset
```

```
# 3. Common causes: untagged orphans, over-provisioned SKUs, egress/data transfer,
```

```
#    forgotten dev resources, log ingestion blowout, autoscale runaway
```

```
# 4. Controls: budgets+alerts, policy to require tags, rightsizing, reservations
```

Investigate top-down in **Azure Cost Management**: open cost analysis and group by service, resource group, and tag to find *what* jumped, *when*, and *who owns it*. Correlate the onset with deployments or config changes. Common culprits: over-provisioned or accidentally-large SKUs, **orphaned/untagged resources** nobody owns, **data egress / cross-region transfer**, forgotten dev/test resources left running, **log ingestion** blowout (a chatty app flooding Log Analytics), or runaway autoscaling. Then control it: set **budgets with alerts** and **anomaly detection** so the next spike is caught early, enforce **required tags** via Azure Policy for accountability, rightsize, and apply reservations/savings plans to the stable baseline.

Interview tip: method first — Cost Management grouped by service/RG/tag to localise the spike, then map to common causes (orphans, oversized SKUs, egress, log ingestion, autoscale). The preventive close (budgets + anomaly alerts + policy-enforced tagging for ownership) shows you turn a fire-drill into a standing control. Mentioning log-ingestion blowout as a cause is a nice real-world detail.

> 📚 **Start here (official docs)**
>
> - Microsoft Azure documentation

certs.eknathalabs.com

Q95 How do persistent volumes work on AKS, and how do you handle volume issues across zones?

Storage

🎙 Spoken answer · ~88 s

On AKS, a PersistentVolumeClaim requests storage, a StorageClass describes how to provision it, and the CSI driver creates the underlying Azure resource, then binds it as a PersistentVolume. Azure Disk volumes are block storage, accessible by one node at a time, and zonal: a disk lives in a single availability zone. This causes the classic problem: if a pod with a zonal disk is rescheduled to a node in another zone, it can't attach the disk and stays pending with a volume node affinity conflict. Mitigations: use a StorageClass with volumeBindingMode WaitForFirstConsumer, so the disk is created in the zone where the pod is first scheduled; use zone-redundant disks, ZRS, which can attach across zones within the region; ensure node pools exist in every zone where volumes may live, because if the zone's node pool is scaled to zero or full, the pod can't run; use topology spread and per-zone StatefulSet replicas so each zone has its own volume and replica; and replicate data at the application layer. Azure Files provides shared storage accessible across zones with ReadWriteMany, with different performance characteristics. Other issues: attach and detach delays on node failure, disk size limits, resize with volume expansion, and snapshots for backup via VolumeSnapshot. Monitor volume usage and set alerts.

Likely follow-ups

- Why does WaitForFirstConsumer matter for zonal disks?
- How do zone-redundant disks change the problem?

```
# CSI drivers provision PVs: Azure Disk (RWO) vs Azure Files (RWX)
```

```
kubectl get storageclass   # managed-csi (disk), azurefile-csi (files)
```

```
# Azure Disk is ZONE-PINNED: a pod using it must schedule in the disk's zone
```

```
#   -> use zone-redundant storage class or topology-aware volume binding
```

```
# volumeBindingMode: WaitForFirstConsumer  (bind disk in the pod's zone)
```

AKS provisions persistent volumes through **CSI drivers**: **Azure Disk** (block, `ReadWriteOnce` — one pod) for databases/stateful pods, and **Azure Files** (SMB/NFS, `ReadWriteMany` — shared across pods) for shared content. The key cross-zone gotcha: a standard **Azure Disk is pinned to a single zone**, so a pod mounting it can only schedule in that zone — if that zone is unavailable or the scheduler wants another zone, the pod gets stuck Pending. Fixes: use **WaitForFirstConsumer** volume binding (provision the disk in the zone where the pod actually lands) and/or zone-redundant storage classes, and for cross-zone shared access prefer Azure Files (RWX). For HA databases, replicate at the data layer rather than relying on a single zonal disk.

Interview tip: distinguish Azure Disk (RWO, **zone-pinned**) from Azure Files (RWX, shareable), then nail the classic failure: a pod stuck Pending because its zonal disk's zone does not match an available node. The fix — `WaitForFirstConsumer` binding and zone-aware/ zone-redundant storage — is the AKS-specific detail that

certs.eknathalabs.com

shows real stateful-workload experience.

> 📚 **Official docs**
> - Azure Kubernetes Service (AKS)

Q96 How do you choose a Cosmos DB partition key and avoid hot partitions?

Databases

🎙 **Spoken answer · ~85 s**

In Cosmos DB, the partition key determines how data and throughput are distributed across physical partitions, so it's the most consequential design decision. A good key has high cardinality, many distinct values; spreads both storage and request load evenly; and appears in most queries as a filter, so queries hit a single partition instead of fanning out across all, which costs more RUs and latency. Each logical partition is limited to twenty gigabytes of data and bounded throughput, so a key with few values or one dominant value creates a hot partition: throttled requests, even though overall provisioned throughput is unused. Examples: userId or tenantId for multi-tenant data usually works; a timestamp or status field usually doesn't, because current writes concentrate on one value. For skewed tenants, use a synthetic key combining fields, such as tenantId plus a suffix, or hierarchical partition keys, which allow up to three levels of keys and avoid the twenty-gigabyte limit problem for large tenants. Design from access patterns: list the queries, choose the key that serves the most frequent and expensive ones, and denormalise data to avoid cross-partition operations. Monitor with the partition key metrics and diagnostic logs to see hot partitions and normalised RU consumption per partition.

Likely follow-ups

- What are hierarchical partition keys?
- Why can a hot partition exist when overall RUs are unused?

```
# Good partition key: HIGH cardinality + EVEN access distribution
#  - good: /userId, /tenantId (if balanced), /deviceId
#  - bad:  /status (few values), /date (today gets all writes = hot)
# Symptoms of a bad key: 429 throttling on some partitions, uneven RU usage
# Composite/synthetic keys to spread load when natural key is skewed
```

Cosmos DB partitions data by the **partition key**, and each logical partition has a throughput (RU) and storage ceiling. A good key has **high cardinality** (many distinct values) and **even access distribution** so reads/writes spread across partitions — `/userId` or `/tenantId` (if balanced) work well. A poor key concentrates load: low-cardinality values (`/status`) or time-based keys (`/date`, where today's partition takes all writes) create **hot**

certs.eknathalabs.com

partitions that throttle (HTTP 429) while others sit idle. When the natural key is skewed, use a **synthetic/composite key** (e.g. combine tenant + a hash suffix) to spread load. Partition key is effectively immutable, so getting it right up front is critical.

Interview tip: the rule is high cardinality + even distribution; the anti-patterns are low-cardinality and time-based keys (today = hot partition). Symptoms: per-partition 429 throttling with uneven RU consumption. Mentioning synthetic/composite keys for skewed data and that the partition key is essentially immutable (design it correctly the first time) is the depth interviewers want.

📚 **Official docs**

- Azure Cosmos DB
- Microsoft: Sharding pattern
- Apache Kafka documentation

Q97 How do you allocate and chargeback shared AKS cluster costs to teams? Cost

🎙 **Spoken answer · ~86 s**

Chargeback for a shared AKS cluster means splitting the shared cost fairly among teams using measurable consumption. Cost components: node VM costs, which are the bulk; storage; networking; control plane tier; monitoring; and shared overhead like system components. Method: label every workload by team, application, and environment, and use namespaces per team. Use the AKS cost analysis add-on, or OpenCost or Kubecost, which allocate node cost to workloads in proportion to their resource requests, or the greater of requests and usage, for CPU and memory, plus storage and network, and show idle cost, capacity that nobody is using. Decide how to allocate idle and shared costs: distribute proportionally to usage, evenly, or keep it as platform overhead. Separate node pools per team is another approach, giving exact attribution at the price of lower bin-packing efficiency. Export the cost data into Azure Cost Management or a reporting dataset, and publish showback dashboards, then chargeback through internal billing if the organisation wants it. Enforce accurate requests, because allocation depends on them: apply LimitRanges and ResourceQuotas, so teams can't over-reserve without paying. Review monthly with teams, highlight waste, and give recommendations. Be transparent about the allocation rules to avoid disputes, and start with showback to build trust before real chargeback.

Likely follow-ups

- How do you allocate idle cluster cost fairly?
- Why does allocation depend on accurate resource requests?

```
# Shared cluster cost is opaque without allocation tooling:
```

```
#  - OpenCost / Kubecost: per-namespace/pod/label cost (compute + storage + network)
```

```
#  - map namespaces/labels -> teams; report showback or chargeback
```

certs.eknathlabs.com

```
# Azure side: resource tags + Cost Management for non-K8s resources
```

```
# Governance: require team/cost-center labels via Gatekeeper/Azure Policy
```

A shared AKS cluster bills as a few VMs, hiding which team consumes what. **Cost-allocation tooling** (OpenCost/Kubecost) attributes the node cost down to namespaces, workloads, and labels by measuring each pod's resource requests/usage — producing per-team cost for compute, storage, and (with config) network. Map namespaces or team labels to owning teams, then publish either **showback** (visibility to drive behaviour) or **chargeback** (actual cross-charging). Enforce the required **team/cost-center labels** at admission (Gatekeeper/Azure Policy) so untagged workloads cannot escape allocation. Pair with Azure tags + Cost Management for the non-Kubernetes resources (databases, storage, networking).

Interview tip: the core insight is that Kubernetes hides per-team cost until you add allocation (Kubecost/OpenCost mapping namespaces/labels → teams). Distinguish showback (visibility) from chargeback (cross-charging), and stress *enforcing* cost labels at admission so allocation is complete. This is a concrete FinOps-on-Kubernetes answer that platform interviewers value.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q98 How do you handle data residency, encryption, and compliance for Azure data services?

Storage

🎙 Spoken answer · ~89 s

Data residency means data stays in specified regions; I enforce it through design and governance. Region choice: deploy services only in approved regions, and use Azure Policy with an allowed locations policy to block anything else. Understand which services replicate data: geo-redundant storage and paired-region backup copies move data to the paired region, so select zone-redundant options within one region, or ensure the pair is also compliant; some global services store metadata elsewhere, so check documentation and data residency commitments. Encryption: data is encrypted at rest by default, and in transit with TLS; for tighter control use customer-managed keys in Key Vault or Managed HSM, with key rotation, access control, and optionally bring your own key; consider confidential computing for data in use. Compliance: use Defender for Cloud regulatory compliance dashboards mapped to standards such as ISO 27001, SOC 2, HIPAA, or GDPR, Azure Policy initiatives to enforce controls, and Microsoft Purview for data classification, lineage, and governance. Access: least privilege, private endpoints, disabled public access, auditing through diagnostic logs, and data loss prevention. Sovereign options include Azure sovereign clouds and customer lockbox for control over Microsoft support access. Document data flows and processing locations, run regular audits, and keep evidence. Involve legal and compliance teams early because requirements vary by regulation and country.

Likely follow-ups

- Why can geo-redundant storage violate residency requirements?
- What do customer-managed keys add?

```
# Residency: pin resources to compliant regions; Azure Policy 'allowed locations'
```

```
# Encryption at rest: on by default; customer-managed keys (CMK) in Key Vault for control
```

```
# Encryption in transit: enforce TLS 1.2+; private endpoints to avoid public paths
```

```
# Compliance: Defender for Cloud regulatory dashboards (PCI/HIPAA/ISO/SOC2)
```

Compliance for data services spans residency, encryption, and evidence. **Data residency**: keep data in approved regions — enforce with an Azure Policy "allowed locations" so no resource is created outside compliant geographies. **Encryption at rest** is on by default with Microsoft-managed keys; for regulatory control over the key lifecycle, use **customer-managed keys** (CMK) stored in Key Vault (you can rotate/revoke, controlling access to your own data). **Encryption in transit**: enforce TLS 1.2+ and use Private Endpoints so data never traverses public networks. For **evidence**, Defender for Cloud's regulatory compliance dashboards continuously assess your estate against PCI/HIPAA/ISO/SOC2 and produce audit-ready reports.

certs.eknathalabs.com

Interview tip: cover residency (policy-enforced allowed regions), encryption at rest (default + CMK when you must control keys), in transit (TLS + private endpoints), and continuous compliance evidence (Defender regulatory dashboards). The CMK point — customer-managed keys give you rotation/revocation control over your data's encryption — is the nuance that signals real compliance work.

📚 **Official docs**

- AWS KMS
- NIST Cybersecurity Framework

Q99 How do you handle database connection management and resilience from AKS pods?

Databases

🎙 **Spoken answer · ~89 s**

Database connections from AKS pods need deliberate management because pods scale and restart frequently. Use connection pooling in each application, with sensible maximum sizes, idle timeouts, and connection lifetimes, and calculate total connections as pods multiplied by pool size, ensuring it stays under the database limit even at maximum autoscaling. Where the database can't handle many connections, add a pooler like PgBouncer for PostgreSQL. Authenticate without stored passwords: use Workload Identity with Entra authentication to Azure SQL and PostgreSQL, so pods obtain tokens, and handle token refresh. Resilience: expect transient faults, such as failovers, throttling, and brief network interruptions; implement retry with exponential backoff and jitter, using built-in retry policies in drivers or libraries like Polly, with limits to avoid retry storms; set connect and command timeouts; use circuit breakers so a failing database doesn't exhaust threads. Handle failover: connect via the failover group listener or server DNS names with short DNS caching, so reconnection finds the new primary. Validate connections on borrow, and recycle them periodically to pick up DNS changes. Release connections promptly, avoid long transactions. Graceful shutdown should close pools cleanly. Use read-only intent for read traffic to replicas. Monitor pool saturation, wait times, database sessions, and failures. Load test failover scenarios to confirm the application recovers without manual intervention.

Likely follow-ups

- How do you calculate the maximum total database connections?
- Why add jitter to retries?

```
# Use Managed Identity / Workload Identity (no DB passwords in pods)
```

```
# Connection pooling: size pool to (replicas x pool) < DB max connections
```

```
# Resilience: retry with exponential backoff + jitter on transient errors
```

```
# Handle failover: reconnect logic; for Azure SQL use failover-group listener
```

certs.eknathalabs.com

```
# PgBouncer sidecar/Deployment to multiplex for PostgreSQL at scale
```

Robust DB access from AKS combines security, efficiency, and resilience. **Auth**: use Workload/Managed Identity so pods get tokens instead of storing DB passwords. **Pooling**: size connection pools with the replicas-×-pool-size-vs-DB-limit math in mind, and at scale front PostgreSQL with **PgBouncer** to multiplex. **Resilience**: implement retries with **exponential backoff and jitter** for transient faults (Azure SQL throttling, brief failovers), and reconnection logic that handles failover — for Azure SQL, connect to the **failover-group listener** so the app does not need a new connection string when the primary moves regions. Validate connections on borrow and set sensible timeouts so a stuck DB does not cascade into pod-wide hangs.

Interview tip: hit auth (Workload Identity, no stored passwords), pooling (the replicas×pool math + PgBouncer), and resilience (retry w/ backoff+jitter, failover-group listener for transparent reconnection). Mentioning connection validation and timeouts to prevent a slow DB from hanging all pods shows you have debugged cascading failures, not just happy-path connectivity.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q100 What is the FinOps operating model and how would you roll it out for an Azure platform? Cost

🎙 **Spoken answer · ~84 s**

FinOps is a cultural and operational practice that makes cloud spending a shared responsibility between engineering, finance, and business, to maximise value. Its lifecycle is inform, optimise, operate. For an Azure platform rollout, start by establishing visibility: a clear subscription and resource group structure, mandatory tagging enforced by Azure Policy, and Cost Management exports to a reporting dataset, with dashboards showing spend by team, product, and environment, and unit metrics like cost per transaction. Introduce showback, then chargeback. Appoint a FinOps lead or small team, with champions in each engineering group. Set up governance: budgets and alerts per team, anomaly detection, policy guardrails on SKUs and regions, and an approval process for large commitments. Optimise in waves: first remove waste, such as idle and oversized resources, then right-size, then commitments, such as reservations and savings plans managed centrally, then architectural improvements like autoscaling, spot, and storage tiering. Embed cost in engineering workflow: cost estimates in design reviews, cost checks in pipelines, and regular reviews in team rituals. Track KPIs: coverage of tagging, commitment utilisation, waste percentage, forecast accuracy, and unit cost trends. Celebrate savings to build momentum, avoid blame, and balance cost against reliability and speed. Iterate quarterly as maturity grows.

Likely follow-ups

- What are the three phases of the FinOps lifecycle?
- Which KPIs show FinOps maturity?

certs.eknathalabs.com

```
# FinOps phases (continuous loop): Inform -> Optimize -> Operate
#  Inform:   tagging, cost allocation (Cost Mgmt + Kubecost), showback dashboards
#  Optimize: rightsizing, spot, reservations/savings plans, kill orphans
#  Operate:  budgets+anomaly alerts, policy guardrails, regular cost reviews
# Make engineers cost-aware: surface cost in PRs (Infracost), per-team dashboards
```

FinOps is a cultural+operational practice giving engineering accountability for cloud cost, run as a continuous loop: **Inform** (visibility — consistent tagging, cost allocation via Cost Management and Kubecost for shared clusters, per-team showback dashboards), **Optimize** (rightsizing from real usage, spot for tolerant workloads, reservations/savings plans for baseline, eliminating orphaned resources), and **Operate** (budgets and anomaly alerts, policy guardrails like required tags and allowed SKUs, regular cross-functional cost reviews). Rollout: start by making cost **visible and attributable** (you cannot optimize what you cannot see), then drive optimization with engineers — surface cost in PRs (Infracost) and give teams dashboards so cost becomes a design input, not an afterthought.

Interview tip: name the Inform→Optimize→Operate loop and stress that *visibility/attribution comes first* (tagging + allocation), because optimization without ownership stalls. The platform-engineering angle: embed cost into the developer workflow (Infracost in PRs, per-team dashboards, policy guardrails) so engineers self-serve cost-awareness. That framing — cost as a first-class platform concern — is the staff/principal signal.

> 📚 **Official docs**
>
> - Microsoft Cost Management
> - AWS Well-Architected: Cost optimization pillar

certs.eknathalabs.com

Q101 How do you migrate a database to Azure with minimal downtime?

Databases

🎙 Spoken answer · ~88 s

Minimal-downtime database migration uses continuous replication followed by a short cutover. Assess first: use Azure Migrate or Data Migration Assistant to check compatibility, sizing, and blockers, and pick the target: Azure SQL Database, Managed Instance, or a Flexible Server. Plan connectivity, with a VPN or ExpressRoute and private endpoints. The Azure Database Migration Service supports online migrations: it performs an initial full load, then continuously applies changes from the source, using change data capture or logical replication, keeping the target in sync. For SQL Server to Managed Instance, the Managed Instance link provides near-real-time replication using distributed availability groups, enabling a low-downtime cutover and even failing back. For PostgreSQL, use logical replication or the migration service. Process: set up the target, migrate schema, run the initial load, and let replication catch up; validate data with row counts and checksums, and test the application against the target, including performance. Rehearse the cutover. At cutover: stop writes on the source, wait until replication lag reaches zero, complete the migration, switch the application's connection string or DNS, and verify. Keep the source available for rollback for a defined period. Minimise risk by lowering DNS TTLs in advance, preparing runbooks, defining go and no-go criteria, scheduling in a low-traffic window, and monitoring closely afterwards.

Likely follow-ups

- What is the Managed Instance link?
- What happens during the final cutover step?

```
# Azure Database Migration Service — online (minimal-downtime) mode:
#  1. initial full load of source -> target
#  2. continuous replication of ongoing changes (CDC) to keep target current
#  3. validate target; brief cutover window when lag ~0
#  4. repoint app connection string -> target; decommission source
```

Minimal-downtime database migration uses **online** migration (continuous replication) rather than a big-bang export/import. **Azure Database Migration Service** in online mode does an initial full load, then continuously replicates ongoing changes (change data capture) so the target stays in near-sync with the live source. You validate the target while the source still serves traffic, then take a **brief cutover** when replication lag approaches zero — stop writes momentarily, let the last changes drain, repoint the application connection string to the target, and decommission the source. Downtime shrinks to the seconds/minutes of cutover rather than the hours a full dump/restore would take. Plan rollback (keep the source intact until validated) and test the cutover beforehand.

certs.eknathalabs.com

Interview tip: contrast online (DMS continuous replication → tiny cutover) with offline (dump/restore → long downtime). The minimal-downtime pattern is full-load + CDC + cutover-at-low-lag + connection repoint. Emphasising validation before cutover and keeping the source as rollback until verified shows you treat a migration as a reversible, tested operation.

📚 **Start here (official docs)**

- Microsoft Azure documentation

Q102 How do you architect for data durability and recovery across an Azure platform?

Storage

🎙 **Spoken answer · ~84 s**

Durability and recovery are layered across storage, databases, and the platform. Storage: choose zone-redundant or geo-zone-redundant storage per criticality, enable soft delete, versioning, and immutability for protection against deletion and corruption, and copy critical backups to a separate account or region. Databases: zone-redundant configurations for HA, geo-replication or failover groups for regional DR, point-in-time restore and long-term retention backups, and for Cosmos DB continuous backup and multi-region replication. Compute and platform state: define infrastructure as code so environments can be rebuilt; keep container images in a geo-replicated registry; store configuration in Git; and keep secrets in Key Vault with soft delete and purge protection, with backup of keys. For AKS, treat clusters as rebuildable, store declarative state in Git through GitOps, and back up persistent volumes and namespaces with Azure Backup for AKS. Protect against human error and malicious action: resource locks, RBAC separation, PIM for privileged roles, immutable vaults, and multi-user authorisation. Define RTO and RPO tiers per workload, and map each tier to the technology. Document runbooks, automate failover where possible, and practise through regular restore tests and DR drills, measuring results. Monitor backup health and replication lag, with alerts. Durability without tested recovery is an assumption, so evidence matters.

Likely follow-ups

- Why keep backup copies in a separate account or region?
- What should be rebuilt from code rather than restored from backup?

```
# Layered durability:
#  - storage redundancy (ZRS/GZRS) for infra-level resilience
#  - DB automated backups + PITR + long-term retention
#  - app-data backups (Velero/Azure Backup for AKS PVs)
#  - immutable/soft-delete to defend against accidental + malicious deletion
```

certs.eknathalabs.com

```
#  - tested restores + documented RTO/RPO per data class
```

Durable-by-design means layering protections and matching them to data criticality. **Infrastructure**: choose storage redundancy (ZRS for zone resilience, GZRS for region) appropriate to the data. **Databases**: automated backups with PITR plus long-term retention for compliance. **Application/cluster data**: back up AKS persistent volumes and Kubernetes state (Velero/Azure Backup), or rely on GitOps for manifests. **Defence in depth against deletion**: soft-delete, versioning, and immutability/legal-hold so neither accidents nor ransomware can permanently destroy data. Critically, define **RTO/RPO per data class** and **test restores** regularly — the architecture is only as good as the last successful recovery drill. Document runbooks so recovery is repeatable under pressure.

Interview tip: present durability as *layers* (storage redundancy → DB backups/PITR → app/PV backups → immutability/soft-delete) tied to per-data-class RTO/RPO, with **tested** restores as the non-negotiable. Calling out immutability/soft-delete as ransomware/accident defence, and "tested restore or it doesn't count," is the principal-level framing that goes beyond "we have backups."

📚 **Official docs**

- AWS Disaster Recovery of workloads (whitepaper)
- Azure Storage redundancy

Q103 How do you set up GitHub Actions to deploy to AKS using OIDC (no stored secrets)?

CI/CD

🎙 **Spoken answer · ~86 s**

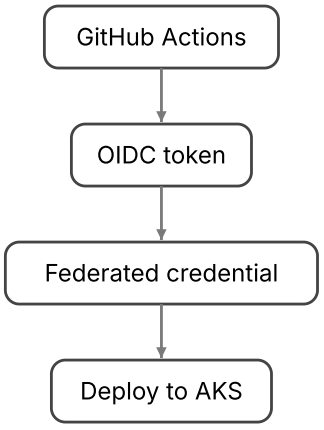
OIDC federation lets GitHub Actions authenticate to Azure without stored secrets. In Entra ID, create an app registration or a user-assigned managed identity, and add a federated credential that trusts GitHub's OIDC issuer for a specific repository and entity, such as a branch, environment, tag, or pull request; the subject claim must match precisely, for example repo:org/repo:environment:production. Grant that identity the minimum Azure roles: AcrPush on the registry, and Azure Kubernetes Service RBAC Writer scoped to the target namespace or a cluster user role plus Kubernetes RBAC. In the workflow, grant the job permissions id-token write and contents read, then use the azure/login action with client ID, tenant ID, and subscription ID, which are not secrets, to obtain a short-lived token. Then use azure/aks-set-context to get cluster credentials, and run kubectl or helm, or build and push images with docker login to ACR via az acr login. Using GitHub environments, require reviewers for production and scope the federated credential to that environment. Benefits: no client secrets to rotate or leak, short-lived tokens, and tight scoping by repository and environment. For private clusters, use self-hosted runners in the network or az aks command invoke. Alternatively, have the workflow update a GitOps repository. Audit sign-ins, and review role assignments.

Likely follow-ups

- What does the subject claim in the federated credential restrict?
- Why use GitHub environments with this setup?

certs.eknathalabs.com

🗺 **OIDC deploy**




```
# 1. Create an Entra ID app + federated credential mapped to the GitHub repo:
```

```
az ad app create --display-name github-deployer
```

```
az ad app federated-credential create --id <APP_ID> --parameters '{
```

```
  "name":"gh-main","issuer":"https://token.actions.githubusercontent.com",
```

```
  "subject":"repo:org/repo:ref:refs/heads/main","audiences":
["api://AzureADTokenExchange"]}'
```

```
# 2. Grant the SP AKS + RBAC permissions (least privilege)
```

```
az role assignment create --assignee <APP_ID> --role 'Azure Kubernetes Service Cluster User
Role' --scope <AKS_ID>
```

```
# 3. Workflow uses azure/login with id-token: write permission
```

```
permissions: { id-token: write, contents: read }
```

OIDC federation removes long-lived client secrets entirely: the GitHub Actions job presents a signed OIDC token, Azure verifies it against the **federated credential** (matched on issuer + subject = repo/branch), and issues a short-lived access token. The `subject` claim scopes which repo/branch/environment can assume the identity. The workflow needs `id-token: write` permission to request the OIDC token. No secret ever lives in GitHub.

📚 **Official docs**

- GitHub Actions: OIDC with AWS
- GitHub Actions: OpenID Connect
- OAuth 2.0 / OIDC overview (OpenID)

certs.eknathalabs.com

Q104 How do you implement progressive delivery (canary / blue-green) on AKS?

CI/CD

🎙 Spoken answer · ~89 s

Progressive delivery on AKS releases to a fraction of users first and widens exposure based on measured health. Tooling options: Argo Rollouts and Flagger are Kubernetes controllers that implement canary and blue-green strategies. With Argo Rollouts, replace the Deployment with a Rollout resource defining steps, such as set weight to ten percent, pause, analyse, then thirty percent, then fully promote. Traffic splitting uses an ingress controller like NGINX, a service mesh like Istio, or Application Gateway integration; without a mesh, replica ratios approximate it. Analysis templates query Prometheus or Azure Monitor for error rate and latency, comparing to thresholds; if the check fails, the controller aborts and rolls back automatically. Blue-green keeps the old ReplicaSet running while the new one is verified through a preview service, then switches the active service, with instant rollback. Pipeline integration: the CI/CD system updates the image tag in Git, GitOps applies it, and the controller drives the rollout. Prerequisites: robust metrics labelled by version, readiness probes, backward-compatible database changes, and sticky routing if needed. Define clear success criteria and a maximum blast radius. Feature flags complement this by decoupling release from deployment. Keep pause and promote controls for manual gates in production. Test the rollback path regularly, and clean up the old version after a stabilisation period.

Likely follow-ups

- How does automated analysis trigger a rollback?
- How is blue-green different from canary in cost and speed?

```
# Argo Rollouts (canary with automated analysis):
```

```
kubectl argo rollouts get rollout myapp --watch
```

```
# Rollout spec: steps with traffic weights + analysis gates
```

```
strategy:
```

```
  canary:
```

```
    steps:
```

```
    - setWeight: 10
```

```
    - pause: {duration: 5m}      # bake time
```

```
    - analysis: {templates: [{templateName: success-rate}]}  # auto-abort on regression
```

```
    - setWeight: 50
```

```
    - pause: {duration: 5m}
```

certs.eknathalabs.com

```
# Flagger is the alternative (works with Istio/AGIC/NGINX + Prometheus metrics)
```

Progressive delivery shifts traffic to the new version gradually while watching metrics, auto-rolling-back on regression. **Argo Rollouts** and **Flagger** are the two main tools on AKS: they integrate with a traffic provider (Istio add-on, AGIC, or NGINX) to split traffic and with Managed Prometheus to evaluate success-rate/latency. Blue-green keeps two full environments and flips traffic at once (instant rollback, double the resources); canary ramps gradually (smaller blast radius, slower). The key safety feature is the **automated analysis gate** that aborts the rollout if golden signals degrade.

📚 **Official docs**

- Google SRE Workbook: Canarying Releases
- Argo Rollouts
- Martin Fowler: BlueGreenDeployment

Q105 How do you manage Helm releases safely in a production AKS pipeline?

CI/CD

🎙 **Spoken answer · ~87 s**

Safe Helm management in production treats charts and values as versioned, reviewed artefacts. Store charts in a registry, such as ACR as an OCI repository, with pinned versions, and keep environment-specific values files in Git, reviewed through pull requests. In the pipeline: lint the chart, render templates with helm template and validate against the Kubernetes schema using kubeconform, run policy checks, and use helm diff to preview changes. Deploy with helm upgrade --install using --atomic, which rolls back automatically on failure, and --wait with a sensible timeout so the pipeline fails if pods don't become ready; set history-max to limit stored release revisions. Use a deployment identity restricted to the namespace. Avoid putting secrets in values files; reference Key Vault via CSI driver or use External Secrets or SOPS. Rollbacks: helm rollback to a previous revision, though remember it doesn't roll back database changes or resources outside the release, and CRDs aren't managed by upgrades, so handle them separately. Prefer GitOps with Flux or ArgoCD to run Helm releases declaratively, with drift detection and automatic reconciliation. Pin dependencies, review upstream chart changes, and test upgrades in staging first. Protect against concurrent releases with pipeline locks. Monitor the rollout and add smoke tests as Helm tests or pipeline steps.

Likely follow-ups

- What does helm --atomic do?
- What can helm rollback not undo?

```
helm upgrade --install myapp ./chart \\
```

```
  --namespace prod --create-namespace \\
```

```
  --values values-prod.yaml \\
```

certs.eknathalabs.com

```
  --atomic --timeout 5m \\
```

```
  --set image.tag=$GIT_SHA
```

```
helm history myapp -n prod        # see revision history
```

```
helm rollback myapp 3 -n prod     # roll back to a known-good revision
```

```
helm diff upgrade myapp ./chart   # (helm-diff plugin) preview changes
```

`--atomic` is the production-critical flag: if the upgrade fails or times out, Helm automatically rolls back to the previous revision, so you never end up in a half-applied state. Pin the image tag to the immutable Git SHA (not `latest`) for traceability and reproducible rollbacks. `helm diff` (plugin) previews exactly what will change before applying — the Helm equivalent of a Terraform plan. `helm history` + `helm rollback` give you fast, revision-based recovery.

> 📚 **Official docs**
>
> - Helm: Chart hooks
> - Helm documentation
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q106 What is the recommended way to handle environment-specific config across dev/staging/prod on AKS?

CI/CD

🎙 Spoken answer · ~85 s

The principle is to build one immutable artefact and vary only configuration per environment. Kubernetes supplies several mechanisms. Kustomize uses a base with overlays per environment that patch values such as replicas, resource limits, ingress hostnames, and image tags, with no templating. Helm uses one chart with per-environment values files. Both live in Git, with a directory or branch structure per environment, and promotion happens through reviewed changes. Non-secret settings go in ConfigMaps; secrets come from Key Vault via the CSI driver with workload identity, or the External Secrets Operator, never from Git in plain text; Sealed Secrets or SOPS are alternatives for encrypted secrets in Git. Keep environments structurally similar, so staging resembles production, with differences limited to scale, endpoints, and credentials. Use separate namespaces, clusters, or subscriptions per environment depending on isolation needs, and Azure App Configuration for centrally managed feature flags and settings with labels per environment. GitOps tools like Flux and ArgoCD reconcile each environment from its path. Follow twelve-factor principles: configuration in the environment, not baked into images. Validate configuration with schema checks and policy in CI, avoid copy-paste drift by keeping shared defaults in the base, and document the promotion process. Make changes auditable and reversible through Git history.

Likely follow-ups

- Why build once and vary only configuration?
- How do Kustomize overlays differ from Helm values?

```
# Kustomize overlays (base + per-env patches):
base/                 # common manifests
overlays/dev/         # kustomization.yaml patches base
overlays/staging/
overlays/prod/
kubectl apply -k overlays/prod
# Helm: one chart, per-env values files:
helm upgrade --install app ./chart -f values-prod.yaml
# Secrets per env: Key Vault (separate vault or scoped secrets) + CSI/ESO
```

certs.eknathalabs.com

Two dominant patterns: **Kustomize overlays** (a shared base plus per-environment patches — no templating, declarative) and **Helm values files** (one chart, one values file per environment). Both keep a single source of truth for the common shape and isolate only what differs (replica counts, resource sizes, hostnames, feature flags). Environment-specific *secrets* come from separate Key Vaults (or scoped secrets) via the CSI driver or External Secrets Operator — never baked into the manifests. With GitOps, each environment maps to a path/branch the controller syncs.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q107 How do you secure the supply chain for container images deployed to AKS? CI/CD

🎙 **Spoken answer · ~88 s**

Securing the supply chain means every image is built from trusted inputs, verified, and only trusted images can run. Build: use minimal pinned base images, scan source dependencies, generate an SBOM, and run builds on isolated ephemeral agents with short-lived credentials. Scan images for vulnerabilities in CI and continuously in Azure Container Registry using Defender for Containers; fail builds on critical findings. Sign images after a successful pipeline using Notation or Cosign, with keys in Key Vault, and attach provenance and SBOM attestations. Registry: private ACR with private endpoints, no anonymous or admin access, AKS pulling via managed identity with AcrPull, and image retention and quarantine policies. Mirror external images into ACR rather than pulling from public registries during deploys. Enforcement at deploy time: Azure Policy for Kubernetes, Ratify, or Kyverno admission rules to allow only images from approved registries, signed by trusted keys, referenced by digest, with no latest tag. Runtime hardening: non-root users, read-only root filesystems, dropped capabilities, and Defender runtime detection. Protect the pipeline itself: branch protection, reviews, restricted workflow permissions, and audit logs. Process: a patching cadence that rebuilds images when base images update, and a way to find affected deployments when a CVE is announced, using SBOMs. Together these controls give integrity from source to cluster.

Likely follow-ups

- How do you find affected workloads when a new CVE is published?
- Why mirror external images into your registry?

```
# 1. Build + push to ACR; scan on push:
```

```
az acr task create --name scan --image myapp:{{.Run.ID}} --context <git> --file Dockerfile
```

```
# Microsoft Defender for Containers scans ACR images for CVEs
```

```
# 2. Sign images (Notation / cosign):
```

```
notation sign myregistry.azurecr.io/myapp@sha256:...
```

certs.eknathalabs.com

```
# 3. Enforce signature + no-CVE at admission (Azure Policy / Ratify + Gatekeeper):
```

```
kubectl get constrainttemplate    # verify-image-signature constraint
```

```
# 4. Only allow images from your ACR (Azure Policy: allowed registries)
```

Defense in depth across the image lifecycle: scan images for vulnerabilities on push (Microsoft Defender for Containers / Trivy in CI), **sign** them (Notation or cosign) to prove provenance, and **enforce at admission** that only signed images from your trusted ACR with no critical CVEs can run — via Azure Policy for AKS (Gatekeeper) plus Ratify for signature verification. Pin by digest (`@sha256:`) not mutable tags. This closes the gap between "built securely" and "what actually runs in the cluster."

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

Q108 How do you roll back a bad deployment on AKS quickly?

CI/CD

> 🎙 **Spoken answer · ~83 s**
>
> Fast rollback requires preparation before you need it. The quickest mechanism for a Deployment is kubectl rollout undo, which reverts to the previous ReplicaSet; kubectl rollout history lists revisions, and you can target one with the to-revision flag. For Helm releases, helm rollback to a prior revision. With GitOps, revert the commit in Git and let Flux or ArgoCD reconcile, which keeps the repository as the truth, and many teams make this the standard path. With progressive delivery tools, abort the rollout, and traffic returns to the stable version immediately; with blue-green, switch the service back. Feature flags offer the fastest rollback of all: turn off the feature with no deployment. Decide quickly using clear signals: an error rate or latency alert tied to a deployment, with automatic rollback if configured. Considerations that complicate rollback: database migrations, which must be backward compatible so the previous version still works; messages or events produced in a new format; and caches. Keep the previous image available in the registry, don't prune it, and keep revisionHistoryLimit adequate. Make rollback a one-click or automatic pipeline step, rehearse it, and measure time to restore. After rollback, communicate, investigate in a non-production environment, and write a review.
>
> **Likely follow-ups**
>
> - Why make database changes backward compatible for rollback?
> - When would you fix forward instead of rolling back?

```
# Native Kubernetes Deployment rollback:
```

```
kubectl rollout history deployment/myapp -n prod
```

```
kubectl rollout undo deployment/myapp -n prod                # to previous revision
```

certs.eknathalabs.com

```
kubectl rollout undo deployment/myapp -n prod --to-revision=5 # to a specific one
```

```
kubectl rollout status deployment/myapp -n prod              # watch it complete
```

```
# Helm:   helm rollback myapp <rev> -n prod
```

```
# GitOps: git revert the manifest commit → Argo/Flux auto-syncs back
```

The fastest mechanism depends on how you deploy. Plain Deployments keep a revision history, so `kubectl rollout undo` reverts to the prior ReplicaSet near-instantly. Helm uses `helm rollback` to a known-good revision. With **GitOps**, you `git revert` the offending manifest commit and the controller reconciles the cluster back — keeping the rollback auditable in Git. Always confirm with `rollout status`. The principle echoes Terraform: rollback is roll-forward to a previous known-good desired state.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q109 How do you manage database schema migrations safely during AKS deployments?

CI/CD

🎙 **Spoken answer · ~88 s**

Schema migrations during AKS deployments need to avoid races, downtime, and incompatibility. Run migrations as a dedicated step: a Kubernetes Job or Helm pre-upgrade hook, or a pipeline task, executed once before the application rollout, instead of at application startup in every pod, where concurrent replicas could run it simultaneously or block readiness. Use a migration tool like Flyway, Liquibase, or the framework's migrations, which take a lock and track applied versions. Make every migration backward compatible with the currently running application version, following expand and contract: add nullable columns, new tables, and indexes first, deploy code that works with both schemas, backfill data in batches, and only later remove old structures in a subsequent release. Avoid long locks: create indexes online, avoid large table rewrites, set lock timeouts, and test on production-sized data in staging to measure duration. Use separate database credentials for migrations, with elevated rights, and for the runtime with least privilege. Take a backup or restore point before risky changes. Plan rollback: keep the schema compatible so the application can be rolled back without reversing the migration; write down migrations only when safe. Gate the pipeline: if the migration fails, stop the rollout. Monitor database load and replication during migration, and communicate schedules for heavy changes.

Likely follow-ups

- Why avoid running migrations at pod startup?
- What does a backward-compatible migration allow during rollback?

```
# Run migrations as a separate, gated step — not in app startup:
```

© EknathaLabs.com

```
kubectl create job --from=cronjob/migrate migrate-$GIT_SHA -n prod
```

```
# or an initContainer / Helm pre-upgrade hook:
```

```
annotations: { "helm.sh/hook": pre-upgrade, "helm.sh/hook-weight": "-5" }
```

```
# Expand/contract pattern across deploys:
```

```
#  1) add column (backward compatible)  2) deploy code using it
```

```
#  3) backfill  4) later: drop old column
```

Treat migrations as a distinct, gated pipeline step (a Job or Helm pre-upgrade hook), not something the app runs on startup — so a rolling deploy does not race multiple replicas trying to migrate. Use the **expand/contract** pattern so old and new app versions both work against the schema during the rollout: add the new structure first, deploy code, backfill, and only later remove the old — each as a separate, independently-rollbackable change. This is what makes zero-downtime deploys and safe rollbacks possible with a stateful database.

> 📚 **Official docs**
>
> - gh-ost: GitHub's online schema migration
> - Azure Kubernetes Service (AKS)
> - Flyway documentation

certs.eknathalabs.com

Q110 How do you design Entra ID / RBAC for least privilege across many teams and subscriptions?

IAM & EntraID

🎙 Spoken answer · ~88 s

Least-privilege design for many teams starts with structure and automation. Use management groups and subscriptions that match ownership and risk, so that permissions can be scoped narrowly: a subscription or resource group per team and environment. Assign Azure RBAC roles to Entra security groups, not individuals, and use built-in roles with the narrowest scope; create custom roles only where necessary. Separate duties: platform teams own shared infrastructure, product teams get contributor-level rights only on their resource groups, with production being more restricted than non-production. Use Privileged Identity Management so elevated roles are eligible, time-bound, approved, and logged, and avoid standing Owner or Contributor at subscription level. For workloads, use managed identities and workload identity with narrowly scoped roles, never shared secrets. For AKS, integrate Entra ID, enable Azure RBAC for Kubernetes, restrict namespaces per team, and disable local accounts. Add guardrails with Azure Policy and deny assignments so teams can't create overly permissive configurations, and use Conditional Access to require MFA and compliant devices. Govern access over time with access reviews, entitlement management packages for requests and expiry, and automated removal on team changes. Manage role assignments as code so changes are reviewed. Monitor with activity logs and alerts on privileged role use, and periodically audit for overly broad assignments.

Likely follow-ups

- Why assign roles to groups instead of individuals?
- How do access reviews keep permissions from accumulating?

```
# Management Group hierarchy mirrors org; policy + RBAC inherit down:
```

```
Root MG → Platform MG / BU MGs → subscriptions (prod/nonprod) → RGs
```

```
# Assign roles to Entra GROUPS, never individual users:
```

```
az role assignment create --assignee-object-id <GROUP_ID> --assignee-principal-type Group
\\
```

```
  --role 'Contributor' --scope <RG_ID>
```

```
# PIM for just-in-time elevation of privileged roles:
```

```
# eligible (not active) assignments; activate with approval + time limit
```

Structure subscriptions under a **Management Group** hierarchy that mirrors the org, so RBAC and Azure Policy inherit downward consistently. Assign roles to **Entra ID groups** (driven by your IdP/HR system), never to individuals — so joining/leaving a team automatically grants/revokes access everywhere. Default people to least privilege (Reader), grant Contributor narrowly at the right scope, and use **Privileged Identity**

certs.eknathalabs.com

Management (PIM) for just-in-time, time-bound, approval-gated elevation of high-risk roles (Owner, User Access Administrator) instead of standing admin. Manage these assignments as code (Terraform/Bicep) for auditability.

📚 **Official docs**

- Kubernetes: RBAC
- Kubernetes: RBAC good practices
- Microsoft Entra ID: What is Microsoft Entra ID?

Q111 A pod on AKS needs to access Azure SQL without storing credentials. How do you set it up?

IAM & EntraID

🎙 **Spoken answer · ~90 s**

To access Azure SQL without stored credentials, use Entra authentication with a workload identity. Steps: enable the OIDC issuer and workload identity on the AKS cluster; create a user-assigned managed identity; create a federated identity credential on it, trusting the cluster's issuer and the subject for the pod's service account, in system:serviceaccount:namespace:name form. Configure the Azure SQL server with an Entra admin, then connect to the database as that admin and create a contained database user from the managed identity, with CREATE USER name FROM EXTERNAL PROVIDER, and grant least-privilege database roles such as db_datareader and db_datawriter, or custom permissions. In Kubernetes, create the service account annotated with the identity's client ID, and label the pod template with azure.workload.identity/use equal to true. In the application, use the Azure Identity library, through DefaultAzureCredential or WorkloadIdentityCredential, and a SQL driver that supports access tokens or the Active Directory default authentication mode, so the connection obtains a token for the database resource scope. No password appears in config, secrets, or images. Handle token expiry by refreshing on new connections, since pooled connections persist; set connection lifetime accordingly. Network: use a private endpoint and restrict public access. Troubleshoot with the error codes if login fails: check the federated credential subject, the database user, and roles. Monitor sign-ins and audit.

Likely follow-ups

- What database object do you create for the identity?
- How does token expiry interact with connection pooling?

```
# 1. Enable Workload Identity on the cluster (OIDC issuer + addon):
```

```
az aks update -g rg -n prod-aks --enable-oidc-issuer --enable-workload-identity
```

```
# 2. Create a user-assigned managed identity + federated credential to the K8s SA:
```

```
az identity create -g rg -n app-mi
```

coris.eknathalabs.com

```
az identity federated-credential create --identity-name app-mi -g rg \\
  --issuer <OIDC_ISSUER_URL> --subject system:serviceaccount:prod:app-sa
# 3. Grant the MI access to SQL (Entra-auth), annotate the ServiceAccount:
# serviceAccount annotation: azure.workload.identity/client-id: <MI_CLIENT_ID>
# 4. App uses DefaultAzureCredential → token → SQL (no password)
```

Workload Identity federates a Kubernetes ServiceAccount with an Entra ID managed identity via OIDC. The pod's projected SA token is exchanged for an Entra token, which authenticates to Azure SQL using Entra authentication — no connection-string password anywhere. Setup: enable the OIDC issuer + workload-identity add-on, create a managed identity with a **federated credential** whose subject is the SA, grant that identity SQL access, and annotate the SA with the identity's client-id. In code, `DefaultAzureCredential()` transparently picks up the workload identity. This is the modern, secret-less replacement for the deprecated pod-managed-identity (aad-pod-identity).

📚 **Official docs**

- Azure Kubernetes Service (AKS)
- Azure SQL documentation

certs.eknathalabs.com

Q112 What is Microsoft Defender for Cloud and how does it fit an AKS security posture?

IAM & EntraID

🎙 Spoken answer · ~88 s

Defender for Cloud gives AKS security visibility and protection in two ways: posture management and threat protection. Posture: it assesses subscriptions and clusters against security benchmarks and produces a secure score and recommendations, such as enabling private clusters, Entra integration, Azure Policy, restricting API server access, and avoiding privileged containers, with attack path analysis showing exposure and risk. Defender for Containers adds registry scanning for vulnerabilities in ACR images and running images, runtime threat detection through a sensor on nodes that spots suspicious process execution, cryptominers, and privilege escalation, and analysis of the Kubernetes audit log for risky API activity. It also integrates with Azure Policy to apply admission rules to workloads. How it fits a security posture: it is the detection and visibility layer on top of preventive design — network isolation, identity, supply chain controls, and policy. Rollout: enable the Defender CSPM plan and Defender for Containers on relevant subscriptions, deploy the sensor, review recommendations and assign owners, and set up continuous export of alerts to Sentinel or ticketing, with workflow automation for response. Tune the noise by suppressing known benign alerts and exempting resources with justification. Measure progress via the secure score and mean time to remediate. It doesn't replace secure engineering practices or incident response processes.

Likely follow-ups

- What does the runtime sensor detect?
- What does attack path analysis show?

```
# Enable Defender plans:
az security pricing create --name Containers --tier Standard
az security pricing create --name KeyVaults --tier Standard
# Defender for Containers provides:
#  - image vulnerability scanning (ACR + running images)
#  - runtime threat detection (suspicious exec, crypto-mining, etc.)
#  - K8s hardening recommendations (CIS benchmark)
# Secure Score tracks posture; export findings to SIEM (Sentinel)
```

Microsoft Defender for Cloud is the CNAPP/CSPM layer: it continuously assesses your security posture against benchmarks (CIS, regulatory), surfaces misconfigurations with a **Secure Score**, and — with Defender for Containers — scans images for CVEs, detects runtime threats (anomalous process execution, crypto-

certs.eknathalabs.com

mining, reverse shells in pods), and recommends AKS hardening. Findings flow into Microsoft Sentinel (SIEM) for correlation and response. It complements (does not replace) admission-time controls like Azure Policy/Gatekeeper: Defender detects and recommends, Policy prevents.

📚 **Official docs**

- Microsoft Defender for Cloud
- Azure Kubernetes Service (AKS)

Q113 How do you implement per-team / per-namespace cost allocation (showback) on AKS? Cost

🎙 **Spoken answer · ~89 s**

Showback attributes cluster cost to teams so they can see and manage what they use. Foundation: a consistent model for ownership, namespaces per team or application, with labels such as team, cost-centre, app, and environment, enforced by admission policy so workloads can't deploy unlabelled. Use the AKS cost analysis add-on, which integrates with Azure Cost Management to break down cluster cost by namespace and workload, or OpenCost and Kubecost, which allocate node cost to pods based on requested and used CPU, memory, GPU, plus storage and network. Cost sharing choices: allocate by the larger of requests and usage to encourage accurate sizing; show idle capacity separately, then decide how to distribute it; treat shared platform components, like ingress, monitoring, and system pods, as overhead split proportionally or kept central. For stronger attribution, use dedicated node pools per team with tags. Publish dashboards in Grafana or Power BI, with monthly reports and per-team budgets and alerts. Enforce ResourceQuotas and LimitRanges to cap consumption and require requests. Make the data actionable: highlight over-provisioned workloads with right-sizing recommendations, unused persistent volumes, and idle namespaces. Hold regular reviews where teams own their numbers. Be transparent about methodology, validate that totals reconcile to the Azure invoice, and iterate. Start with showback; add chargeback only when the allocation is trusted.

Likely follow-ups

- Why enforce labels with admission policy?
- Why reconcile allocated totals to the invoice?

```
# 1. Tag resources for cost attribution (team, env, cost-center):
```

```
az aks update -g rg -n prod-aks --tags team=payments env=prod costcenter=cc-123
```

```
# 2. AKS Cost Analysis add-on (OpenCost-based) for namespace-level breakdown:
```

```
az aks update -g rg -n prod-aks --enable-cost-analysis
```

```
# 3. Azure Cost Management: group by tag, set budgets + alerts:
```

certs.eknathalabs.com

```
az consumption budget create --budget-name payments-monthly --amount 5000 ...
```

```
# 4. OpenCost / Kubecost for in-cluster pod-level cost
```

Cluster costs are shared, so allocation needs a breakdown *inside* the cluster plus tagging *outside*. The AKS **Cost Analysis** add-on (built on OpenCost) attributes node cost to namespaces/workloads by their resource requests, giving per-team showback. Combine with consistent resource **tags** (team/env/cost-center) so Azure Cost Management can group spend, and set per-team **budgets with alerts**. For finer pod-level granularity and chargeback, OpenCost/Kubecost map CPU/memory/storage/network to teams. The platform move: make this self-service so teams see their own spend and own optimizing it.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

Q114 AKS spend spiked unexpectedly this month. How do you investigate and bring it down? Cost

🎙 **Spoken answer · ~88 s**

I treat an AKS spend spike as an investigation, then a control exercise. Begin in Cost Management: compare daily cost against prior periods, grouped by resource type, resource group, and meter, to see whether the increase is compute, storage, networking, or monitoring. For an AKS cluster, check the node resource group, where VMs, disks, load balancers, and public IPs live. Check node counts over time: autoscaler activity showing nodes scaled up and not back down, a raised maximum, or new node pools, perhaps with expensive GPU or large VMs. Identify why: a runaway workload with huge replica counts, requests inflated so more nodes are needed, pods stuck preventing scale-down, or a load test left running. Check Log Analytics ingestion for log floods from a crash loop or verbose logging, egress and cross-zone traffic, unattached persistent volumes and snapshots, and public IPs or load balancers created by services. Use cost analysis at namespace level to find the responsible workload. Bring it down: remove or scale down the offending resources, fix requests and limits, tune the autoscaler's scale-down settings, enable spot where suitable, delete orphaned disks, cap log ingestion, and move dev and test clusters to schedules. Then prevent recurrence with budgets, alerts, quotas, policies, and regular reviews, and document the cause.

Likely follow-ups

- Where do you look first for AKS compute cost growth?
- What commonly stops the autoscaler from scaling down?

```
# 1. Where did it grow? Cost Management cost analysis, group by service/resource:
```

```
az consumption usage list --start-date 2026-05-01 --end-date 2026-05-31 -o table
```

```
# 2. Common culprits:
```

certs.eknathalabs.com

```
#   - Cluster Autoscaler scaled out (a workload over-requesting CPU/mem)
#   - egress / cross-AZ / cross-region data transfer
#   - orphaned managed disks / public IPs / load balancers from deleted workloads
#   - logging volume into Log Analytics (ingestion is billed by GB)
# 3. Right-size requests, add Spot pools, set Log Analytics daily cap
```

Start in Azure Cost Management to find *what* grew (group by service and resource). Frequent AKS cost spikes come from: the Cluster Autoscaler adding nodes because a workload over-requests resources (fix the requests), a surge in **data transfer** (egress, cross-AZ chatter), **orphaned resources** left by deleted workloads (unattached disks, idle public IPs/load balancers), and **Log Analytics ingestion** (logs are billed per GB — a chatty app or debug logging can dominate the bill). Remediate with right-sizing, Spot node pools for tolerant workloads, a Log Analytics daily cap, and a sweep for orphaned resources. Then add a budget alert so the next spike is caught early.

> 📚 **Official docs**
>
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q115 How do you decide between Reserved Instances, Savings Plans, and Spot for AKS compute?

Cost

🎙 Spoken answer · ~88 s

The three suit different parts of an AKS compute profile. Reservations or savings plans cover the stable baseline: the minimum node count you run around the clock. Reserved instances give the biggest discount for a fixed VM family and size in a region, good for system pools and steady user pools whose shape won't change; savings plans trade a smaller discount for flexibility across VM families and regions, better when you expect to change sizes, adopt newer generations, or shift between pools. Spot node pools cover tolerant workloads: batch, CI agents, stateless workers, and dev or test, at deep discounts but subject to eviction at short notice, so they need multiple replicas, graceful handling, tolerations, and fallback pools. On-demand covers the variable remainder and anything critical. Method: measure usage over several months to find the floor, commit to a conservative portion of it, perhaps sixty to eighty percent, stagger terms, and review utilisation. Right-size and optimise before committing, so you don't lock in waste, and consider planned changes such as moving to Arm VMs. Use the cluster autoscaler or node auto-provisioning so on-demand and spot capacity follows demand. Note that AKS bills VMs at standard rates, so reservations apply to the underlying VMs automatically. Track coverage and utilisation as KPIs.

Likely follow-ups

- How do you decide what portion of the baseline to commit?
- Which AKS workloads belong on spot?

```
# Reserved VM Instances: 1 or 3 year, specific VM family/region, up to ~72% off
```

```
# Azure Savings Plans: 1 or 3 year hourly $ commitment, flexible across VM types/regions
```

```
# Spot: up to ~90% off, evictable with 30s notice, no SLA
```

```
# Typical mix for an AKS estate:
```

```
#   baseline system + steady workloads → Reserved / Savings Plan
```

```
#   bursty / batch / fault-tolerant     → Spot node pools
```

```
#   unpredictable spikes                → on-demand
```

Match the commitment to the workload's predictability. **Reserved Instances** give the deepest discount but lock you to a VM family/region — best for a stable, known baseline. **Savings Plans** trade a little discount for flexibility (an hourly $ commitment that applies across VM types/regions) — better when your instance mix evolves. **Spot** is cheapest but evictable — only for fault-tolerant, stateless, or batch workloads with PodDisruptionBudgets. The mature AKS pattern blends all three: cover the steady baseline with a Savings Plan/Reservations, run tolerant workloads on Spot pools, and leave a thin on-demand layer for unpredictable bursts.

certs.eknathalabs.com

📚 **Official docs**

- AWS Savings Plans
- Azure Reservations
- Azure Kubernetes Service (AKS)

Q116 How do you define and track SLOs for a service running on AKS?

Monitoring

🎙 **Spoken answer · ~86 s**

To define SLOs for an AKS service, start from user journeys and pick a small number of SLIs, usually availability, the proportion of successful requests, and latency, the proportion under a threshold such as three hundred milliseconds. Source the data from the ingress controller or Application Gateway metrics, or from application telemetry, using Prometheus or Application Insights, measured as close to the user as practical. Set SLO targets based on user expectations and historical performance, like ninety-nine point nine percent over a rolling thirty days, and derive the error budget, the allowed failure. Implement with recording rules in managed Prometheus or KQL queries, computing good over total events across windows. Visualise in Grafana: current SLI, budget remaining, and burn rate. Alert on multi-window burn rates: a fast burn pages, a slow burn tickets. Document the SLOs with owners, so they're agreed with product stakeholders. Use the error budget in decisions: if much remains, release faster and experiment; if depleted, slow down and prioritise reliability. Account for dependencies, since your SLO can't exceed the combined availability of your critical dependencies. Review SLOs quarterly, refine definitions, and keep them few and meaningful. Track compliance in reports for leadership. Start simple and improve rather than waiting for perfect measurements.

Likely follow-ups

- Why is the error budget useful for release decisions?
- How do dependencies limit your achievable SLO?

```
# Pick SLIs from golden signals (availability, latency):
```

```
# Availability SLI (PromQL): good requests / total requests
```

```
sum(rate(http_requests_total{code!~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

```
# Latency SLI: fraction of requests under threshold
```

```
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

```
# SLO: 99.9% availability over 28d → error budget = 0.1%
```

```
# Alert on BURN RATE, not raw threshold (multiwindow: fast + slow burn)
```

certs.eknathlabs.com

Define **SLIs** (measurable signals like success ratio and latency-under-threshold) from Managed Prometheus, set an **SLO** target (e.g. 99.9% over 28 days), and derive the **error budget** (the allowed unreliability). The key practice is alerting on **burn rate** — how fast you are consuming the error budget — using multi-window multi-burn-rate alerts (a fast-burn alert pages immediately on a severe spike; a slow-burn alert warns on gradual erosion). This pages on user-impacting problems while suppressing noise, and the error budget becomes the objective input to the velocity-vs-reliability decision (freeze risky changes when the budget is spent).

📚 **Official docs**

- Google SRE Book: Service Level Objectives
- Azure Kubernetes Service (AKS)

Q117 How do you implement distributed tracing for microservices on AKS?

Monitoring

🎙 **Spoken answer · ~85 s**

Distributed tracing on AKS uses OpenTelemetry for instrumentation and Azure Monitor Application Insights as a backend. Each service is instrumented, with the language SDK or auto-instrumentation agent, to create spans for inbound requests and outbound calls, propagating W3C trace context headers across HTTP and messaging so spans link into one trace. Telemetry is sent to an OpenTelemetry Collector in the cluster, deployed as a DaemonSet or gateway, which handles batching, sampling, and exporting to Application Insights, or alternatively to Jaeger or Tempo. Application Insights then provides the application map, transaction timelines, and failure analysis. Add meaningful attributes, such as customer tier, version, and operation names, and include the trace ID in structured logs for correlation, so engineers can pivot from a slow trace to the relevant log lines. Use sampling to manage cost: probabilistic head sampling at the SDK, and tail sampling at the collector to retain all errors and unusually slow traces. Cover asynchronous flows by propagating context in message properties. Verify coverage by checking that every hop appears in traces; gaps indicate uninstrumented components. Use traces for latency analysis, dependency discovery, and validation of deployments through comparing versions. Secure and size the collector, monitor its drops, and avoid sensitive data in attributes.

Likely follow-ups

- Why propagate trace context through message queues?
- What is the benefit of tail sampling at the collector?

```
# Instrument with OpenTelemetry (vendor-neutral SDK + auto-instrumentation):
```

```
# app emits spans → OTel Collector (DaemonSet/Deployment) → backend
```

```
# Backends: Application Insights, or Managed Grafana + Tempo/Jaeger
```

```
# Service mesh (Istio add-on) can emit spans without code changes,
```

```
#   but app must propagate trace context headers (traceparent / b3)
```

ceris.eknathalabs.com

```
# Correlate traces ↔ logs ↔ metrics via trace_id
```

OpenTelemetry is the standard: instrument services with the OTel SDK (or auto-instrumentation), emit spans to an **OTel Collector** running in-cluster, and export to a backend — Azure **Application Insights** (native APM) or Grafana Tempo/Jaeger. The critical detail is **context propagation**: each service must forward the trace headers (`traceparent`) so spans stitch into one end-to-end trace; even an Istio sidecar emitting spans relies on the app passing the headers through. The big win is correlating traces with logs and metrics by `trace_id`, so you can jump from a latency spike to the exact slow span across services.

📚 **Official docs**

- OpenTelemetry: Signals (traces, metrics, logs)
- Azure Kubernetes Service (AKS)

Q118 Intermittent 5xx errors appear only under load on AKS. How do you diagnose?

Troubleshooting

🎙 **Spoken answer · ~87 s**

Errors that appear only under load point to saturation, concurrency, or scaling limits. Reproduce with a load test while observing. Look at the pattern: which endpoints, which pods, which error codes, and at what request rate they begin. Check pod resource behaviour: CPU throttling when limits are hit, memory growth and OOM kills, thread pool or connection pool exhaustion, and file descriptor limits. Check scaling: whether HPA reacts fast enough, whether new pods are ready, and if the node autoscaler delays capacity; pods starting while traffic arrives can fail if readiness is misconfigured. Check dependencies: database connection limits and slow queries, throttling from downstream services or Azure resources, Redis saturation, and DNS load on CoreDNS. Check the network path: ingress controller capacity and its connection and timeout settings, Application Gateway autoscale limits, SNAT port exhaustion on outbound connections, conntrack table limits, and load balancer idle timeouts. Examine upstream timeouts and keep-alive mismatches that cause 502s. Look for retry storms amplifying load. Use traces to see where latency grows, and metrics to see saturation signals approach limits. Fix the bottleneck: tune pools and limits, raise replicas or capacity, add caching, introduce rate limiting and backpressure, optimise queries, and fix probes. Retest at higher load, and set alerts on saturation metrics.

Likely follow-ups

- What does throttling look like for CPU-limited pods?
- How can retries make an overload worse?

```
# 1. Confirm pattern: do 5xx correlate with traffic/HPA scaling events?
```

```
kubectl get hpa -n prod -w       # is it scaling fast enough?
```

```
# 2. CPU throttling (silent killer under load):
```

cors.eknathalabs.com

```
rate(container_cpu_cfs_throttled_seconds_total[5m]) > 0
```

```
# 3. Connection / pool exhaustion to DB or upstream:
```

```
#    DB max_connections hit; app connection pool too small
```

```
# 4. Readiness flapping under load → pods pulled from Endpoints
```

```
kubectl get events -n prod | grep -i unhealthy
```

```
# 5. Node-level: conntrack table full, SNAT port exhaustion (egress)
```

Load-only 5xx point to a resource that saturates under pressure. Work the likely causes: **CPU throttling** (CFS limits choke the app silently — check throttled-seconds; often the fix is higher/removed CPU limits), **HPA scaling too slowly** (requests pile up before new pods arrive — tune target + scale-up behavior, or pre-scale), **connection-pool / DB max-connections exhaustion**, **readiness probes flapping** under load (pods get yanked from Endpoints, concentrating load on the rest), and at node level **SNAT/conntrack exhaustion** for high outbound connection counts. Correlate the 5xx timeline with scaling events and golden signals to localize which one it is, then load-test the fix.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- Azure Kubernetes Service (AKS)

Q119 An AKS cluster upgrade is stuck or failing. How do you handle it?

Troubleshooting

🎙 **Spoken answer · ~67 s**

A stuck or failing AKS upgrade needs systematic diagnosis; AKS reports the failure reason in the operation status and activity log. Check the cluster and node pool provisioning state, and the error message from az aks show or the portal's activity log. Common causes. Insufficient quota or capacity: surge nodes can't be created because of VM core quota limits or regional capacity; request more quota or reduce max-surge. Subnet IP exhaustion with Azure CNI: no addresses for surge nodes. PodDisruptionBudgets blocking drains: the upgrade waits because evicting pods would violate the budget; relax the PDB temporarily or fix the workload, noting that AKS has a drain timeout. Pods that can't be evicted or reschedule, like those with local storage or insufficient capacity elsewhere. Webhook or policy failures that break system components. Network or firewall rules blocking required endpoints, so new nodes can't join. Deprecated APIs or incompatible add-ons breaking after the control plane upgrade. Node join failures or custom script issues.

Likely follow-ups

- How do PodDisruptionBudgets block an upgrade?
- What pre-checks prevent an upgrade from failing?

certs.eknathalabs.com

```
az aks show -g rg -n prod-aks --query provisioningState   # Failed / Upgrading?
```

```
kubectl get nodes                 # mixed versions? a node stuck draining?
```

```
kubectl describe node <stuck>     # cordoned, pods not evicting?
```

```
# common blocker: a PodDisruptionBudget that can't be satisfied blocks drain
```

```
kubectl get pdb -A
```

```
# retry the upgrade (idempotent), or upgrade control plane and node pools separately:
```

```
az aks upgrade -g rg -n prod-aks --control-plane-only
```

```
az aks nodepool upgrade -g rg --cluster-name prod-aks -n workers --kubernetes-version 1.30
```

AKS upgrades go control-plane-first, then node pools, draining nodes one at a time. The most common stall is a node that cannot **drain** because a **PodDisruptionBudget** can never be satisfied (e.g. minAvailable equals replica count), or a pod with no controller / long terminationGracePeriod. Diagnose with `provisioningState`, node versions, and PDBs. Fixes: relax the blocking PDB, ensure replicas can tolerate one-node disruption, then retry (the operation is idempotent). Upgrading the control plane and node pools **separately** (and one pool at a time) isolates failures and limits blast radius. Always upgrade non-prod first and stay within the supported version skew.

> 📚 **Official docs**
>
> - Kubernetes: Upgrading kubeadm clusters
> - AKS: Upgrade a cluster
> - Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q120 Capstone: design the production readiness checklist for a new service going live on AKS.

Troubleshooting

🎙 Spoken answer · ~86 s

A production readiness checklist makes sure a service is operable before launch. Architecture and reliability: at least two or three replicas spread across zones with anti-affinity or topology spread, PodDisruptionBudgets, sensible resource requests and limits, readiness, liveness, and startup probes configured properly, graceful shutdown, autoscaling with tested limits, timeouts, retries, and circuit breakers for dependencies, and a load test showing capacity against expected peaks. Observability: golden-signal metrics, structured logs with correlation IDs, tracing, dashboards, defined SLOs, and actionable alerts with runbooks and an on-call owner. Security: least-privilege workload identity, no secrets in images, secrets from Key Vault, network policies, non-root containers with restricted security context, images scanned and signed, private endpoints for data stores, and dependency vulnerability checks. Delivery: infrastructure as code, a CI/CD pipeline with automated tests, progressive rollout and tested rollback, backward-compatible migrations, and configuration per environment. Data: backups with tested restores, defined RPO and RTO, and DR plan. Operations: documentation, ownership recorded in the service catalogue, incident response contacts, capacity and cost estimates with budgets and tags, and a launch plan with go and no-go criteria. Compliance and privacy reviews where relevant. Run a game day to test failure scenarios. Review the checklist with an independent reviewer, and track exceptions with owners and dates.

Likely follow-ups

- Which items on the checklist would block a launch?
- Why run a game day before going live?

```
Reliability:  liveness+readiness+startup probes; resource requests/limits; PDB; multi-AZ; HPA
```

```
Rollout:      immutable image by digest; canary/blue-green; --atomic Helm; tested rollback
```

```
Security:     Workload Identity (no secrets); Key Vault CSI; signed images; NetworkPolicy; non-root
```

```
Networking:   Private Endpoints for data; NSGs; ingress + TLS; WAF if internet-facing
```

```
Observability: golden-signal dashboards; SLOs + burn-rate alerts; structured logs; tracing
```

```
Data:         backups + tested restore; expand/contract migrations; connection-pool sizing
```

```
Cost:         right-sized requests; Spot where tolerant; tags for showback; budget alert
```

```
Ops:          runbook; on-call + escalation; capacity/load test; chaos/failure drill
```

certs.eknathalabs.com

A production-readiness review is the staff-level synthesis of everything: **reliability** (correct probes, requests/limits, PodDisruptionBudgets, multi-AZ spread, autoscaling), **safe rollout** (immutable digests, progressive delivery, atomic deploys, a *tested* rollback), **security** (secret-less Workload Identity, Key Vault integration, signed images enforced at admission, NetworkPolicies, non-root containers), **networking** (Private Endpoints for data, ingress TLS, WAF when exposed), **observability** (golden-signal dashboards, SLOs with burn-rate alerts, logs, tracing), **data** (backups with tested restores, expand/contract migrations), **cost** (right-sizing, Spot, tagging, budgets), and **operational readiness** (runbook, on-call, load and failure testing). The signal interviewers want: you treat "going live" as a deliberate, checklist-driven gate — and ideally codify it as an automated, self-service readiness check so the safe path is the default.

📚 **Official docs**

- Kubernetes: Liveness, readiness and startup probes
- Azure Kubernetes Service (AKS)

Q121 What is the difference between Azure RBAC roles and Microsoft Entra roles, and where does each apply?

IAM & EntraID

🎙 **Spoken answer · ~59 s**

Azure RBAC governs what a principal can do to Azure resources. A role assignment ties a principal, a role definition such as Reader, Contributor or a custom role, and a scope, which can be a management group, subscription, resource group or single resource, and permissions inherit downward. Microsoft Entra roles govern the directory: creating users and groups, registering applications, managing Conditional Access or resetting passwords. A Global Administrator, for example, does not automatically have rights over subscriptions, though they can elevate themselves, and a subscription Owner has no power in the directory. I keep both models least-privilege: use groups for assignments, scope narrowly, prefer built-in roles, use Privileged Identity Management for time-limited elevation, and review access regularly. When troubleshooting a denied action, I first work out which plane the action belongs to, then look at assignments, deny assignments and Azure Policy.

Likely follow-ups

- What is a deny assignment?
- How does elevate access work?

Azure RBAC controls access to Azure resources (subscriptions, resource groups, VMs, storage) using role assignments at a scope. Entra roles control the identity directory itself: users, groups, apps, Conditional Access. They are separate systems with separate assignment points.

certs.eknathalabs.com

```
az role assignment create --assignee <id> --role 'Contributor' --scope
/subscriptions/<sub>/resourceGroups/app-rg
```

```
az role assignment list --assignee <id> --all -o table
```

```
az rest --method get --url https://graph.microsoft.com/v1.0/directoryRoles
```

Interview tip: Mention the elevate-access toggle for Global Administrators, which is the bridge between the two worlds, and that it should be rare and audited.

📚 **Official docs**

- Kubernetes: RBAC
- Azure RBAC overview

Q122 System-assigned versus user-assigned managed identities: when do you use each?

IAM & EntraID

🎙 **Spoken answer · ~60 s**

Both kinds of managed identity give an Azure resource an Entra identity so that code can get tokens without storing credentials. A system-assigned identity is created with the resource, has the same lifecycle, and is the simplest choice for a single VM or app that needs access to one thing. A user-assigned identity is created as its own resource, can be shared by multiple resources, and persists if they are replaced. I prefer user-assigned in infrastructure as code because I can create the identity and its role assignments first, then attach it, avoiding race conditions during deployment, and it fits scenarios like scale sets, many instances of the same app, or blue-green swaps. The trade-off is that a shared identity widens the blast radius, so I do not share one across unrelated workloads. On AKS I use Workload Identity federated to a user-assigned identity.

Likely follow-ups

- How does a pod use a managed identity on AKS?
- What happens to role assignments when a system identity is deleted?

A system-assigned identity is tied to one resource and deleted with it. A user-assigned identity is a standalone resource that can be attached to many resources and survives their deletion. Both give tokens without secrets.

```
az identity create -g app-rg -n web-uami
```

```
az vm identity assign -g app-rg -n vm1 --identities web-uami
```

```
az role assignment create --assignee-object-id <principalId> --role 'Storage Blob Data
Reader' --scope <storage-id>
```

certs.eknathalabs.com

Interview tip: User-assigned identities let you pre-create role assignments before the resource exists, which avoids deployment ordering problems.

📚 Official docs

- Azure: Managed identities

Q123 Explain Azure Policy effects such as Audit, Deny, Modify and DeployIfNotExists, and how you roll out a new policy safely.

IAM & EntraID

🎙 Spoken answer · ~56 s

Azure Policy evaluates resources against rules at creation and continuously afterwards. The effect decides the outcome. Audit records a non-compliant event without blocking. Deny rejects the request before the resource is created or changed. Modify can add, replace or remove properties such as tags or settings, using a managed identity. DeployIfNotExists and AuditIfNotExists look for a related resource, for example diagnostic settings or an agent, and deploy it or report it missing. To roll out safely, I assign the policy in Audit or with enforcement mode set to DoNotEnforce, review compliance results, communicate with teams, add exemptions where justified, run remediation tasks for existing resources, and only then switch to Deny. I group related policies into initiatives, assign them at management group level, and manage them as code with testing in a sandbox subscription.

Likely follow-ups

- How do exemptions work?
- Why do remediation tasks need a managed identity?

Audit flags non-compliance only. Deny blocks the request. Modify adds or changes properties like tags. DeployIfNotExists deploys a related resource, such as diagnostic settings, when missing. Assignments can be set to DoNotEnforce first.

```
az policy assignment create --name require-tags --policy <def-id> --scope
/subscriptions/<sub> --enforcement-mode DoNotEnforce
```

```
az policy state list --filter "complianceState eq 'NonCompliant'" -o table
```

```
az policy remediation create -n fix-diag --policy-assignment require-diag
```

Interview tip: Start with Audit or DoNotEnforce, read the compliance data, fix existing resources, then switch to Deny.

📚 Official docs

- Azure Policy

certs.eknathalabs.com

Q124 How do NSGs, application security groups and service tags work together, and how are rules evaluated?

Networking

🎙 Spoken answer · ~59 s

A network security group contains inbound and outbound rules, each with a priority from one hundred to four thousand ninety-six. Azure checks rules from the lowest number upward and stops at the first match. There are default rules at the end that allow VNet and load balancer traffic and deny the rest. NSGs can be attached to a subnet, a network interface, or both, in which case both must allow the traffic. Service tags like Storage, Sql or AzureLoadBalancer stand for Microsoft-managed address ranges that update automatically, so rules do not break when IPs change. Application security groups let me group network interfaces by role, for example web, app and db, and write rules between groups rather than address lists. For troubleshooting I use IP flow verify and effective security rules in Network Watcher, and NSG flow logs for traffic analysis.

Likely follow-ups

- What are the default NSG rules?
- How does IP flow verify help?

🗺 NSG evaluation



An NSG holds numbered allow and deny rules for a subnet or NIC; the lowest priority number that matches wins. Service tags represent Azure service IP ranges, and application security groups let you write rules by workload role instead of IP address.

```
az network nsg rule create -g net-rg --nsg-name web-nsg -n allow-https --priority 100 --
access Allow --protocol Tcp --destination-port-ranges 443 --source-address-prefixes
Internet
```

```
az network watcher show-security-group-view --vm vm1 -g app-rg
```

```
az network nic list-effective-nsg -g app-rg --name nic1
```

Interview tip: Effective security rules and IP flow verify in Network Watcher tell you exactly which rule allowed or blocked a flow.

📚 Official docs

- Azure Network security groups

certs.eknathalabs.com

Q125 What do Azure Bastion and just-in-time VM access provide compared with exposing RDP or SSH?

Networking

🎙 Spoken answer · ~61 s

Exposing RDP or SSH to the internet invites constant scanning and brute force attacks. Azure Bastion is a managed PaaS service deployed in a dedicated subnet. Users connect through the Azure portal or a native client over TLS, and Bastion reaches the VM over its private address, so the VM needs no public IP. With the Standard SKU I get native client support, shareable links and Entra ID sign-in, and I can combine it with Conditional Access and recorded sessions through logging. Just-in-time access from Defender for Cloud takes a different approach: management ports stay closed in the NSG, and when someone requests access, a rule is created for their source IP for a limited time, with the request logged and approved through RBAC. I often use Bastion for access and JIT or PIM for authorisation, and for automation I prefer agent-based tools like Run Command.

Likely follow-ups

- Why does Bastion need its own subnet?
- How is JIT approval logged?

Bastion is a managed jump service that gives browser or native-client RDP and SSH over TLS to VMs with only private IPs. Just-in-time access in Defender for Cloud opens management ports on an NSG only for a requested time and source.

```
az network bastion create -g net-rg -n hub-bastion --vnet-name hub --public-ip-address bastion-pip --sku Standard
```

```
az network bastion ssh --name hub-bastion -g net-rg --target-resource-id <vm-id> --auth-type AAD
```

Interview tip: The aim is no public IPs and no standing open management ports on any VM.

📚 Start here (official docs)

- Microsoft Azure documentation

certs.eknathalabs.com

Q126 Azure Virtual WAN versus a traditional hub-and-spoke network: when do you pick each?

Networking

🎙 Spoken answer · ~56 s

In a hub-and-spoke design I build the hub VNet myself with Azure Firewall or a network virtual appliance, VPN and ExpressRoute gateways, and peer each spoke VNet to it, using user-defined routes to force traffic through the firewall. It gives the most control and is cheaper for small estates, but the routing tables, peerings and gateway transit settings become a lot to maintain, especially across regions. Virtual WAN provides a Microsoft-managed hub, called a virtual hub, with built-in routing, automatic any-to-any connectivity between branches and VNets, and integrated VPN, ExpressRoute and firewall options. It reduces manual route management and scales to hundreds of branches, but costs more and has some restrictions on customisation. I pick Virtual WAN for many sites and regions, and traditional hub-and-spoke when requirements are specific or the network is modest.

Likely follow-ups

- What is a secured virtual hub?
- How do you do inspection between spokes?

Hub-and-spoke is a self-managed pattern: a VNet hub with firewall, gateways and peering to spokes. Virtual WAN is a managed hub service that automates routing, branch connectivity and transitive connections at scale, with less manual route work and a higher service cost.

```
az network vwan create -g net-rg -n corp-wan --type Standard
```

```
az network vhub create -g net-rg -n hub-we --vwan corp-wan --address-prefix 10.100.0.0/23 -
l westeurope
```

```
az network vhub connection create -g net-rg --vhub-name hub-we -n spoke1 --remote-vnet
spoke1-vnet
```

Interview tip: A fair answer: choose hub-and-spoke for control and a handful of regions, Virtual WAN when you have many branches and regions.

📚 Official docs

- Azure Virtual WAN
- Azure hub-spoke network topology

certs.eknathalabs.com

Q127 Azure Front Door, Traffic Manager and Application Gateway: how do you choose for global versus regional traffic?

Networking

🎙 Spoken answer · ~55 s

Traffic Manager works at DNS level. It answers queries with the endpoint chosen by priority, weighted, performance or geographic rules, and never sees the traffic, so failover depends on DNS caching and it works for any protocol. Front Door is a global HTTP and HTTPS entry point on Microsoft's edge network, offering anycast, TLS termination, caching, WAF, URL routing and quick health-based failover. Application Gateway is regional, lives inside your VNet, and offers WAF, path routing, session affinity and AKS ingress integration. A common pattern for multi-region applications is Front Door at the edge, with an Application Gateway or internal load balancer per region, and private origins using Private Link. I choose Traffic Manager for non-HTTP workloads or simple DNS routing, and add header validation so users cannot bypass the edge security.

Likely follow-ups

- Why can DNS failover be slow?
- How do you protect origins from direct access?

Traffic Manager is DNS-based routing with no data path. Front Door is a global Layer 7 edge with CDN, WAF and fast failover. Application Gateway is a regional Layer 7 load balancer with WAF. Combine Front Door in front of regional gateways.

```
az afd profile create -g edge-rg --profile-name global --sku Premium_AzureFrontDoor
```

```
az network traffic-manager profile create -g edge-rg -n tm --routing-method Performance --unique-dns-name myapp-tm
```

Interview tip: Remember to lock backends to accept traffic only from Front Door, using its service tag and the X-Azure-FDID header.

📚 Official docs

- Azure Application Gateway
- Azure Front Door
- Azure Traffic Manager

certs.eknathalabs.com

Q128 System versus user node pools on AKS: why separate them and how do you configure each?

Containers & AKS

🎙 Spoken answer · ~55 s

AKS supports two node pool modes. System pools host the essential components of the cluster, for instance CoreDNS, konnectivity and metrics-server, and every cluster needs at least one. User pools are where application pods run, and they can scale to zero. I separate them so a runaway application cannot starve cluster components. I apply the CriticalAddonsOnly taint to the system pool so only tolerated pods land there. System pools are sized modestly but kept highly available, with three nodes spread across availability zones, and use a general-purpose VM size. User pools are sized for the workload and I often create several: different VM families for memory-heavy or GPU workloads, Spot pools for tolerant jobs, separate pools per tenant or compliance boundary, each with its own autoscaler settings and upgrade surge configuration.

Likely follow-ups

- Why can user pools scale to zero but system pools cannot?
- How do you upgrade pools independently?

System pools host critical add-ons like CoreDNS and metrics-server and must have at least one node. User pools run application workloads. Separating them with a taint on the system pool prevents apps from starving cluster components.

```
az aks nodepool add -g aks-rg --cluster-name prod -n apps --mode User --node-count 3 --
zones 1 2 3
```

```
az aks nodepool update -g aks-rg --cluster-name prod -n system --node-taints
CriticalAddonsOnly=true:NoSchedule
```

```
az aks nodepool list -g aks-rg --cluster-name prod -o table
```

Interview tip: Run at least three system nodes across zones in production, and keep system pools on a stable VM size.

📚 Official docs

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q129 How do AKS upgrade channels and node OS image upgrades work, and how do you keep clusters patched safely?

Containers & AKS

🎙 Spoken answer · ~58 s

AKS has two independent upgrade dimensions. The cluster auto-upgrade channel controls the Kubernetes version: patch keeps you on the latest patch of your minor version, stable follows the latest supported minor minus one, rapid takes the newest, and node-image only updates node images. The node OS upgrade channel controls the operating system: Unmanaged leaves patching to you, SecurityPatch applies security updates without reimaging, NodeImage replaces nodes with a fresh image weekly. I configure maintenance windows so changes happen at quiet times, set the max surge on node pools to control speed, and make sure PodDisruptionBudgets and multiple replicas protect workloads during drains. Lower environments use faster channels so problems appear there first. I also monitor for deprecated APIs and keep track of the version support window, which is limited. Skipping upgrades is risky because older versions fall out of support.

Likely follow-ups

- What does max surge control?
- What breaks an AKS upgrade most often?

Cluster auto-upgrade channels (patch, stable, rapid, node-image) move the Kubernetes version automatically. Node OS upgrade channels apply security patches or new images. Planned maintenance windows and surge settings control when and how fast.

```
az aks update -g aks-rg -n prod --auto-upgrade-channel patch --node-os-upgrade-channel NodeImage
```

```
az aks maintenanceconfiguration add -g aks-rg --cluster-name prod -n aksManagedAutoUpgradeSchedule --schedule-type Weekly --day-of-week Sunday --start-hour 2 --duration 4
```

```
az aks nodepool update -g aks-rg --cluster-name prod -n apps --max-surge 33%
```

Interview tip: Roll non-production first with a faster channel so production follows a version that has already survived testing.

📚 Official docs

- Kubernetes: Upgrading kubeadm clusters
- AKS: Upgrade a cluster
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q130 How does Azure Container Registry support geo-replication, ACR Tasks and secure image pulls from AKS?

Containers & AKS

🎙 Spoken answer · ~58 s

Azure Container Registry stores OCI images and Helm charts. With the Premium SKU I can enable geo-replication, so one registry name serves images from replicas in several regions, which speeds up pulls for regional clusters, reduces data transfer and keeps working if a region has trouble. ACR Tasks run builds, tests and base image updates inside Azure, triggered by commits or base image changes, which helps keep images patched. For authentication, AKS clusters use the kubelet's managed identity, which is granted the AcrPull role on the registry, set with the attach-acr option, so no pull secrets are stored. To lock it down I use private endpoints, disable the admin account, restrict public network access, enable vulnerability scanning with Defender, sign images, and use retention policies to delete old tags and untagged manifests so storage costs stay under control.

Likely follow-ups

- Why avoid the ACR admin user?
- How do you automate base image patching?

Premium ACR can replicate a registry to multiple regions behind one login server for faster pulls and resilience. ACR Tasks build and patch images in the cloud. AKS pulls with its kubelet managed identity granted AcrPull, with no passwords.

```
az acr replication create -r myacr -l northeurope
```

```
az aks update -g aks-rg -n prod --attach-acr myacr
```

```
az acr task create -r myacr -n build-web --image web:{{.Run.ID}} --context https://<git-url> --file Dockerfile --commit-trigger-enabled false
```

Interview tip: Turn on content trust or use Notation signing and Defender for Containers scanning to complete the supply chain story.

📚 Official docs

- Kubernetes: Images (pull policy, registries)
- Azure Container Registry
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q131 What is node auto-provisioning (Karpenter) on AKS, and how is it different from the Cluster Autoscaler?

Containers & AKS

🎙 Spoken answer · ~59 s

The Cluster Autoscaler works on node pools that you define in advance. Each pool uses a fixed VM size, so to support different pod shapes you create many pools, and scale-up picks a pool rather than the best size. Node auto-provisioning is AKS's managed Karpenter. You declare constraints in NodePool and node class resources, such as allowed VM families, architectures, zones, Spot or on-demand and CPU and memory limits. When pods are pending, it chooses the cheapest suitable VM size that fits them, launches it directly, and later consolidates by replacing underused nodes with fewer or smaller ones. This often improves bin packing and reduces cost, and removes the need to manage many pools. Considerations are setting limits, using disruption budgets to control consolidation, keeping a small system pool, and understanding that some features of classic pools may not be available.

Likely follow-ups

- How does consolidation avoid disrupting workloads?
- What limits would you set on a NodePool?

The Cluster Autoscaler scales pre-defined node pools, each with one VM size. Node auto-provisioning, built on Karpenter, picks VM sizes dynamically for pending pods and consolidates nodes, reducing pool sprawl and wasted capacity.

```
az aks update -g aks-rg -n prod --node-provisioning-mode Auto
```

```
kubectl get nodepools.karpenter.sh
```

```
kubectl get nodeclaims
```

Interview tip: Define limits on each NodePool so a runaway deployment cannot provision unlimited nodes.

📚 Official docs

- Kubernetes Autoscaler: Cluster Autoscaler
- AKS: Cluster autoscaler
- Karpenter documentation

certs.eknathalabs.com

Q132 Explain Blob access tiers, lifecycle management and immutability for compliance data.

Storage

🎙 Spoken answer · ~59 s

Storage tiers let me match cost to access patterns. Hot has higher storage cost and lowest access cost, Cool and Cold are cheaper to store but charge more to read and have minimum retention days, and Archive is offline, cheapest to store, but needs rehydration that can take hours. Lifecycle management policies automate transitions based on last modified or last accessed time and can delete after a retention period, so I define them as code per container prefix. For compliance data such as audit logs or financial records, immutable storage provides write once read many behaviour. A time-based retention policy prevents modification or deletion until the period ends, and a legal hold blocks changes until removed. A locked policy cannot be shortened, so I test with an unlocked policy first. I also enable versioning, soft delete and geo-redundancy for important data.

Likely follow-ups

- What is the early deletion charge?
- How does rehydration from Archive work?

Hot, Cool, Cold and Archive trade lower storage cost for higher access cost and retrieval delay. Lifecycle rules move or delete blobs by age. Immutable storage with legal hold or time-based retention makes blobs WORM, so they cannot be altered or deleted.

```
az storage account management-policy create --account-name stlogs -g data-rg --policy @lifecycle.json
```

```
az storage container immutability-policy create --account-name stlogs -c audit --period 365
```

```
az storage blob set-tier --account-name stlogs -c logs -n 2026/10/a.log --tier Archive
```

Interview tip: Archive blobs need rehydration, which can take hours, so never put data there that incident response might need quickly.

📚 Official docs

- Azure Blob lifecycle management

certs.eknathalabs.com

Q133 Azure Files versus Azure NetApp Files versus Blob for shared storage on AKS and VMs.

Storage

🎙 Spoken answer · ~60 s

Azure Files provides fully managed file shares over SMB and NFS, integrates with the Azure Files CSI driver for ReadWriteMany volumes on AKS, supports snapshots and backup, and has standard and premium performance. It is the right default for shared configuration, content or moderate workloads. Azure NetApp Files is built on NetApp hardware, giving very low latency, high throughput and fine-grained performance tiers, which suits databases like SAP HANA, HPC and heavy shared workloads, at a higher price and with capacity pool minimums. Blob storage is object storage with a flat namespace, ideal for data lakes, media and backups. It can be mounted with blobfuse or the NFS feature, but semantics and latency differ from a true file system, so I avoid it for applications needing POSIX behaviour or many small writes. For single-pod block storage with the best IOPS, I use Azure Disk.

Likely follow-ups

- Why can't Azure Disk be ReadWriteMany for normal use?
- When is premium Files worth it?

Azure Files offers managed SMB and NFS shares and is the easy default. NetApp Files delivers high-performance, low-latency NFS and SMB with enterprise features at higher cost. Blob is object storage, mounted via blobfuse for read-heavy use but not a general POSIX filesystem.

```
kubectl get storageclass
```

```
kubectl apply -f pvc-azurefile.yaml  # storageClassName: azurefile-csi-premium
```

```
az netappfiles volume list -g data-rg --account-name anf --pool-name pool1 -o table
```

Interview tip: Say that RWX volumes on AKS are typically Azure Files or NetApp Files, while Azure Disk is single-attach RWO.

📚 Official docs

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q134 Shared access signatures versus Entra ID authorisation for Azure Storage: which should you use?

Storage

🎙 Spoken answer · ~59 s

For applications and people inside my tenant, I use Entra ID authentication with Azure RBAC data roles such as Storage Blob Data Reader. It uses managed identities, produces identity-based logs, supports Conditional Access and needs no stored secrets. A shared access signature is a URL with a token that grants specific permissions on specific resources for a limited time, which is ideal for giving an external client or a browser upload access without an Azure identity. There are three types: account SAS and service SAS, signed with the storage account key, and user delegation SAS, signed with Entra credentials, which is more secure and what I prefer. Account keys give full control and cannot be selectively revoked, so for production I disable shared key access, rotate keys if exposed, keep SAS expiry short, restrict by IP and protocol, and monitor access logs.

Likely follow-ups

- How do you revoke an issued SAS?
- What is a user delegation key?

Entra ID with RBAC is the preferred method: identity-based, auditable, revocable, no secrets. SAS tokens are bearer URLs for delegated, time-limited access. User delegation SAS is signed with Entra credentials and is safer than account-key SAS. Disable shared key access where possible.

```
az storage blob generate-sas --account-name stapp -c files -n report.pdf --permissions r --expiry 2026-10-21T18:00Z --as-user --auth-mode login
```

```
az storage account update -n stapp -g data-rg --allow-shared-key-access false
```

```
az role assignment create --assignee <id> --role 'Storage Blob Data Contributor' --scope <storage-id>
```

Interview tip: Account-key SAS cannot be revoked individually, so rotating the key is the only emergency stop. Use stored access policies or user delegation SAS.

📚 Official docs

- Microsoft Entra ID: What is Microsoft Entra ID?
- Azure Storage introduction
- Azure Storage: Shared access signatures

certs.eknathalabs.com

Q135 Explain the five Cosmos DB consistency levels and how you choose between them.

Databases

🎙 Spoken answer · ~58 s

Cosmos DB offers five levels along a spectrum. Strong gives linearizable reads, always seeing the latest committed write, but with higher latency and reduced availability across regions. Bounded Staleness guarantees reads lag behind writes by at most a set number of versions or a time interval, giving predictable ordering with global distribution. Session, the default, guarantees that within a client session you read your own writes and see monotonic reads, which satisfies most application needs at low cost. Consistent Prefix ensures reads never see out-of-order writes but may be stale. Eventual gives the lowest latency and highest availability, with no ordering guarantee. I start with Session, move to Bounded Staleness when a defined freshness matters across regions, and use Strong only for cases like financial balances where the latency is acceptable. Clients can also relax the account level per request.

Likely follow-ups

- What is a session token?
- Why does Strong cost more RUs?

Strong, Bounded Staleness, Session, Consistent Prefix and Eventual trade consistency for latency, availability and throughput. Session is the default and a good fit for most apps because each client reads its own writes.

```
az cosmosdb update -g data-rg -n cosmos-prod --default-consistency-level Session
```

```
az cosmosdb show -g data-rg -n cosmos-prod --query consistencyPolicy
```

Interview tip: Strong consistency is not available across multiple write regions, and it costs more RUs for reads. Know both caveats.

📚 Official docs

- Azure Cosmos DB

certs.eknathalabs.com

Q136 Azure SQL elastic pools, serverless and Hyperscale: when does each fit?

Databases

🎙 **Spoken answer · ~62 s**

These solve different problems. An elastic pool gives a shared pool of CPU and storage to many databases whose peaks do not coincide, such as a database per tenant in a SaaS product, so I pay for the pool rather than for the sum of individual peaks. Serverless is a compute tier for single databases that automatically scales between minimum and maximum vCores based on load, bills per second, and can pause after an idle delay, so it is good for development, test or sporadic workloads, though resume takes time. Hyperscale uses a distributed architecture with page servers and log service, allowing storage up to a hundred terabytes, fast scale up and out, near instant backups and quick restores regardless of size, plus multiple named read replicas. I choose Hyperscale for large or fast-growing databases, pools for multi-tenant estates, and General Purpose or Business Critical for steady OLTP.

Likely follow-ups

- What happens during auto-pause resume?
- How do read replicas work in Hyperscale?

Elastic pools share resources across many databases with uneven usage. Serverless auto-scales compute and can pause when idle, billing per second. Hyperscale separates compute and storage for very large databases up to 100 TB with fast scale and restores.

```
az sql elastic-pool create -g data-rg -s sql-prod -n pool1 --edition GeneralPurpose --
family Gen5 --capacity 8
```

```
az sql db create -g data-rg -s sql-prod -n reports --edition GeneralPurpose --compute-model
Serverless --auto-pause-delay 60
```

```
az sql db update -g data-rg -s sql-prod -n big --service-objective HS_Gen5_8
```

Interview tip: Serverless suits dev, test and intermittent workloads. The first query after auto-pause is slow because the database must resume.

📚 **Official docs**

- AWS Lambda Developer Guide
- Azure SQL documentation

certs.eknathalabs.com

Q137 How does high availability work for Azure Database for PostgreSQL Flexible Server, and what does a failover look like?

Databases

🎙 **Spoken answer · ~55 s**

With high availability enabled on a Flexible Server, Azure maintains a standby replica and replicates changes synchronously, so a committed transaction exists on both. In zone-redundant mode, the standby is in a different availability zone, protecting against zone failure, and in same-zone mode it is in the same zone, protecting against host or server failure. If the primary becomes unhealthy, or during some maintenance events, the service fails over automatically by promoting the standby and updating DNS, so the connection string stays the same. Applications see dropped connections for roughly a minute or two, so they must retry with backoff and re-resolve DNS. HA does not protect against human error or corruption, so I also rely on automated backups with point-in-time restore, and use read replicas for read scaling and cross-region disaster recovery.

Likely follow-ups

- HA versus read replicas?
- How do you handle the brief connection loss?

Zone-redundant HA keeps a standby in another availability zone with synchronous replication, and automatic failover promotes it, usually within one to two minutes, keeping the same connection string. Same-zone HA protects against server failure only.

```
az postgres flexible-server update -g data-rg -n pg-prod --high-availability ZoneRedundant
--standby-zone 2
```

```
az postgres flexible-server restart -g data-rg -n pg-prod --failover Forced
```

```
az postgres flexible-server show -g data-rg -n pg-prod --query highAvailability
```

Interview tip: Test with a forced failover in a non-production server so your connection retry logic is proven.

📚 **Start here (official docs)**

- Microsoft Azure documentation

certs.eknathalabs.com

Q138 How do Application Insights sampling and distributed tracing work, and how do you control telemetry cost?

Monitoring

🎙 Spoken answer · ~58 s

Application Insights instruments an application with an SDK or auto-instrumentation and sends requests, dependency calls, exceptions, custom events and traces to a Log Analytics workspace. Each item carries an operation ID, so the end-to-end transaction view can show a request flowing through multiple services, using the W3C trace context headers to propagate identity across HTTP calls and messages. Since volume drives cost, sampling is the main control. Adaptive sampling automatically reduces the rate when traffic is high, and it keeps related telemetry for the same operation together, with the sampling percentage stored so metrics can be adjusted. I can also use fixed rate or ingestion sampling, filter out noisy health check requests, limit custom dimensions, set a daily cap as a safety valve, and send high-volume logs to cheaper table plans. I align this with OpenTelemetry where possible for portability.

Likely follow-ups

- Why is adaptive sampling better than random drop?
- How do you correlate logs with traces?

Application Insights collects requests, dependencies, exceptions and traces, correlated by operation ID. Adaptive sampling keeps a percentage of telemetry, preserving related items together, which cuts ingestion while still giving accurate counts. Distributed tracing uses W3C trace context.

```
requests | where success == false | summarize count() by resultCode, bin(timestamp, 5m)
```

```
dependencies | where duration > 2000 | summarize avg(duration) by target
```

```
az monitor app-insights component show -g mon-rg -a web-ai --query samplingPercentage
```

Interview tip: Sample requests and dependencies, but never sample away exceptions you rely on for alerting without checking the config.

📚 Official docs

- Azure Monitor
- OpenTelemetry: Signals (traces, metrics, logs)
- OpenTelemetry: Sampling

certs.eknathalabs.com

Q139 How do you design a Log Analytics workspace strategy: one or many, retention, and table plans?

Monitoring

🎙 Spoken answer · ~60 s

I prefer few workspaces, because cross-workspace queries are more complex and alerts and dashboards are easier in one place. I split only when needed: data residency by region, strict separation of security and operations data, different retention or ownership, or billing needs. Within a workspace, access can be managed using resource-context RBAC, so a team that has access to its own resources sees only their logs, and table-level RBAC for sensitive tables. For cost I set retention per table, with interactive retention for recent data and long-term retention or archive for compliance, choose the Basic or Auxiliary plans for verbose, rarely queried logs such as container stdout, use data collection rules to filter and transform before ingestion, use commitment tiers when daily volume is large, and export to storage for cheap long-term archive. I monitor ingestion per table and set alerts on spikes.

Likely follow-ups

- What is resource-context access?
- What are data collection rules?

Start with a small number of workspaces split by region, compliance and access boundaries, not by team. Set retention per table, use Basic or Auxiliary table plans for high-volume low-value logs, and use data collection rules to filter before ingestion.

```
az monitor log-analytics workspace create -g mon-rg -n law-prod --retention-time 90
```

```
az monitor log-analytics workspace table update -g mon-rg --workspace-name law-prod -n
ContainerLogV2 --plan Basic
```

```
az monitor log-analytics workspace data-export create -g mon-rg --workspace-name law-prod -
n archive --tables AuditLogs --destination <storage-id>
```

Interview tip: Table-level RBAC and resource-context access let teams see only their own logs inside a shared workspace.

📚 Official docs

- Azure Monitor
- Prometheus: Storage (TSDB)
- Google SRE Book: Distributed tracing & logs (Monitoring chapter)

certs.eknathalabs.com

Q140 Metric alerts, log search alerts and dynamic thresholds in Azure Monitor: when do you use each?

Monitoring

🎙 Spoken answer · ~58 s

Metric alerts evaluate numeric time series, such as CPU or request latency, as often as every minute, support multiple dimensions, and are inexpensive and fast, so I use them for core health signals. Log search alerts run a KQL query against Log Analytics or Application Insights on a schedule, which allows complex conditions like error rates by customer, absence of heartbeats, or correlations across tables, but they have more latency and cost per evaluation. Dynamic thresholds apply machine learning to a metric's history, accounting for daily and weekly patterns, and alert on deviations, which suits metrics without a fixed good value. Alerts fire to action groups that notify people, call webhooks, run automation or open tickets. I scope them with tags, use severity levels, add alert processing rules for maintenance windows, and prefer symptom-based alerts over cause-based ones.

Likely follow-ups

- What do action groups contain?
- How would you alert on a missing heartbeat?

Metric alerts are near real-time and cheap, evaluated on platform or custom metrics. Log search alerts run a KQL query on a schedule and handle complex conditions. Dynamic thresholds learn normal behaviour so you avoid hand-tuned numbers.

```
az monitor metrics alert create -n high-cpu -g mon-rg --scopes <vm-id> --condition 'avg
Percentage CPU > 85' --window-size 5m --evaluation-frequency 1m --action <action-group-id>
```

```
az monitor scheduled-query create -n errors -g mon-rg --scopes <law-id> --condition 'count
> 10' --condition-query 'AppExceptions | where TimeGenerated > ago(5m)'
```

Interview tip: Route alerts through action groups, and use alert processing rules to suppress them during planned maintenance.

📚 Official docs

- Azure Monitor

certs.eknathalabs.com

Q141 How do Azure DevOps service connections use workload identity federation instead of secrets?

CI/CD

🎙 Spoken answer · ~55 s

Traditionally, service connections used a service principal with a client secret that was stored in Azure DevOps, expired every year or two, and could leak. With workload identity federation, I create an app registration or a user-assigned managed identity and add a federated credential that trusts Azure DevOps as an issuer for a specific organization, project and service connection subject. When a pipeline runs, Azure DevOps issues a signed token for that job, which Entra ID exchanges for a short-lived access token, so there is nothing to store, rotate or steal. I grant the identity the minimum RBAC role at the narrowest scope, use separate connections per environment, require approvals and checks on production, and restrict which pipelines can use each connection. The same pattern applies to GitHub Actions with OIDC.

Likely follow-ups

- What is in the subject claim?
- How do you restrict which pipelines use the connection?

A service connection can use an app registration or managed identity with a federated credential that trusts the Azure DevOps issuer and subject. Pipelines get short-lived tokens, so no client secret is stored or needs rotation.

```
az ad app federated-credential create --id <app-id> --parameters @fedcred.json
```

```
az role assignment create --assignee <app-id> --role Contributor --scope
/subscriptions/<sub>/resourceGroups/app-rg
```

```
az devops service-endpoint list -o table
```

Interview tip: Scope the identity's role to a resource group per environment, and use environment approvals for production service connections.

📚 Official docs

- Azure: Managed identities
- Google Cloud: Workload Identity Federation
- Prometheus: Federation

certs.eknathalabs.com

Q142 How do App Service deployment slots enable zero-downtime releases and fast rollback?

CI/CD

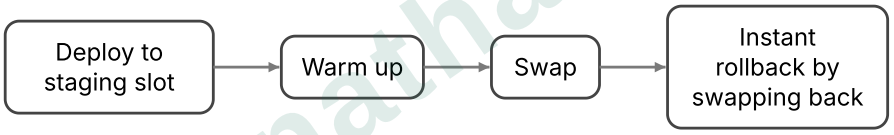
🎙 Spoken answer · ~60 s

Deployment slots are full app instances that share the same App Service plan, each with its own URL and configuration. I deploy a new build to a staging slot, run smoke tests there, and let the swap operation warm up the instances by sending requests to the configured paths. Then the swap exchanges the routing so that the warmed instances take production traffic, with no cold start or downtime. If problems appear, swapping again restores the previous version immediately, because the old code is still running in the other slot. Settings can be marked as slot-specific so that, for instance, production keeps its own database connection string. I can also route a small percentage of traffic to a slot for canary testing. Considerations are that slots share plan resources, background jobs may run in both slots, and database changes need to be backward compatible.

Likely follow-ups

- Which settings should be sticky?
- How does swap with preview work?

🗺 Slot swap



A slot is a live app with its own hostname and settings. You deploy to staging, warm it up, then swap, which redirects production traffic to the already running instance. Swapping back is an instant rollback. Slot-sticky settings stay with the slot.

```
az webapp deployment slot create -g app-rg -n web --slot staging
```

```
az webapp deploy -g app-rg -n web --slot staging --src-path app.zip
```

```
az webapp deployment slot swap -g app-rg -n web --slot staging --target-slot production
```

Interview tip: Use swap with preview or auto-swap warm-up to avoid sending users to a cold instance, and mark connection strings that differ as slot settings.

📚 Official docs

- Kubernetes: Deployments

certs.eknathalabs.com

Q143 How do Azure DevOps environments, approvals and checks protect production deployments?

CI/CD

🎙 Spoken answer · ~58 s

In Azure Pipelines, a deployment job targets an environment, such as staging or production. Protections are configured on the environment resource itself, not in the pipeline file. I add manual approvals from a specific group, with a timeout, branch control so only the main branch can deploy, an exclusive lock so one deployment runs at a time, invoke Azure Function or REST API checks to validate change tickets or monitoring status, and business-hours restrictions. I also apply checks on the service connection and variable groups, so a pipeline cannot reach production secrets without passing them. Because the checks sit outside of YAML, a developer editing the pipeline cannot bypass them. Combined with required reviewers on pull requests and artifact promotion rather than rebuilding, this gives an auditable path to production, and deployment history on the environment helps with traceability.

Likely follow-ups

- Why not put approvals only in YAML?
- What is an exclusive lock?

An environment represents a deployment target. Approvals, branch controls, exclusive locks and business-hour checks can be attached to it, so a pipeline stage targeting that environment pauses until all checks pass, regardless of who edits the YAML.

```
az devops invoke --area distributedtask --resource environments --route-parameters
project=app --api-version 7.1
```

```
az pipelines run --name deploy-prod --branch main
```

Interview tip: Put checks on the environment and service connection, not only in pipeline YAML, because anyone who can edit YAML can remove YAML-based gates.

📚 Official docs

- GitHub Actions: Environments & deployment protection
- Azure DevOps documentation

certs.eknathalabs.com

Q144 What are VM Scale Set orchestration modes, and how do Flexible and Uniform differ?

Compute

🎙 Spoken answer · ~59 s

A virtual machine scale set manages a group of VMs for scaling and availability. In Uniform orchestration, all instances are created from one scale set model, are treated as identical, and are managed through the scale set API, which is simple for stateless fleets. Flexible orchestration is the newer model: VMs behave more like regular Azure VMs that happen to be in a group, you can mix sizes and Spot with on-demand through instance mix, spread them across zones and fault domains with a fault domain count of your choice, and use standard VM APIs, networking and extensions. Flexible also integrates with Azure Compute Gallery images and works well for stateful or heterogeneous applications. Autoscale rules based on metrics or schedules adjust capacity, with cool-down periods to avoid flapping. I choose Flexible for new deployments unless a feature depends on Uniform.

Likely follow-ups

- How do instance mixing and Spot work together?
- What are fault domains?

Flexible orchestration lets you mix VM sizes and manage VMs like standalone VMs inside a scale set, with spread across zones and fault domains. Uniform manages identical instances from one model, with simpler autoscaling. Flexible is now recommended for new designs.

```
az vmss create -g app-rg -n web-vmss --orchestration-mode Flexible --image Ubuntu2204 --
zones 1 2 3 --instance-count 3
```

```
az monitor autoscale create -g app-rg --resource web-vmss --resource-type
Microsoft.Compute/virtualMachineScaleSets --min-count 3 --max-count 12 --count 3
```

Interview tip: Mention that AKS node pools are VM scale sets under the hood, so scale set concepts explain AKS behaviour.

📚 Official docs

- Azure Virtual Machine Scale Sets

certs.eknathalabs.com

Q145 App Service, Azure Container Apps and AKS: how do you decide between them for a web API?

Compute

🎙 Spoken answer · ~58 s

I decide based on how much control I need versus how much operational work I accept. App Service is the easiest: deploy code or a container, with built-in TLS, slots, autoscale and authentication, and it fits conventional web apps and APIs. Container Apps runs containers on a managed Kubernetes-based platform without exposing it, with scale to zero, event-driven scaling through KEDA, revisions with traffic splitting, and Dapr for microservice patterns, which suits event-driven and microservice workloads of moderate complexity. AKS provides the full Kubernetes API, custom controllers and operators, service meshes, advanced scheduling and networking, and multi-tenant platform building, at the cost of cluster operations and a steeper skill requirement. Team size and skills matter. Most teams start on App Service or Container Apps, and choose AKS when they need Kubernetes-specific capabilities or an internal platform for many teams.

Likely follow-ups

- What can't Container Apps do that AKS can?
- How do revisions help in Container Apps?

App Service is the simplest PaaS for web apps. Container Apps is serverless containers with KEDA scaling and Dapr, no cluster management. AKS gives the full Kubernetes API, custom networking and operators, but you run the platform.

```
az containerapp create -g app-rg -n api --environment cae --image myacr.azurecr.io/api:1.0
--min-replicas 0 --max-replicas 20 --ingress external --target-port 8080
```

```
az webapp up -n web-api --runtime PYTHON:3.12
```

Interview tip: Choose the least complex platform that meets requirements, and migrate up to AKS only when you truly need Kubernetes features.

📚 Official docs

- Azure Container Apps
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q146 How do Azure Hybrid Benefit and reservations stack, and where do they apply? Cost

🎙 Spoken answer · ~61 s

Hybrid Benefit lets organisations with Software Assurance or qualifying subscriptions apply their existing Windows Server, SQL Server, and certain Linux licences to Azure resources, so they pay only for the base compute rate instead of paying for the licence again. It applies to virtual machines, Azure SQL, SQL Managed Instance, AKS Windows nodes and others, and can be turned on with a licence type setting. Reservations and savings plans give a discount on the compute portion in exchange for a one or three year commitment. Since one removes the licence charge and the other lowers the compute rate, they stack, which can cut Windows or SQL workloads by a large margin. The caveats are licence compliance, tracking core counts and eligibility, and committing only to steady baseline capacity. I use Azure Advisor and cost analysis to find candidates, and a policy or script to audit licence settings.

Likely follow-ups

- What is the difference between reservations and savings plans?
- How would you audit licence settings?

Hybrid Benefit lets you use existing Windows Server, SQL Server or Linux subscriptions licences on Azure to remove the licence cost from the price. Reservations or savings plans then discount the compute. The two can be combined for the deepest savings.

```
az vm update -g app-rg -n win1 --license-type Windows_Server
```

```
az sql db update -g data-rg -s sql-prod -n orders --license-type BasePrice
```

```
az consumption reservation summary list --grain monthly --reservation-order-id <id>
```

Interview tip: Check licence compliance and Software Assurance eligibility before enabling the benefit across a fleet.

📚 Official docs

- Azure Reservations

certs.eknathalabs.com

Q147 An AKS node pool fails to scale up with errors about quota or SKU availability. How do you resolve it?

Troubleshooting

🎙 Spoken answer · ~60 s

I start with the exact error, found in the autoscaler logs, the activity log and the scale set events. If it says quota exceeded, either total regional vCPUs or the specific VM family quota is at the limit, so I check usage with the quota commands and request an increase, which can take time, while freeing capacity by removing unused resources. If it says the SKU is not available or allocation failed, the chosen zone or region lacks capacity or the size is restricted for my subscription. Then I try a different size, an additional node pool with another family, other zones, or a different region for new capacity. Long term I avoid depending on a single VM size by allowing several in node auto-provisioning or multiple pools with the autoscaler priority expander, pre-request quota ahead of launches, and monitor quota usage with alerts.

Likely follow-ups

- What is the priority expander?
- How do you plan quota for a launch?

Read the scale set or cluster autoscaler error. QuotaExceeded means regional vCPU or VM-family quota is full, so request an increase. SkuNotAvailable or AllocationFailed means the zone has no capacity, so use another size, zone or region.

```
az vm list-usage -l westeurope -o table | grep -i 'DSv5\|Total Regional'
```

```
az vm list-skus -l westeurope --size Standard_D8s_v5 --zone -o table
```

```
kubectl -n kube-system logs deploy/cluster-autoscaler | grep -i fail
```

Interview tip: Add multiple VM sizes across node pools, or use node auto-provisioning, to reduce dependence on one SKU.

📚 Official docs

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q148 Front Door returns 503 errors or reports all origins unhealthy. How do you troubleshoot?

Troubleshooting

🎙 Spoken answer · ~58 s

First I look at the origin group health in the metrics and the probe settings: path, protocol, interval and the success codes. A probe path that returns a redirect, authentication error or heavy response can mark healthy origins as unhealthy. Then I test the origin directly with curl using the same host header, checking certificate name mismatches, since Front Door validates the certificate against the origin host name. If the origin uses Private Link, I confirm the private endpoint connection was approved. I check network restrictions: the origin must allow the AzureFrontDoor.Backend service tag and validate the X-Azure-FDID header, and WAF or rate limits must not block probe traffic. I also review routing rules and caching, which can serve errors. Once fixed, I reduce probe cost by using a lightweight endpoint, and add alerts on origin health percentage.

Likely follow-ups

- Why do probes come from many IPs?
- What is the X-Azure-FDID header?

Check origin health probe results, probe path, protocol and expected status, then the origin's own availability and access restrictions, private link approvals, host header and certificate name, and whether the backend only accepts the Front Door ID.

```
az afd origin-group show -g edge-rg --profile-name global -n og1 --query
healthProbeSettings
```

```
curl -v -H 'Host: app.contoso.com' https://<origin-fqdn>/health
```

```
az afd origin show -g edge-rg --profile-name global --origin-group-name og1 -n origin1
```

Interview tip: Probes come from many edge IPs, so aggressive WAF or IP restrictions on the origin frequently cause false unhealthy status.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)
- Azure Front Door

certs.eknathalabs.com

Q149 AKS pods get ErrImagePull from Azure Container Registry with an unauthorized error. What do you check?

Troubleshooting

🎙 Spoken answer · ~60 s

The pod events give the real error. For unauthorized or authentication required, the kubelet's managed identity probably does not have the AcrPull role on the registry, which can happen when a cluster is created without attach-acr or when the registry is in another subscription. The az aks check-acr command validates DNS resolution, network reachability and authentication from inside the cluster. If the error is a timeout or name resolution failure, the registry may have a firewall or private endpoint, and the nodes cannot reach it or the private DNS zone is not linked to the node VNet. I also verify the image name, tag and registry login server, since typos are frequent, and whether the tag exists. For images from other registries I use pull secrets. Fixes are granting AcrPull, correcting network rules or DNS, and then deleting the failing pods so they retry.

Likely follow-ups

- What does attach-acr do?
- How do you debug private DNS for ACR?

The kubelet identity probably lacks AcrPull on the registry, the registry is behind a firewall or private endpoint the nodes cannot reach, or the image reference is wrong. Verify with az aks check-acr and the pod events.

```
az aks check-acr -g aks-rg -n prod --acr myacr.azurecr.io
```

```
az role assignment list --assignee $(az aks show -g aks-rg -n prod --query
identityProfile.kubeletidentity.objectId -o tsv) --scope <acr-id>
```

```
kubectl describe pod <p> | tail -15
```

Interview tip: After switching to a private registry endpoint, make sure the private DNS zone is linked to the cluster's VNet.

📚 Official docs

- Kubernetes: Debug Pods
- Kubernetes: Pull an image from a private registry
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q150 kubectl cannot reach a private AKS cluster, or stops working after enabling API server authorised IP ranges. How do you fix access?

Troubleshooting

🎙 **Spoken answer · ~61 s**

With a private cluster the API server has no public endpoint, so my client must be in the VNet, in a peered VNet, on a VPN or ExpressRoute connection, or on a jump host or build agent inside the network, and it must resolve the private DNS zone for the API server name. If DNS fails, the zone is probably not linked to my network or custom DNS servers do not forward the privatelink zone. For public clusters with authorised IP ranges, requests from outside the list are rejected, so I add the current egress IPs of engineers and CI systems, remembering that NAT gateways or proxies change the visible address. As a workaround I can run commands through az aks command invoke, which executes inside the cluster via the Azure control plane. For CI, I use self-hosted agents inside the network, and document the connectivity design.

Likely follow-ups

- How does az aks command invoke work?
- What DNS zone does a private AKS cluster use?

A private cluster exposes the API server only on a private IP, so you must connect from the VNet, a peered network, VPN, or use az aks command invoke. For authorised ranges, add your egress IP and mind NAT changes.

```
az aks command invoke -g aks-rg -n prod --command 'kubectl get nodes'
```

```
az aks update -g aks-rg -n prod --api-server-authorized-ip-ranges
203.0.113.10/32,198.51.100.0/24
```

```
nslookup prod-dns-abc.privatelink.westeurope.azmk8s.io
```

Interview tip: Command invoke is the escape hatch when you lock yourself out, but only if you still have RBAC rights on the cluster resource.

📚 **Official docs**

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q151 A pod cannot mount an Azure Disk because of a zone mismatch or attach error. How do you diagnose it? Troubleshooting

🎙 Spoken answer · ~63 s

I read the events on the pod, which typically show a multi-attach error, a failed attach, or a message that the volume has node affinity conflicts. For zone problems, I compare the zone in the persistent volume's node affinity with the zones of available nodes. If all nodes in that zone are full or have been scaled away, the pod cannot be scheduled elsewhere. Solutions include ensuring there are nodes in each zone with enough capacity, using the WaitForFirstConsumer volume binding mode so the disk is created where the pod lands, and using ZRS disks, which can attach across zones. For attach errors after node failure, the disk may still be listed as attached to the old VM, and I check the disk's managedBy property and the VMSS instance state, and wait for detachment or detach it manually once the node is confirmed gone. Prevention involves PodDisruptionBudgets, node health alerts and backups.

Likely follow-ups

- What does ZRS on a managed disk do?
- Why use WaitForFirstConsumer?

Azure Disks are zonal and attach to one node. A disk in zone 1 cannot attach to a node in zone 2, and a disk still attached to a failed node blocks reattachment. Check the PV node affinity, pod events, and the VMSS disk attachment.

```
kubectl describe pod <p> | grep -i -A3 'attach\|volume'
```

```
kubectl get pv <pv> -o jsonpath='{.spec.nodeAffinity}'
```

```
az disk show -g MC_aks-rg_prod_westeurope -n <disk> --query '[zones,managedBy,diskState]'
```

Interview tip: Use zone-redundant storage disks (ZRS) for stateful sets that must move across zones, and the WaitForFirstConsumer binding mode.

📚 Official docs

- Azure availability zones

certs.eknathalabs.com

Q152 A pod gets 403 Forbidden from Azure Key Vault even though the identity looks correct. What do you check?

Troubleshooting

🎙 Spoken answer · ~64 s

The 403 message includes the principal's object ID, so I first verify that it is the identity I expect, since pods sometimes pick up a different identity than intended. Then I check the vault's permission model: if RBAC is enabled, the identity needs a role such as Key Vault Secrets User at the vault or secret scope, and old-style access policies are ignored, whereas with access policies the identity must be listed there. Next, I look at the network side: if the vault firewall is set to selected networks or private endpoint only, the cluster's outbound IP or subnet must be allowed, and DNS must resolve to the private IP. I also check that the secret is not deleted or disabled and that role assignments have propagated, which can take a few minutes. With Workload Identity I verify the federated credential subject and service account annotation, and with the CSI driver the SecretProviderClass settings.

Likely follow-ups

- RBAC versus access policies in Key Vault?
- How does the Secrets Store CSI driver authenticate?

Confirm which authorisation model the vault uses (RBAC or access policies) and that the identity has the right role, then check the vault firewall and private endpoint rules, soft-delete state of the secret, and that the token is for the correct identity and audience.

```
az keyvault show -n kv-prod --query
'{rbac:properties.enableRbacAuthorization,fw:properties.networkAcls}'
```

```
az role assignment list --assignee <client-id> --scope $(az keyvault show -n kv-prod --
query id -o tsv)
```

```
kubectl logs <p> -c azwi-proxy
```

Interview tip: The error message includes the caller's object ID. Compare it with the identity you expect.

📚 Official docs

- Azure Key Vault
- HashiCorp Vault documentation

certs.eknathalabs.com

Q153 An Azure SQL database sits at 100% CPU or DTU and the app slows down. How do you find the cause and respond?

Databases

🎙 Spoken answer · ~63 s

First I confirm what is saturated, whether CPU, data IO, log IO or workers, using the metrics and the resource utilisation views. Then I open Query Performance Insight or Query Store to see the top queries by CPU, duration and execution count, and check whether a recent deployment or plan change caused a regression. Common causes are missing or unused indexes, parameter sniffing, scans on large tables, chatty ORMs issuing thousands of small queries, and blocking or deadlocks. Quick relief is to scale the database up or enable read scale-out for reporting, which is done online with a short interruption. Then I fix the root cause: add or adjust indexes, rewrite queries, force a known good plan in Query Store, fix the N+1 pattern, and add caching. Automatic tuning can create and drop indexes, and I set up alerts on CPU percentage and long-running queries to catch it earlier next time.

Likely follow-ups

- What is parameter sniffing?
- What does Query Store record?

Use Query Performance Insight and Query Store to find top consuming queries, check missing indexes, plan regressions and blocking, then apply tuning. Scale up temporarily to relieve pressure while you fix the root cause.

```
SELECT TOP 10 qs.total_worker_time/qs.execution_count AS avg_cpu, st.text FROM
sys.dm_exec_query_stats qs CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st ORDER BY
avg_cpu DESC;
```

```
SELECT * FROM sys.dm_db_missing_index_details;
```

```
az sql db update -g data-rg -s sql-prod -n orders --service-objective S6
```

Interview tip: Scaling up is a stopgap and takes effect with a brief connection drop. Fix the query or index afterwards.

📚 Official docs

- Azure SQL documentation

certs.eknathalabs.com

Q154 A Cosmos DB container returns 429 errors under load. How do you fix throttling without overspending?

Databases

🎙 Spoken answer · ~61 s

A 429 status means the request rate exceeded the throughput available, measured in request units per second. I look at the normalised RU consumption metric split by partition key range. If a single partition is at one hundred percent while others are idle, it is a hot partition, and the fix is a better partition key or a synthetic key that spreads load. If all partitions are busy, the container needs more throughput, so I switch to autoscale, which scales between ten percent and the maximum, or raise the provisioned limit, while keeping an eye on cost. I also reduce RU consumption per operation: review indexing policy to exclude unused paths, make queries include the partition key, use point reads instead of queries, project fewer fields and use smaller documents. Clients should handle 429 with the SDK's retry-after logic, and I alert on throttled request counts.

Likely follow-ups

- What is a point read?
- How does autoscale billing work?

429 means requests exceed provisioned RU/s on a partition or the container. Check normalised RU consumption per partition to see whether it is a hot key or overall load. Fix with better partition key, autoscale, indexing policy changes, query tuning and retry handling.

```
az cosmosdb sql container throughput update -g data-rg -a cosmos-prod -d appdb -n orders --max-throughput 20000
```

```
az monitor metrics list --resource <cosmos-id> --metric NormalizedRUConsumption --dimension PartitionKeyRangeId
```

```
SELECT * FROM c WHERE c.tenantId = @t
```

Interview tip: Every query should include the partition key. Cross-partition fan-out queries burn RUs quickly.

📚 Official docs

- RFC 6585: HTTP 429 Too Many Requests
- Azure Cosmos DB
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q155 Applications get 503 ServerBusy or throttling errors from an Azure Storage account. What do you do?

Storage

🎙 Spoken answer · ~62 s

Each storage account and each partition has limits on requests per second and bandwidth. When we exceed them, the service responds with 503 server busy or 500 operation timeout. I look at the transaction metrics split by response type and API name to see which operations are throttled, and whether the limit hit was account-wide or on a particular blob prefix. Remedies include spreading load by using more diverse blob names, since the service partitions by name range, adding exponential backoff retries with the SDK, caching or putting Azure CDN or Front Door in front of read-heavy content, moving hot data to a premium block blob account for lower latency and higher transaction rates, and splitting workloads across multiple storage accounts. I also check for chatty clients making many small requests that could be batched. A request to Microsoft for higher account limits is possible for some cases.

Likely follow-ups

- Why do timestamp prefixes cause hot spots?
- When would you use premium block blobs?

The account or a partition hit scalability targets for requests or bandwidth. Check metrics for throttled transactions and ingress or egress, spread load across blobs and prefixes, add retry with backoff, use premium tiers or multiple accounts, and add a CDN for reads.

```
az monitor metrics list --resource <storage-id> --metric Transactions --dimension ResponseType --interval PT1M
```

```
az storage account show -n stapp --query '{sku:sku.name,kind:kind}'
```

```
az storage blob list --account-name stapp -c data --prefix 2026/ --num-results 5
```

Interview tip: Sequential naming such as timestamps in prefixes can concentrate load on a partition. Hash or reverse the prefix.

📚 Official docs

- RFC 9110: HTTP Semantics (status codes, methods)
- Azure Storage introduction
- Microsoft: Cloud design pattern – Throttling

certs.eknathalabs.com

Q156 Log Analytics ingestion suddenly spiked and the bill jumped. How do you find the source and bring it down?

Monitoring

🎙 Spoken answer · ~62 s

I begin with the Usage table, grouped by data type and day, to see which table grew and when. Then I drill down using the billed size column grouped by resource ID, namespace or computer to find the producer. Typical culprits are container stdout logs from a chatty or crash looping pod, an application with debug logging turned on, security events set to collect everything, diagnostic settings sending all categories from a busy resource, or a retry storm. Fixes depend on the source: reduce log levels, fix the loop, use data collection rules to filter or drop noisy records before ingestion, switch verbose tables to the Basic plan, shorten retention, or send low-value data to storage instead. A daily cap can protect the budget but causes data loss when reached, so I alert at lower thresholds first. I add an ingestion anomaly alert and review the top tables monthly.

Likely follow-ups

- What does the daily cap risk?
- How do data collection rules reduce cost?

Query the Usage table by data type, resource and solution to find what grew. Typical causes are verbose container logs, debug logging, diagnostic settings sending all categories, or a loop. Filter at the source, change table plans and set a daily cap.

```
Usage | where TimeGenerated > ago(7d) | summarize GB = sum(Quantity)/1000 by DataType, bin(TimeGenerated, 1d) | order by GB desc
```

```
_BilledSize | summarize sum(_BilledSize) by _ResourceId | top 10 by sum_BilledSize
```

```
az monitor log-analytics workspace update -g mon-rg -n law-prod --quota 50
```

Interview tip: The built-in Usage and Billing workbooks in Log Analytics show ingestion by table and resource without writing queries.

📚 Official docs

- Azure Monitor

certs.eknathalabs.com

Q157 A Conditional Access policy locked administrators out of the tenant. How do you recover, and how do you prevent it?

IAM & EntraID

🎙 **Spoken answer · ~60 s**

If a policy blocks all admins, for example by requiring a compliant device nobody has, recovery depends on having planned for it. Microsoft recommends at least two emergency access accounts that are cloud-only, use long random passwords or FIDO2 keys stored securely offline, are excluded from every Conditional Access policy, and have permanent Global Administrator rights. I sign in with one, review the sign-in logs to identify the policy and fix or disable it, then re-enable once verified. If no such account exists, recovery goes through Microsoft support, which is slow. To prevent the problem I always create new policies in report-only mode first and review the impact in the insights workbook, use the What If tool, roll out to a pilot group, avoid targeting all users and all apps together without exclusions, require change review, and alert on any use of break-glass accounts.

Likely follow-ups

- What is report-only mode?
- Why must break-glass accounts avoid MFA dependencies on one method?

Use emergency access (break-glass) accounts excluded from all Conditional Access policies, protected with strong credentials and monitored. Prevention is report-only mode first, exclusions, staged rollout and What If testing.

```
az ad user show --id breakglass1@contoso.onmicrosoft.com
```

```
az monitor activity-log list --offset 1h --query "[?
contains(operationName.value,'conditionalAccess')]"
```

```
SigninLogs | where ConditionalAccessStatus == 'failure' | summarize count() by
UserPrincipalName, tostring(ConditionalAccessPolicies)
```

Interview tip: Alert on any sign-in by a break-glass account. It should happen almost never.

📚 **Official docs**

- Microsoft Entra: Conditional Access

certs.eknathalabs.com

Q158 A pipeline suddenly fails because a service principal client secret expired. How do you fix it and remove the risk permanently?

CI/CD

🎙 Spoken answer · ~56 s

The immediate fix is to create a new client secret for the app registration, store it in the secure place, update the service connection or variable, and re-run the pipeline. Using the append option avoids breaking consumers still on the old secret. Then I address the cause: secrets expire by design, and teams forget. I migrate service connections to workload identity federation, or to managed identities for self-hosted agents, so there is no secret at all. For any remaining secrets or certificates, I inventory them with a script or Microsoft Graph, record owners, set the shortest sensible lifetimes with automated rotation, and create alerts on expiry dates thirty and seven days ahead, using Key Vault expiry events or Azure Monitor queries. I also add ownership requirements to service principal creation and review them quarterly.

Likely follow-ups

- How do you rotate without downtime?
- How can Key Vault events help?

Short term, create a new secret and update the service connection. Long term, move to workload identity federation or managed identities so there is no secret to expire, and alert on credential expiry for anything that remains.

```
az ad app credential list --id <app-id> --query '[].{id:keyId,end:endDateTime}'
```

```
az ad app credential reset --id <app-id> --years 1 --append
```

```
az devops service-endpoint list --query "[?authorization.scheme=='ServicePrincipal'].name"
```

Interview tip: Ask who owns each service principal. Orphaned identities with no owner are the root of most expiry incidents.

📚 Official docs

- Azure: Managed identities

certs.eknathalabs.com

Q159 A self-hosted Azure DevOps agent pool is offline and pipelines are queued. How do you troubleshoot and make agents resilient?

CI/CD

🎙 Spoken answer · ~62 s

First, I see whether the agent service is running on the machine and read the agent diagnostic logs. Common causes are the machine being down or out of disk space because of accumulated build workspaces and Docker layers, expired or revoked authentication tokens, an outbound proxy or firewall change blocking access to Azure DevOps endpoints, a version update that failed, or the pool having insufficient parallel jobs, which shows as queued rather than offline. I restart the service, free disk, renew credentials, or re-register the agent as needed. For resilience, I stop treating agents like pets: I use virtual machine scale set agent pools or agents running in Kubernetes, which are created from a clean image and removed after use, configured with minimum idle agents for responsiveness. I also add monitoring on queue time and agent count, and use Microsoft-hosted agents as a fallback where network access allows.

Likely follow-ups

- What is a scale set agent pool?
- How do you control parallel job limits?

Check agent service status and logs, network access to dev.azure.com, expired PAT or authentication, disk space, and capacity limits. Make agents ephemeral using scale sets or Kubernetes so unhealthy ones are replaced automatically.

```
sudo ./svc.sh status
```

```
tail -n 50 _diag/Agent_*.log
```

```
az pipelines agent list --pool-id 12 -o table
```

```
df -h /home/vsts
```

Interview tip: Scale set agents are recreated per job or on health failure, which removes drift and the usual full-disk problem.

📚 Official docs

- Azure availability zones
- Azure DevOps documentation

certs.eknathalabs.com

Q160 An App Service slot swap fails or sends users to a broken app. How do you recover and prevent it?

CI/CD

🎙 **Spoken answer · ~62 s**

If production is broken after a swap, the quickest recovery is to swap again, which puts the previous version back since it is still running in the other slot. Then I find out what went wrong. The swap warm-up may have failed because the application did not respond on the configured path or took too long to start, which can leave the target unwarmed. Settings might not have been marked as slot-specific, so staging values for connection strings or feature flags moved to production, or production settings did not apply. A database migration may have changed the schema in a way that old code cannot handle, which makes rollback unsafe. Prevention includes using swap with preview to test the staging slot with production settings first, configuring health checks and warm-up paths, marking sticky settings carefully, making migrations backward compatible, and running smoke tests automatically before and after the swap.

Likely follow-ups

- What does swap with preview do?
- How do you make database changes rollback-safe?

Swap back immediately to restore the previous version. Then check warm-up failures, slot-specific settings and connection strings, startup time, and incompatible database changes. Use swap with preview and health checks to catch issues before traffic moves.

```
az webapp deployment slot swap -g app-rg -n web --slot staging --target-slot production
```

```
az webapp config appsettings list -g app-rg -n web --slot staging
```

```
az webapp log tail -g app-rg -n web --slot staging
```

Interview tip: Backward-compatible database migrations are what make a swap rollback safe. Otherwise, the old code meets the new schema.

📚 **Start here (official docs)**

- Microsoft Azure documentation

certs.eknathalabs.com

Q161 A Bicep or ARM deployment fails halfway with RequestConflict or a resource lock error. How do you handle it?

CI/CD

🎙 Spoken answer · ~59 s

I start with the deployment operations, which show which resource failed and the exact error and correlation ID. A resource lock on the group or resource returns a clear error, and I either remove the lock temporarily with authorisation or adjust the template. RequestConflict commonly means another operation on the same resource is still in progress, or that I'm trying to change an immutable property. Soft-deleted resources such as Key Vaults or Cognitive Services keep the name reserved, so I purge or recover them. Missing dependency order can be solved with references or explicit dependsOn. Before deployment I run what-if to preview the changes, validate the template, lint it, and deploy through a pipeline using incremental mode. Deployment stacks can manage the lifecycle and prevent drift. I also retry transient failures and keep templates idempotent so a rerun finishes the job.

Likely follow-ups

- Incremental versus complete mode?
- What are deployment stacks?

Read the deployment operations for the failing resource and its correlation ID. Locks, in-progress operations, soft-deleted resources with the same name, and dependencies are common causes. Use what-if before deploying and make templates idempotent.

```
az deployment group what-if -g app-rg --template-file main.bicep --parameters env=prod
```

```
az deployment operation group list -g app-rg -n main --query "[?
properties.provisioningState=='Failed']"
```

```
az lock list -g app-rg -o table
```

Interview tip: Incremental mode adds and updates but does not delete resources. Complete mode deletes anything not in the template, so use it with extreme care.

📚 Official docs

- Bicep overview
- Azure Resource Manager overview

certs.eknathalabs.com

Q162 Terraform on Azure fails with 'state blob is already locked'. What is going on and what is the safe fix? CI/CD

🎙 Spoken answer · ~65 s

Terraform's azurerm backend stores state in a blob and locks it by taking a lease, so two runs cannot change the same state at once. This error means a lease is held. Either another run is genuinely in progress, or a previous run was killed, for instance a cancelled pipeline or a lost connection, before releasing it. The safe approach is to first check the pipelines and people to be sure no apply is running, because breaking the lock during a real apply can corrupt state. Then I use terraform force-unlock with the lock ID shown in the error, or break the lease on the blob. After unlocking, I run a plan to verify state matches reality, and if a partial apply occurred, I let Terraform reconcile or import missing resources. To prevent it I use pipeline concurrency controls per environment, avoid cancelling apply jobs, and enable blob versioning and soft delete on the state container.

Likely follow-ups

- Why enable versioning on the state container?
- What happens if two applies run at once?

The azurerm backend uses a blob lease as a lock. A previous run crashed or a pipeline was cancelled, leaving the lease, or another run is genuinely active. Confirm no apply is running before breaking the lease or using force-unlock.

```
terraform force-unlock <LOCK_ID>
```

```
az storage blob show --account-name tfstate --container-name state --name prod.tfstate --query properties.lease
```

```
az storage blob lease break --account-name tfstate --container-name state --blob-name prod.tfstate
```

Interview tip: Serialise pipelines per state file with concurrency groups so two applies cannot collide in the first place.

📚 Start here (official docs)

- Microsoft Azure documentation

certs.eknathalabs.com

Q163 You cannot SSH or RDP to an Azure VM. What is your diagnostic order? Troubleshooting

🎙 Spoken answer · ~65 s

I move from outside in. First I make sure the VM is running and its health is fine, and look at the boot diagnostics screenshot and serial log for boot failures or a full disk. Then the network path: if I'm using a public IP, it exists and the NSG on the subnet and NIC allows my source IP on port 22 or 3389, and effective security rules and IP flow verify tell me exactly which rule blocks me. I check effective routes for a user-defined route sending traffic into a firewall that drops it, and whether just-in-time access needs a request. Inside the guest, the SSH daemon or Remote Desktop service may be stopped, the guest firewall may block, or credentials or keys may be wrong. Run Command, the serial console and the reset access extension help when the network path is fine but the guest is not. Bastion avoids many of these problems.

Likely follow-ups

- What does the serial console allow?
- How does IP flow verify work?

Check VM state and boot diagnostics, then network path: public IP or Bastion, NSG at subnet and NIC, effective routes, and guest firewall. Use IP flow verify, Run Command, serial console and the reset password or repair VM options.

```
az vm get-instance-view -g app-rg -n vm1 --query instanceView.statuses[].displayStatus
```

```
az network watcher test-ip-flow -g app-rg --vm vm1 --direction Inbound --protocol TCP --
local 10.0.1.4:22 --remote 203.0.113.5:50000
```

```
az vm run-command invoke -g app-rg -n vm1 --command-id RunShellScript --scripts 'systemctl
status sshd'
```

Interview tip: Boot diagnostics screenshot and the serial console work even when the network is completely broken.

📚 Start here (official docs)

- Microsoft Azure documentation

certs.eknathalabs.com

Q164 ExpressRoute or VPN connectivity works one way or flaps after adding Azure Firewall. What could be wrong?

Networking

🎙 **Spoken answer · ~60 s**

When a stateful firewall sits in the path, it must see both directions of a flow. Asymmetric routing happens when the request goes through the firewall but the reply returns directly via the gateway, or the other way round, so the firewall never sees the full session and drops packets. I diagnose with effective routes on the VM network interface and on the gateway subnet, and learned and advertised BGP routes. Typical fixes are adding user-defined routes on the spoke subnets for the on-premises prefixes pointing to the firewall, and on the gateway subnet for spoke prefixes pointing to the firewall, disabling BGP route propagation on spoke route tables, and ensuring more specific routes do not bypass the firewall. Packet captures, connection monitor and firewall logs confirm the path. For flapping, I check BGP session timers, tunnel health, and MTU or MSS issues.

Likely follow-ups

- Why does a stateful firewall drop asymmetric flows?
- What does BGP route propagation do?

Asymmetric routing: return traffic takes a different path than the request, so the stateful firewall drops it. Check user-defined routes, BGP route propagation, gateway transit settings, and make both directions traverse the same appliance.

```
az network nic show-effective-route-table -g app-rg -n nic1 -o table
```

```
az network vnet-gateway list-learned-routes -g net-rg -n er-gw -o table
```

```
az network route-table route create -g net-rg --route-table-name spoke-rt -n to-onprem --
address-prefix 10.200.0.0/16 --next-hop-type VirtualAppliance --next-hop-ip-address
10.0.1.4
```

Interview tip: Draw the packet path for the request and the reply. If they differ, a stateful device in the middle will drop one of them.

📚 **Official docs**

- Cloudflare Learning: What is a VPN?
- Azure Firewall
- Azure ExpressRoute

certs.eknathalabs.com

Q165 The AKS cluster autoscaler does not scale down idle nodes. How do you find out why?

Troubleshooting

🎙 Spoken answer · ~55 s

The autoscaler status ConfigMap and logs list why each candidate node was not removed. Reasons include pods that are not managed by a controller, pods using local storage or hostPath, pods with the safe-to-evict false annotation, a PodDisruptionBudget that would be violated, pods with strict affinity that cannot be placed elsewhere, system pods without a PDB, or node utilisation above the scale-down threshold of about fifty percent of requests. Timing settings also matter, since a node must be unneeded for around ten minutes after a scale-up before removal. I fix the cause by adding controllers, relaxing PDBs, moving local data to proper volumes, removing blocking annotations, and tuning profile settings such as scale-down utilisation threshold and delay after add. Over-requested resources make nodes look busy, so I also right-size pod requests.

Likely follow-ups

- How does utilisation threshold work?
- What is the safe-to-evict annotation?

The autoscaler skips nodes it considers unsafe to remove: pods with local storage, restrictive PDBs, pods without controllers, annotations that block eviction, kube-system pods, or nodes below the utilisation threshold or within the cool-down. Read its status and logs.

```
kubectl get cm cluster-autoscaler-status -n kube-system -o yaml
```

```
kubectl logs -n kube-system -l app=cluster-autoscaler | grep -i 'scale-down\|not removable'
```

```
kubectl get pdb -A
```

Interview tip: The annotation cluster-autoscaler.kubernetes.io/safe-to-evict=false pins a node, and forgotten annotations are a classic cause.

📚 Official docs

- Kubernetes Autoscaler: Cluster Autoscaler
- AKS: Cluster autoscaler
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q166 Spot node evictions are disrupting workloads on AKS. How do you design for them?

Compute

🎙 **Spoken answer · ~57 s**

Azure Spot VMs use spare capacity at a steep discount, but Azure can evict them with roughly thirty seconds of notice when it needs the capacity back or when price exceeds your maximum. On AKS, Spot node pools are tainted by default so only pods that tolerate it run there. I put fault-tolerant work on them: stateless workers, batch jobs, CI runners, and replicas beyond the minimum that on-demand pools already cover. I design for interruption by handling SIGTERM and shutting down quickly, keeping jobs idempotent with checkpointing or queues for retry, using PodDisruptionBudgets and topology spread across several Spot pools with different VM sizes and zones to reduce simultaneous eviction, and listening for scheduled event notices. The user-facing baseline stays on a regular pool. I monitor eviction rates and compare real savings against the added complexity.

Likely follow-ups

- Why use several Spot pools with different sizes?
- What is a Spot eviction policy?

Spot capacity can be evicted with about 30 seconds notice. Use Spot only for interruptible work, spread across pools and sizes, keep a regular on-demand pool for critical pods, handle SIGTERM, use PDBs, retries and checkpointing, and set a max price strategy.

```
az aks nodepool add -g aks-rg --cluster-name prod -n spot1 --priority Spot --eviction-policy Delete --spot-max-price -1 --enable-cluster-autoscaler --min-count 0 --max-count 20
```

```
kubectl taint nodes -l kubernetes.azure.com/scalesetpriority=spot kubernetes.azure.com/scalesetpriority=spot:NoSchedule
```

Interview tip: AKS taints Spot nodes by default, so workloads must opt in with a toleration. That keeps critical pods off by accident.

📚 **Official docs**

- Kubernetes: Node-pressure eviction
- AKS: Best practices
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q167 After adding a user-defined route to Azure Firewall for egress control, new AKS nodes fail to join or pull images. What is missing?

Troubleshooting

🎙 Spoken answer · ~60 s

With outbound type set to user-defined routing, all egress from the nodes goes to the firewall, so the firewall policy must allow what AKS needs to function. That includes the API server on port 443 and the tunnel connection, NTP, the Microsoft Container Registry and Azure management endpoints for image pulls, node image updates and monitoring, and OS package repositories. Missing any of these results in nodes stuck in NotReady, image pull failures or failed extensions. The simplest approach is an application rule using the AzureKubernetesService FQDN tag, which Microsoft keeps current, plus network rules for UDP 1194 or 9000 tunnel and NTP where applicable. I check firewall logs for denied requests from node IPs, and compare against Microsoft's documented egress requirements. I also add rules for my own registries and any monitoring endpoints, and test in a non-production cluster before rolling out egress restrictions.

Likely follow-ups

- What is an FQDN tag?
- How do you read denied traffic in firewall logs?

AKS nodes need outbound access to required endpoints such as the Microsoft Container Registry, the API server, Azure management and package repositories. A firewall that blocks them breaks bootstrap. Allow the AzureKubernetesService FQDN tag and required network rules.

```
az network firewall application-rule create -g net-rg -f fw -c aks-rules -n aks-fqdn --
source-addresses 10.1.0.0/16 --protocols http=80 https=443 --fqdn-tags
AzureKubernetesService --priority 100 --action Allow
```

```
az aks show -g aks-rg -n prod --query 'networkProfile.outboundType'
```

```
kubectl get nodes
```

Interview tip: Use the AzureKubernetesService FQDN tag so Microsoft maintains the required list for you.

📚 Official docs

- Azure Firewall
- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q168 A deployment fails with QuotaExceeded or 'not enough IP addresses in the subnet'. What are the fixes?

Troubleshooting

🎙 Spoken answer · ~64 s

For quota errors I list current usage against limits in the region and request an increase for the specific VM family or total regional vCPUs through the quota blade, as this can take time. As a workaround I deploy a different size or family, or use another region. For IP exhaustion the cause is subnet size: Azure reserves five addresses per subnet, and with traditional Azure CNI each pod consumes an address from the subnet, plus extra during upgrades, so large clusters drain a small subnet quickly. Remedies include moving to Azure CNI Overlay so pods use a private overlay range and only nodes use subnet addresses, using dynamic pod IP allocation with a separate pod subnet, adding larger subnets, or creating new node pools in a new subnet. Prevention comes from IP address planning at design time, tracking utilisation with alerts, and choosing a pod CIDR that does not overlap with on-premises networks.

Likely follow-ups

- Why does CNI Overlay save IPs?
- How many IPs does Azure reserve per subnet?

Check the regional vCPU quota and subnet free IPs. Request more quota, use a different size or region, or expand or add subnets. On AKS with Azure CNI each pod uses a VNet IP, so plan with CNI Overlay or dynamic IP allocation.

```
az vm list-usage -l westeurope -o table
```

```
az network vnet subnet show -g net-rg --vnet-name spoke1 -n aks --query
'{prefix:addressPrefix,used:ipConfigurations|length(@)}'
```

```
az aks update -g aks-rg -n prod --network-plugin-mode overlay
```

Interview tip: Azure reserves five addresses in every subnet, so a /24 gives 251 usable IPs.

📚 Official docs

- Azure Kubernetes Service (AKS)

certs.eknathalabs.com

Q169 A cost review finds orphaned disks, unused public IPs, old snapshots and idle gateways. How do you clean up safely and keep it clean?

Cost

🎙 Spoken answer · ~56 s

I use Azure Resource Graph to query across subscriptions for unattached managed disks, public IP addresses with no association, old snapshots, empty load balancers, idle VPN or application gateways, and stopped but not deallocated VMs, and combine the result with Advisor and cost analysis to prioritise by savings. Before deleting, I identify owners from tags or activity logs, notify them, and for data resources take a final snapshot or export. I tag candidates with a planned deletion date, wait for objections, and then delete in stages, starting with non-production. To keep things clean I enforce owner and expiry tags through Azure Policy, schedule recurring Resource Graph reports, automate shutdown of dev resources, use lifecycle rules on snapshots and backups, and run a monthly cost review. Dashboards by team make waste visible and create accountability.

Likely follow-ups

- What is Azure Resource Graph?
- Deallocated versus stopped VMs?

Find candidates with Resource Graph queries and Advisor, confirm ownership and last use, snapshot or tag before deleting, and delete in stages. Prevent recurrence with tagging policies, lifecycle automation and budget alerts.

```
az graph query -q "Resources | where type =~ 'microsoft.compute/disks' and
isempty(managedBy) | project name, resourceGroup, sku.name, properties.diskSizeGB"
```

```
az graph query -q "Resources | where type =~ 'microsoft.network/publicipaddresses' and
isnull(properties.ipConfiguration) | project name, resourceGroup"
```

```
az snapshot list --query "[?timeCreated<'2026-04-01'].name" -o tsv
```

Interview tip: Tag resources with a pending-delete date and owner first, wait a notice period, then remove. Nobody gets surprised.

📚 Start here (official docs)

- Microsoft Azure documentation

certs.eknathalabs.com

Q170 Monthly cost for Application Gateway, Firewall and NAT in the hub is far higher than expected. How do you analyse and reduce it?

Cost

🎙 Spoken answer · ~60 s

I use cost analysis grouped by meter to separate fixed deployment charges from data processing charges. Azure Firewall Premium, multiple Application Gateway instances with high minimum capacity, and several NAT or VPN gateways can quietly run into thousands per month. Reduction steps: delete idle or duplicate gateways, consolidate multiple application gateways using multi-site listeners, lower minimum autoscale counts, and choose the right SKU, for example Standard instead of Premium firewall if advanced inspection is not needed. For data processing, I find the top talkers with firewall logs and see whether bulk flows such as backups, replication or storage traffic can bypass inspection using private endpoints, service endpoints, or peering paths that don't need the firewall. I review regional data transfer and cross-zone traffic too. Reservations apply to some network services, and I document the security trade-offs of each change with the security team.

Likely follow-ups

- How do you find top talkers through the firewall?
- When is Firewall Premium justified?

Break the bill down by meter. Fixed hourly charges for gateways and firewalls plus per-GB processing dominate. Right-size SKUs, consolidate instances, remove idle ones, reduce hairpin traffic, use Private Link and service endpoints, and move to Basic or Standard SKUs where features allow.

```
az costmanagement query --type ActualCost --scope /subscriptions/<sub> --timeframe
MonthToDate --dataset-grouping name=MeterCategory type=Dimension
```

```
az network firewall show -g net-rg -n fw --query sku
```

```
AzureDiagnostics | where Category == 'AzureFirewallNetworkRule' | summarize
bytes=sum(todouble(BytesTransferred)) by SourceIp | top 10 by bytes
```

Interview tip: Check data processing meters. Large backups or replication traffic through a firewall can cost more than the firewall itself.

📚 Official docs

- RFC 3022: Traditional NAT
- Azure Application Gateway

certs.eknathalabs.com